



**CMOS DESIGN OF RECONFIGURABLE DATA-PANNING
ALGORITHM BASED SORTER FOR 8-INPUT, 8-BIT DIGITAL DATA
USING THE 0.35UM LIBRARY**

A Thesis
Presented to the
Graduate School of the
Gokongwei College of Engineering
De La Salle University

In Partial Fulfillment of the
Requirements for the Degree of
Master of Science in Electronics and Communications Engineering

by
TOLENTINO, Anthony S.

May, 2022



De La Salle University

THESIS APPROVAL SHEET

This thesis entitled:

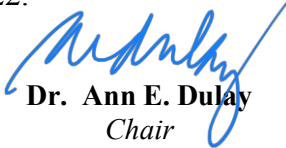
“CMOS Design of Reconfigurable Data-Panning Algorithm Based Sorter for 8-Input, 8-Bit Digital Data Using the 0.35um Library”

prepared and submitted by **Anthony S. Tolentino**, in partial fulfillment of the requirements for the degree of **Master of Science in Electronics and Communications Engineering** has been examined and is recommended for acceptance and approval for ORAL EXAMINATION.



Dr. Roderick Y. Yap
Adviser

Approved by the Committee on Oral Examination with a defense grade of passed on May 30, 2022.



Dr. Ann E. Dulay
Chair



Engr. Donabel D. Abuan
Member

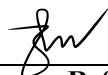


Engr. Jose Martin Z. Maningo
Member

Accepted in partial fulfillment of the requirements for the degree of **Master of Science in Electronics and Communications Engineering**.



Dr. Argel A. Bandala
Department Chair



Dr. Rosemary R. Seva
Acting Dean

Gokongwei College of Engineering



ACKNOWLEDGMENT

First of all, I would like to acknowledge the Almighty Creator for giving me the strength, wisdom, and patience in finishing this research and guiding me through day-to-day. You are the one who let me finish my degree and for that I am very thankful and keep on trusting you for my future.

I also would like to give my warmest thanks to my thesis adviser, Dr. Roderick Y. Yap for guiding me through this long journey. Thank you very much Sir for your patience and answering all of my queries even if it is already late in the evening. I really enjoy the process and the journey of completing this study. I also would like to acknowledge my thesis panel committee for making my defense experience an enjoyable moment, and for giving their brilliant comments and suggestions.

Finally, I would like to thank my wife, Keth, for giving me all the support during this journey. Your prayers for me are what made this research complete. This is for you and for our future family.



ABSTRACT

The aim of this study is to design a CMOS based data-panning algorithm sorter using Tanner Tools and LTSPICE software. Existing data sorter algorithms have circuit implementations with larger circuit components making the data sorting process more power consuming and had difficulties in hardware design implementation. It also aims to design a reconfigurable circuit that allows user to select a number of data to be simulated inside the data-panning based sorter.

The design was able to simulate 8 inputs and be sorted from lowest to highest simultaneously after 8 clock cycles. It has also a minimum hardware component compared to the other data sorter algorithms in terms of multiplexers, data registers, and magnitude comparators. It uses $2*N$ multiplexers, $N*(b+1)$ data registers, and $N/2$ magnitude comparators for N represents the number of inputs, while b represents the number of bits of the inputs. A total area of 1.35 mm^2 for the entire reconfigurable data-panning sorter circuit using Tanner Tools has been designed with a power consumption of 555 mW and a frequency of 20 μS .

Index Terms— data sorter, CMOS, data-panning algorithm, reconfigurable modification



TABLE OF CONTENTS

Thesis Approval Sheet	ii
Acknowledgment	iii
Abstract	iv
Table of Contents	v
List of Figures	viii
List of Tables	x
Abbreviations and Acronyms	xi
Notations	xii
Glossary	xiii
Listings	xiv
Chapter 1 INTRODUCTION	1
1.1 Background of the Study	2
1.2 Prior Studies	4
1.3 Problem Statement	6
1.4 Objectives and Deliverables	7
1.4.1 General Objectives	7
1.4.2 Specific Objectives	7
1.4.3 Expected Deliverables	8
1.5 Significance of the Study	9
1.5.1 Technical Benefit	9
1.5.2 Industrial Impact	9
1.5.3 Economical Welfare	10
1.6 Assumptions, Scope and Delimitations	10
1.6.1 Assumptions	11
1.6.2 Scope	11
1.6.3 Delimitations	11
1.7 Description and Methodology	11
1.8 Overview	13



Chapter 2 LITERATURE REVIEW	14
2.1 Existing Work	15
2.2 Lacking in the Approaches	25
2.3 Summary	27
Chapter 3 THEORETICAL CONSIDERATIONS	28
3.1 Logic Design	30
3.1.1 Input and Sorting Registers	30
3.1.2 Compare and Swap Circuit	33
3.1.3 Single Bit Circular Shift Register	36
3.1.4 Reconfigurable Circuit	37
3.2 CMOS Implementation	39
3.3 Tanner Tools	39
3.4 Spice Netlist	40
3.5 Summary	40
Chapter 4 DESIGN CONSIDERATIONS	41
4.1 Selection of Data Panning Algorithm Based Sorter	42
4.2 Complete Block Diagram of the Reconfigurable Data-Panning Algorithm Based Sorter	43
4.3 Logic Design	48
4.3.1 Data Sorting Registers	48
4.3.2 Compare and Swap Circuit	51
4.3.3 Single Bit Circular Shift Register	53
4.3.4 Reconfigurable Circuit	54
4.4 Transistor Design and Sizing	55
4.5 Tanner L-Edit Layout	58
4.6 Summary	59
Chapter 5 METHODOLOGY	60
5.1 Implementation	63
5.1.1 Selection Data-Panning Sorter	63
5.1.2 Design and Testing of Individual Components	63
5.1.3 Layout of Data-Panning Algorithm Sorter	64
5.1.4 Extraction of the Layout to Spice Netlist	64
5.1.5 Input/Output Node Naming	65
5.2 Evaluation	68
5.2.1 Comparison of Sorted Data	68
5.2.2 Evaluation using the Process Corner Library	68
5.2.3 Data Sorter Characteristics	69
5.3 Summary	69



Chapter 6 RESULTS AND DISCUSSIONS	70
6.1 Sorted Data	71
6.1.1 First Trial - 1 st Set of Data Values to be Sorted	72
6.1.2 Second Trial - 2 nd Set of Data Values to be Sorted	78
6.2 Simulation of the Reconfigurable Circuit	84
6.3 Geometric Layout	88
6.4 Process Corner Library Simulation	92
6.5 Size, Power Consumption, Frequency, and Temperature	95
6.5.1 Size	95
6.5.2 Power Consumption	96
6.5.3 Frequency	98
6.5.4 Temperature	98
6.6 Summary	100
Chapter 7 CONCLUSIONS, RECOMMENDATIONS, AND FUTURE DIRECTIVES	101
7.1 Concluding Remarks	102
7.2 Contributions	103
7.3 Recommendations	104
7.4 Future Prospects	104
References	105
Appendix A SPICE NETLIST	112
Appendix B ANSWERS TO QUESTIONS TO THIS THESIS	126
Appendix C VITA	132



LIST OF FIGURES

Figure 1- 1 Pedroni's 4-Input b-bit Data Panning Algorithm Sorter [1]	3
Figure 3- 1: Conceptual Framework of the Study	29
Figure 3- 2. Pedroni's 4 Input b-bit Data-Panning Algorithm Sorter [1], [2].....	30
Figure 3- 3. Input Storing Registers [72].....	31
Figure 3- 4. 1 bit, 2x1 Multiplexer [72].....	32
Figure 3- 5. Storage Register of the Data Sorter [72]	33
Figure 3- 6. Pedroni's Compare-and-Swap Circuit Implementation [1], [2].....	34
Figure 3- 7. Magnitude Comparator using N Bit Full Adder [74].....	35
Figure 3- 8. 8-bit Full Adder Circuit Diagram [73].....	36
Figure 3- 9. Full Adder Implementation using NAND gates[73]	36
Figure 3- 10. Pedroni's Single bit Circular Shift Register [2]	37
Figure 3- 11. Ring Counter with logic High as Circulated Output [73]	37
Figure 3- 12. Circuit Implementation of AND using NAND gate and inverter [72].....	38
Figure 4- 1. Process Flow Chart of the Study.....	45
Figure 4- 2. Complete Block Diagram of a 2 Input Reconfigurable Data Sorter	46
Figure 4- 3. Complete Block Diagram of an 8 Input Reconfigurable Data Sorter	47
Figure 4- 4. Logic Diagram of the Data Storing Register [72].....	49
Figure 4- 5. 8-bit 2x1 Multiplexer using NAND Gates [72]	50
Figure 4- 6. Master-Slave D-Type Flip Flop Circuit using NAND gates [73]	51
Figure 4- 7. Compare-and-Swap Circuit [72]	52
Figure 4- 8. Magnitude Comparator, showing $A > B$ [72]	53
Figure 4- 9. Single Bit Circular Shift Register [72].....	54
Figure 4- 10. Reconfigurable Circuit Using AND gates with similar Select Input [72] ..	55
Figure 4- 11. Inverter Diagram Using CMOS Transistors [75].....	56
Figure 4- 12. Nand Gate using CMOS Transistors [75]	57
Figure 4- 13. NOR Gate using CMOS Transistors [75]	58
Figure 4- 14. Sample Tanner Layout for Low Noise Amplifier MEA Pixel by Muller and Tanner [76].....	59



Figure 5- 1. Methodology Flowchart of the Study	62
Figure 6- 1. Graphs of Sorted Data of the 1 st place (Highest Value) up to the 4 th place Sorted Value for the 1 st Trial	74
Figure 6- 2. Graphs of Sorted Data of the 5 th place up to the 8 th (lowest value) place Sorted Value.....	76
Figure 6- 3. Graphs of Sorted Data of the 1 st place (Highest Value) up to the 4 th place Sorted Value for the 2 nd Trial.....	80
Figure 6- 4. Graphs of Sorted Data of the 4 th place up to the 8 th place (Lowest Value) Sorted Value for the 2 nd Trial.....	82
Figure 6- 5. Graph for Input A, for select line equal to logic high and low	84
Figure 6- 6. Graph for Input B, for select line equal to logic high and low.....	85
Figure 6- 7. Graph for Input C, for select line equal to logic high and low.....	85
Figure 6- 8. Graph for Input D, for select line equal to logic high and low	86
Figure 6- 9. Graph for Input E, for select line equal to logic high and low	86
Figure 6- 10. Graph for Input F, for select line equal to logic high and low	87
Figure 6- 11. Graph for Input G, for select line equal to logic high and low	87
Figure 6- 12. Graph for Input H, for select line equal to logic high and low	88
Figure 6- 13. Tanner Layout for Single Bit Circular Shift Register	89
Figure 6- 14. Tanner Layout for Data Registers	89
Figure 6- 15. Tanner Layout for Compare and Swap Circuit.....	90
Figure 6- 16. Tanner Layout for Reconfigurable Circuit.....	91
Figure 6- 17. Tanner Layout for the Final Data Sorter.....	91
Figure 6- 18. Graphical Representation for Typical-Typical Process Corner Library ...	92
Figure 6- 19. Graphical Representation for Fast-Fast Process Corner Library	93
Figure 6- 20. Graphical Representation for Slow-Fast Process Corner Library.....	93
Figure 6- 21. Graphical Representation for Fast-Slow Process Corner Library.....	94
Figure 6- 22. Graphical Representation for Slow-Slow Process Corner Library	94
Figure 6- 23. Block Diagram of Data sorter Circuit with external Resistor for solving Power Consumption.....	97
Figure 6- 24. Current Measurement using LTSpice and .print command	98
Figure 6- 25. Sorted Data of the Least Value, with temperature = 240 deg C	99



LIST OF TABLES

Table 1- 1. RELATED STUDIES ON SORTER ALGORITHM TECHNIQUES	5
TABLE 1- 2 : EXPECTED DELIVERABLES PER OBJECTIVE	8
TABLE 2- 1. DATA SORTER ALGORITHMS AND FEATURES	24
TABLE 2- 2.UNADDRESSED AREAS AND DEFICIENCIES	26
Table 4- 1. Comparison of the Performance of Data Sorting Algorithms. Upper part is resource usage; lower part: numeric example for b=8 bits and N=8 inputs and 24 inputs	42
TABLE 5- 1 .SUMMARY OF METHODS FOR REACHING THE OBJECTIVES	61
Table 5- 2. Summary of Input Naming Nodes.....	66
Table 5- 3. Summary of Output Naming Nodes	67
Table 6- 1. Expected Sorted Value per clock cycle of the data sorter (1 st Trial).....	73
Table 6- 2. Expected Sorted Value per clock cycle of the data sorter (2 nd Trial).....	79
Table 6- 3. Summary of Sizes of Circuits.....	95
Table 6-4. Summary Of Results For Achieving The Objectives	99



ABBREVIATIONS AND ACRONYMS

ATM	Automated Teller Machine.....	2
CAD	Computer Aided Drawing.....	21
CAS	Compare-and-Swap.....	17
CFAR	Constant False Alarm Rate.....	18
CMOS	Complementary Metal Oxide Semiconductor.....	4
EDA	Exploratory Data Analysis.....	4
FET	Field Effect Transistor.....	20
FF	Flip-Flop.....	4
FPGA	Field Programmable Gate Array.....	18
IC	Integrated Circuit.....	3
KSA	Kogge Stone Adder.....	22
LUT	Look-Up Table.....	18
MUX	Multiplexer.....	4
NMOS	N-Channel Metal Oxide Semiconductor.....	57
PMOS	P-Channel Metal Oxide Semiconductor.....	57
PWM	Pulse Width Modulation.....	23
VLSI	Very Large-Scale Integration.....	10



NOTATIONS

b	Number of bits of an input.....	5
u	Micro (10^{-6}).....	45
N	Number of Inputs	5
$\frac{w}{l}$	Channel aspect ratio.....	57
$vx0$	Least significant bit of the inputs	79
$vx7$	Most significant bit of the inputs.....	79
$v10$	Least significant bit of the HIGHEST Sorted Output.....	79
$v17$	Most significant bit of the HIGHEST Sorted Output.....	79
$v80$	Least significant bit of the LOWEST Sorted Output.....	79
$v87$	Most significant bit of the LOWEST Sorted Output.....	79

Throughout this document, mathematical notations conform to ISO 80000-2 standard, e.g., variable names are printed in italics, the only exception being acronyms like, e.g. *SNR*, which are printed in regular font. Constants are also set in regular font like j . Standard functions and operators are also set in regular font, e.g. in $\sin(\cdot)$, $\max\{\cdot\}$. Commonly used notations are t , f , $j = \sqrt{-1}$, n and $\exp(\cdot)$, which refer to the time variable, frequency variable, imaginary unit, n th variable, and exponential function, respectively.



GLOSSARY

Integrated Circuit	is defined as a microchip on which thousands and hundreds of electrical components, such as resistors, capacitors and transistors, are fabricated.
Panning Algorithm	An alternate movement of data from left to right
Reconfigurable	to rearrange the elements or settings of (a system, device, computer application, etc)
Sorter	A device or mechanism that arranges or group numbers with respect to specific purpose. In this paper, highest to lowest.
Spice Software	is a general-purpose, open-source analog electronic circuit simulator.
Tanner EDA	is a suite of tools for the design of integrated circuits.
transistor	is a semiconductor device used to amplify or switch electrical signals and power
VLSI (Very Large-Scale Integration)	refers to an integrated circuit technology with numerous devices on a single chip



LISTINGS

Spice	
Netlist	112



De La Salle University

Chapter 1

INTRODUCTION



1.1 Background of the Study

The emergence of technology in the world of electronics has been evident in the past years due to the need for convenience as well as having a better mode of communication. Data Sorting is one of the key factors for the improvement of certain systems like ATM switches access control, data distributors, and median as well as other order statistics filters for removal of impulse noise from images [1], [2]. As an addition, data sorting is also used in critical applications like database and searching. Each data sorting technique has different approaches and features that involves hardware implementations, complexity, and speed of the process [3]. Since data sorting is used, there are several available implementations and algorithms such as Systolic, Bitonic, Weave, and Insertion Sorters [1], [2]. These algorithms and implementations have different styles and composition with advantages depending on the application of the sorting method to be used. As per disadvantage, almost all of these implementations use a large amount of physical implementation in terms of the number of components inside the circuit which could affect the speed, size, as well as the power consumption of the circuit [1].

A data sorter presented by Pedroni is using data-panning principle for the sorting of digital data. The sorter uses data panning like movement to sort the data inserted at the input. This data sorting algorithm uses less components compared to other data sorting



algorithms and gives lesser power consumption as well as higher speed [1]. The circuit presented can be modified by inserting a reconfigurable system in which the user can select the number of inputs to be sorted. It could mean that lesser pins and components will be used for sorting making it faster in data processing [4], [5]. The circuit design by Pedroni can be found in Figure 1-1.

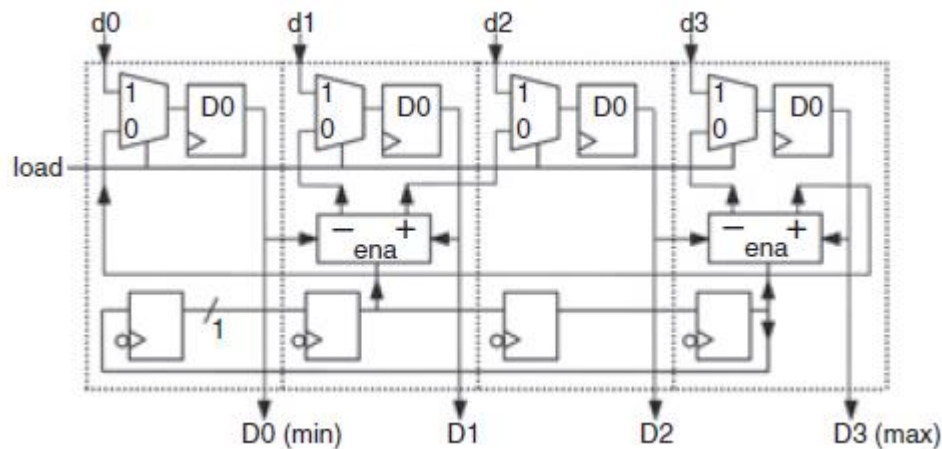


Figure 1- 1 Pedroni's 4-Input b-bit Data Panning Algorithm Sorter [1]

Full Custom Integrated Circuit Design is another emerging technology because it provides a simplification and minimization of the circuits used in the industry[6]. There are two types of IC Design namely: Semi Custom and Full Custom Design. Semi-Custom Design is usually a programming type of design in which the circuit is available except for the parameters and sizes that will be used for the entire circuit[7]. While in the Full Custom Design, the designer will provide a whole circuit usually in a transistor



level design and uses software like the Tanner EDA Tools for its circuit design[8]. The output of the circuit is dependent on the sizes of the transistors to be used as well as the circuit topologies of the whole design[9]–[11]. The whole system also depends on the CMOS Technology that will be used. The technology signifies the maximum sizes of the transistors in the design. Having a smaller size gives a smaller power consumption of the whole circuit although the trade-off is that the circuit is also slower compared to large sizes. The recent goal now of the IC Designers is to optimize and have both small power consumption and have a faster circuit[12]–[14].

Having the data panning sorter as well as the reconfigurable system and designed in the Full Custom Layout will be the main desire of the study. Application of the Full Custom Layout will be done in this study by determining the circuit needed and taking it in the transistor layout. The design and characterization of the whole circuit will be implemented in the Tanner EDA Tools Software and simulations will be performed to test and have a better overall efficiency of the reconfigurable data panning sorter.

1.2 Prior Studies

Table1.1 shows the comparison of different sorting algorithm circuits in terms of their number of hardware elements. Each algorithm has multiplexers, flip-flops,



magnitude comparators. As shown, the data-panning algorithm sorter provides the lowest hardware implementation.

The studies presented don't have an equivalent CMOS design which will be the main goal of the researcher in designing the data-panning based algorithm sorter. Since it has the least number of hardware components, it can also be minimized thru a design utilizing CMOS components, thus lowering more the power consumption of the entire data-panning algorithm circuit [9], [15]–[17] An elaborate discussion of the studies highlighting the differences can be found in Chapter 2.

Table 1- 1. RELATED STUDIES ON SORTER ALGORITHM TECHNIQUES

Sorter Type	Flip-Flops	Comparator 2xb	Muxes 2xb	Muxes 3x1	Clock Cycles	
Systolic	N(N+1)b	(N/2)(N-1)	2(N/2)(N-1)	-	1 (*)	
Weave	N(2b+1)	N/2 (**)	3N+1	N	N	
Insertion	2Nb	N	3N	-	N	
Panning	N(b+1)	N/2	2N	-	N	
(*) After pipeline is full (Latency = N) (**) More complex (requires control signals)						
Sorter	N	Flip-Flops	Comparator	Muxes	Muxes	Clock



Type			2xb	2xb	3x1	Cycles
Systolic	8	576	28	56	-	1(*)
	24	4800	276	552	-	1(*)
Weave	8	136	4 (**)	25	8	8
	24	408	12 (**)	73	24	24
Insertion	8	128	8	24	-	8
	24	384	12	72	-	24
Panning	8	72	4	16	-	8
	24	216	12	48	-	24

1.3 Problem Statement

Providing a low powered and fast system will be the main problem of the system. Since other data sorting algorithms consume a lot of circuit and components in their main architecture, the data panning sorter will be addressing the issue together with the insertion of the reconfigurable system. Currently, there are no CMOS designs of the circuit implementation of all the data sorting algorithms. Each circuit design has introduced the number of hardware requirement to implement the sorting process and among them, data panning sorting algorithm have provided the lowest hardware implementation, thus having a lower power and size requirement, as well as a faster



sorting process. Having a CMOS design for the data panning sorter will cater the issue of bigger circuit implementation for sorting methods and procedures.

1.4 Objectives and Deliverables

Below are the general objective and specific objectives, shown together with the expected output in Table 1-2.

1.4.1 General Objective (GO)

GO: To design CMOS Based reconfigurable data panning sorter using the 0.35um library

1.4.2 Specific Objectives (SOs)

- SO1: To design a CMOS Based Panning Sorter circuit that can accept up to 8 inputs
- SO2: To make the sorter reconfigurable by making the inputs variable from minimum of 2 to maximum of 8
- SO3: To design the Geometric Layout of the Sorter Circuit



- SO4: To evaluate the performance of the sorter circuit using each of the 5 corner libraries
- SO5: To characterize the circuit designed in terms of area, power consumption, temperature, and frequency

1.4.3 Expected Deliverables

Table 1-2 shows the outputs, products, results, achievements, gains, realizations, and/or yields of the Thesis.

TABLE 1-2 . EXPECTED DELIVERABLES PER OBJECTIVE

Objectives	Expected Deliverables
GO: To design CMOS Based reconfigurable data panning sorter using the 0.35um library	1. A CMOS Design layout and simulation of a Reconfigurable Data Panning Based Algorithm Digital Sorter
SO1: To design a CMOS Based Panning Sorter circuit that can accept up to 8 inputs	1. A CMOS Design with simulation of the data-panning based algorithm sorter. 2. A simulation using MS Excel of the expected output of the algorithm 3. A working simulation using LTSpice for design verification
SO2: To make the sorter reconfigurable by making the inputs variable from minimum of 2 to maximum of 8	1. Reconfigurable circuit that selects the number of inputs with design simulation and verification.
SO3: To design the Geometric Layout of the Sorter Circuit	1. Layout using Tanner Tools and following the 0.35um library for future fabrication procedures.
SO4: To evaluate the performance of the sorter circuit using each of the 5 corner libraries	1. Differentiation and evaluation of the entire circuit using the five corner libraries namely: typical-typical, fast-fast, slow-slow, fast-slow, and slow-fast



SO5: To characterize the circuit designed in terms of area, power consumption, temperature and frequency

1. Graphs and computations of the area, power consumption, temperature, and frequency of the circuit implementation.

1.5 **Significance of the Study**

The main concern of the study is to minimize the data sorting process of certain applications using a minimal hardware requirement of the data-panning algorithm digital sorter. The effects will be significant to various aspects and the creation of this study will pave a way for the following areas

1.5.1 **Technical Benefit**

The previous design of the Data Panning Sorter will be added by a reconfigurable system which will contribute to the whole-body knowledge in the CMOS Design field. It could be a way of developing other technologies in the future since CMOS design is emerging.

1.5.2 **Industrial Impact**

The design of the reconfigurable data sorter would be significant and beneficial to industries since many applications use data sorting techniques. The creation of



the proposed data sorter could lead to higher production in the VLSI field and be applied to different types of procedures and systems.

1.5.3 **Economical Welfare**

The effects of the proposed low powered data sorter can be beneficial economically since it uses minimal power due to its simplified design architecture. Having the data sorter presented could also be beneficial to different communication systems since it minimizes the delay time of communication.

1.6 **Assumptions, Scope, and Delimitations**

The main goal of this study is to design a reconfigurable data panning sorter in terms of the architectural design, logic design, circuit design, and physical design making it available for fabrication. The geometric layout will be created using Tanner LEdit as well as the simulation of the design. The design will also consider a maximum number of 8 inputs with 8 bits (0-255) but can be reconfigured if lesser inputs will be used. Another consideration is that no actual fabrication will be done in the study. The output design will be limited up to simulation using the 5 corner libraries.



1.6.1 Assumptions

1. Voltage source value is equal to 3V;
2. Logic High corresponds to 3V, logic low corresponds to 0V

1.6.2 Scope

1. The data sorter can accept up to 8 inputs;
2. The sorter will have 8-bit inputs, from 0-255 in decimal, 00000000 – 11111111 in binary;
3. Tanner LEdit will be used for Geometric Layout, and LTSpice for Simulation.

1.6.3 Delimitations

1. No actual fabrication will be made;
2. The simulation is tested using the five corner libraries.

1.7 Description and Methodology



Figure 1-2 shows process flowchart of the methodology of this study. This thesis focuses on the data-panning based algorithm sorter with a reconfigurable feature and be implemented on a 0.35um CMOS design library. Each block of the diagram is discussed thoroughly in Chapter 5 of this paper.

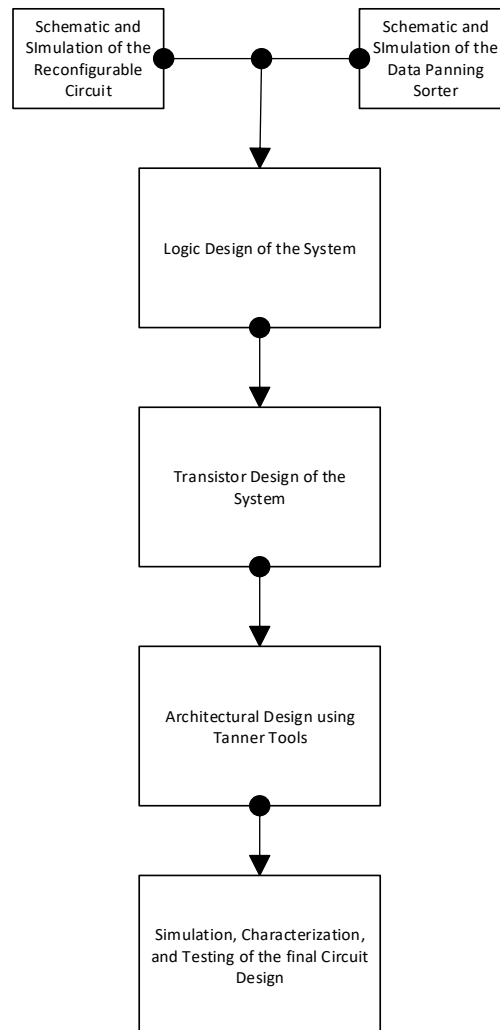


Fig 1-2. Process Flowchart of the Study



1.8 Overview

The thesis simulated the design of the CMOS based data-panning algorithm digital sorter with a reconfigurable feature and characterized through the five corner libraries. This study is ordered as follows. Chapter 2 discusses the methods and focus of prior related research to highlight the involvement of this study. Chapter 3 discovers theories relevant to solving the problem, followed by conversing specific techniques in Chapter 4. Chapter 5 presents the methodologies used in the application and assessment of the study. In Chapter 6, the results per objective are discussed and analyzed. Finally, Chapter 7 concludes the study and summarizes key findings and future directives for researchers.



Chapter 2

LITERATURE REVIEW



2.1 Existing Work

In [18], parallel sorting has been described as low hardware requirement implementation, and the sorting speed is dependent on the number of inputs inserted in the sorter. A massive data can be efficiently processed using parallel insertion and computing of digital data in applications that require intensive amount of data as presented in [19]. A comparison is well presented in [18] as well as in [20] in terms of parallel sorting techniques to achieve a lower hardware implementation and lower power consumption of the simulation. It consists of three components all of which are operating concurrently in systolic processing fashion namely: linear arrays that support constant-time operations, a multilevel sorting network, and a termination detection tree. In [2], different algorithms of data sorting have been used. Many includes systolic data sorter, weave sorter, insertion sorter, odd-even transposition sorter, batcher odd-even sorter, recursive odd-even sorter, and bitonic digital sorter. Each of these algorithms have different approaches including the number of components to used, complexity, and number of clock cycles [1].

A systolic sorter presented by [21] indicates different types of systolic sorter namely Batcher's bitonic sort, Stone's shuffle sort, and odd-even transposition sort on a mesh-connected array. The problem involved in these sorters are wiring spaces in between nodes, due to its hardware size requirement. A reconfiguration of the circuit is implemented to address the issue. Another testing of a systolic sorter is presented by



[22] by using error checking code to detect permanent and transient errors in the sorter. A modified systolic sorter is discussed by [23] which are amenable in VLSI applications. The features of the modified systolic sorter include simple control patterns of the algorithm, regular, repetitive and near-neighbor type of interconnection among comparators and the systolic data movement. These minimize the hardware implementation of the systolic sorter with a hardware terminator to complete the sorting process. The systolic sorter allows the data to be sorted every clock cycle. It has the advantage of quickly having the data sorted but still has the larger number of components since it used registers every time the data is to be sorted. The data will be inserted inside a register and then into a compare-and-swap (CAS) circuit. After the data is sorted in the first clock cycle, it will be forwarded to another register. The large size of the circuit is due to the presence of the registers each and every time the data is being sorted. [1] Another algorithm related to data sorting is the weave data sorter. This type of sorter is smaller compared to the systolic sorter but still requires large number of components. It involves a control circuit in the CAS units which gives a more complex design. Each input will require their own registers, and a 3x1 MUX in which is composed of two 2x1 MUX. Each input also requires registers after the 3x1 MUX. The control circuit is inserted to the select lines of the multiplexers. [2]

In [24], a strategy for designing stable insertion data sorters is discussed. The design is optimized by using simultaneous comparators to minimize the hardware requirement for digital parallel sorting process. A linear sorter is presented using first-in first-out



(FIFO) scheme in [25] with a feature of discarding the previous or older datum to be replaced by the new sorted data. It is also implemented in an FPGA and features identical hardware components having minimal implementations for its circuit design. The insertion sorter has been applied in Constant False Alarm Rates (CFAR) algorithms as presented in [26] also in a First-In First-Out (FIFO) manner. In [27], the insertion sorter was used in biomedical applications, specifically Membrane Computing. The insertion type of data sorter is gaining an advantage in terms of its compactness compared to the previous data sorters presented. The only problem with this type of sorter is its time complexity since it is linearly operated compared to the other configurations. It involves $2N$ registers, as well as N number of CAS units [1]

In [28], an odd-even sorter is used for vector quantizer. The main advantage of this sorter is that it is lower in terms of hardware implementation but slower compared to a fully parallel algorithm sorter. It is a reconfigured version of the Bitonic and Batcher Sorter. The amount of the CAS units has been lessened in the data sorter algorithm. The division of the inputs was made in the compare and swap process and been sorted separately. Then, odd-even entries are sorted in the next stage. Another sorting process is implemented in the last stages of the Batcher Odd-Even Sorter to totally finish the data processing. The paper presented in [29] shows an improved Odd-Even Sorter using two design of FPGA-based architectures that uses even lesser hardware components compared to [28]. It uses lower number of Flip-Flops (FF) and Look-Up Tables (LUT).



In [3], modified versions of the odd-even sorter has been presented to have more options in data sorting algorithms and provide users the data sorter that they needed.

Another type of sorter is presented in [30] which is a Bitonic Sorter, a generalization of the Batcher's sorter. The algorithm is a 4 way decomposition instead of two way. The applications of the Bitonic sorter are for switching fabric and broadband networks. Different topologies of the Bitonic sorter are also presented in [31]. A comparison of the Batcher sorter and the Pipeline sorter is shown in [32] and shows that the Batcher sorter has minimal hardware implementation compared to the Pipeline. The Bitonic data sorter uses a merged ascending and descending mode of sequences as its algorithm. The number of CAS units in this algorithm is still relatively large which consumes a lot of components in the design of the digital data sorter to be made.

In [33], a parallel sorting algorithm is introduced and name Cubesort. It performs a number of repetitions and rounds to sort N data items which can also be used in general routing and sorting. In [34], parallel hardware merge sorter has been presented and shows that it has an improvement in area as well as speed scalability by the process of variable sorting. It also uses FPGA in sorting implementation. A linear sorter is familiarized in [35] that proposes an extended non-strict partially ordered set-based configurable linear sorter which covers the issue of having larger hardware implementation. The circuit presented is smaller compared to the previous data sorter algorithms. In [36], a modified sorter is used to achieve better efficiency and low power consumption. It aims to minimize the Look-Up Tables and registers in the previous



sorter algorithm hardware implementation and also uses field programmable gate array (FPGAs) for its implementation.

There are already a number of researches related to CMOS design of different circuits whether they are analog or digital types of circuits. The presence of this researches led to the composition of this study thru the designing of the circuit of a digital data sorter with a reconfigurable circuit into a CMOS design using a software tool named Tanner Tools. A study regarding CMOS design is presented in [37] that shows the regress of the sizing of CMOS circuits beginning from 1 μ m to 0.1 μ m. This is a significant improvement since it allows the designers in also minimizing the power requirement of the newly designed CMOS circuits. In [38], a study presented shows that the replacement of field effect transistors (FETs) into an equivalent CMOS-based carbon nanotube pass-transistor logic integrated circuits in minimizing the hardware implementations of FETs. Another design implemented by [39] that shows the effect of using CMOS design circuits in achieving lower costing for the circuit implementation. The design uses thin film transistors (TFTs) in the circuit implementation which reduces the manufacturing costs. In [40], CMOS circuits are used in cellular applications to modify the complexity as well as the power requirement of the communication systems and an attempt to minimize the costing of designing them. The transceiver architecture is modified to achieve the target power consumption and costing. Moreover, the circuit topology and circuit implementation were also altered. Another design of CMOS technology applied to communication systems is presented in [41]. A low noise



amplifier was designed using CMOS technology to minimize the sizing as well as the power requirement of the circuit. In [42], another application of CMOS technology was used in designing a low noise amplifier using a Compact Wu Folded configuration. The design is implemented in 0.35um technology and paired with an external inductor and reduces the size of the whole circuit of the low noise amplifier and minimizing its power requirement. An implementation of a CMOS based design was presented in [43] for hybrid memristors. This design utilized logic gates namely AND, OR, NAND, NOR, XOR, and XNOR with minimum hardware requirements. An additional 1-bit full adder was also included in the circuit. It has an improved speed and a lesser area in the CMOS circuit design. In another literature, [44] presented a CAD design to optimize the current flowing through a MOS transistor as well as its sizing. The circuit was simulated using the 0.5um library of CMOS. The circuit presented in [45] uses CMOS based transistors in communication circuits specifically in the Terahertz frequencies. The designs were created for low power and high speed applications. In [46], the implementation uses the 0.18um library to introduce a Ka-band true time delay circuit. It showed a minimal size for the circuit design with less than 0.03 mm² for its total area. A CMOS based low power voltage reference is designed in [47] using a 2 transistor Self-Cascode MOSFET (SCM) structure. The circuit has features of low power implementation, minimal area, and temperature compensation. In [48], another low voltage reference has been designed and implemented using CMOS technology. The study was made due to the demand of having a lower sizing of hardware components, leading to lower power consumption of



the circuits. It also uses 0.18um CMOS technology libraries for its implementation and provided lower cycle time making it faster compared to the previous circuit design and implementations. Another study is discussed in [49] that uses 0.18um CMOS technology in designing a low-dropout voltage regulator. The design has a size of 27um x 34 um area and achieved a 1.48uW power. The circuit design is used for wearable devices applications that requires low power consumption. In [50], the authors used CMOS design in creating a Kogge Stone Adder (KSA) in which it was implemented using Cadence Design Suite at GPDK 180nm technology. It was emphasized that the parallel adders that have been used are essential in the basic blocks of arithmetic logic units of microprocessor systems. The authors also discussed a comparative analysis with different size from 4-bit to 16-bit parameters. The design presented was also created from architectural design up to the logic design. Previous research presented in [51] also used Cadence but this time it is a different version of the software. It uses Cadence Virtuoso Schematic and Layout Editors. The study is a Translation Lookaside Buffer that was designed in CMOS 65nm technology. The study details every step of creating the CMOS design of the circuit starting from its architectural design then to its logic design, circuit design and finally to its physical design. The goal of the study is to minimize the total number of components of the circuit by using hardening in the main row elements of the common matrix in arrays of Content Addressable Memory (CAM) cells and RAM cells. The authors used a combinational logic for the creation of the coincidence logic of CAM by using masking and compensation. A study presented in



[52] is another research that deals with CMOS design and applications. The design focuses on the application of CMOS design to Communication systems specifically for Wideband Beamforming Applications. The authors used a 0.18 μ m technology for their design which is somehow related to this proposed study of a reconfigurable digital data sorter. The study is focused on designing an Adjustable Active CMOS True Time Delay Circuit. There is also a study by [53] in which a Buck-Boost Regulator was implemented in the CMOS field intended for Wireless Sensor Networks. It used 90nm technology and had been implemented using Pulse Width Modulation (PWM) schemes. Moreover, the design provided a larger amount of efficiency in terms of the power transfer from the input to the output. A study of a low power and low voltage design is presented in [54] that emphasizes the importance of having minimal power dissipation of electronics systems. The study showed that lowering the threshold voltage results to the increase of the power dissipation. In order for the power dissipation to be minimized, the authors used the subthreshold CMOS regime as well as the supply voltage and other CMOS technology parameters. In [10], the implementation of CMOS design in creating a 5-stage ring oscillator is the focus of the study. The purpose of the ring oscillator is to be an alternative use for the temperature sensors available in the market. Since most sensors that have been used in monitoring of temperatures are sensitive in terms of the maximum temperature absorbed, larger sizes of these sensors and systems are required for it to withstand. Multiple literatures have used Tanner Tools in their design and simulation [8], [55]–[59]. Each study has presented their own area for the layout design,



simulated and applied in their respective fields. Articles [60]–[66] used the 0.35 μ m CMOS library process technology in designing their respective electronic circuits. Each design has their own unique identity and met the conditions of lower size as well as having lower power consumption. A voltage supply with ranges of 3V–3.8V has been used by these circuits in their respective designs.

The usage of pass transistors is endorsed in [67] that serves as an alternative way to minimize the circuit implementation of any digital circuit. Instead of designing the traditional digital circuit, the pass transistor enables the design to be lower in hardware implementation without sacrificing the functionality of the circuit and increasing its efficiency. In [68], a 0.18 μ m CMOS based circuit has been designed and uses magnitude comparators with a low voltage supply and minimizing the effects of the power consumption with an additional decrease of sizing in the design layout.

Using NAND gates as the foundation components of digital circuits is presented in [17] and shows lower power consumption and circuit implementation. A paper stated that the usage of logic gates is designed to support the target of power consumption minimization. A removal of the direct current as well as lowered value of a leakage current is proposed in [69]. In [70], the design of a novel master-slave flipflop was introduced to compensate for the power consumption of the circuit. In [71], the author used pass transistor logic (PTL) in designing a flipflop. It minimizes the size of the circuit as well as its power consumption.



TABLE 2- 1. DATA SORTER ALGORITHMS AND FEATURES

Ref	Data Sorter Algorithm/ Hardware Application	CMOS Implementation	Highlights
[3]	Systolic Bitonic Weave Insertion Panning	None	A compiled discussion of all the existing data sorting algorithm, summarized form and discussion per data sorting algorithm
[3]	Merge	None	FPGA based, lower number of flip-flops used
[28]	Odd-Even	None	Parallel Sorting, fewer clock cycle implementation
[29]	Odd-Even Merged	None	2 FPGA based architecture, improved lower sizing
[30]	Bitonic	None	4-way Decomposition method, instead of 2-way
[31]	Bitonic	None	Design used for Assistance in Fault Diagnosis
[32]	Bitonic	None	Used optoelectronic recirculating algorithm
[33]	Parallel	None	Used Cubesort – a parallel algorithm for data sorting
[34]	Odd-Even Merged	None	2 FPGA based architecture, improved lower sizing
[35]	Linear	None	FPGA based architecture with linear sorting algorithm
[36]	Multi-Dimensional Sorting Algorithm (MDSA)	None	Uses Ultra-Low-Power Hardware Architecture (ULP)
[37]	Limit Scaling	Yes	Shows data in limitations of CMOS sizing
[38]	Pass transistor	Yes	Used CMOS in designing pass transistor for carbon nanotubes
[39]	Pseudo-CMOS	Yes	Used single v type thin film transistors
[40]	RF design	Yes	CMOS application for RF circuits
[41]	LN amplifier	Yes	CMOS in low noise amplifier design for microwave
[42]	LN amplifier	Yes	CMOS in low noise amplifier design for microwave using Wu folded Active Inductor
[43]	Memristor	Yes	Used LTspice in simulation of hybrid memristor
[44]	CAD	Yes	Optimized transistor sizing used in



	methodology		Analog CMOS design
[45]	Wireless Communication Application	Yes	CMOS in low noise amplifier design for microwave

2.2 Lacking in the Approaches

The presented literature contains different types of data sorting algorithms and shows distinct components from each algorithm. While these algorithms show minimal hardware, the least number comes from the data-panning algorithm. These algorithms lack CMOS implementation in their design which could lead to even a smaller hardware implementation and lower power consumption. The data-panning sorter presented also lacks a reconfigurable feature that allows user to have a freedom in selecting the number of inputs in the sorter.

There is a lack of research focus on the following areas:

1. Most of the data sorter algorithms have provided features that allows the user to have lower number of hardware implementations and minimal power consumption. Due to the current trend of technology that requires lower hardware, the data-panning algorithm have provided the lowest among all of the algorithms, though no further reconfiguration has been made to the data-panning algorithm.
2. The data-panning algorithm lacks the CMOS implementation, which has the potential to become even lower in terms of hardware implementation. Also, the presented data sorter algorithms have no CMOS representation in their respective designs.



3. No modification of the current data sorter algorithms have been presented to make the hardware configuration even lower. An insertion of a reconfigurable circuit is lacking in the CMOS designs to improve this feature.

TABLE 2- 2.UNADDRESSED AREAS AND DEFICIENCIES

Ref	Data Sorter Algorithm/ Application	What is Lacking?
[3]	Systolic Bitonic Weave Insertion Panning	NO CMOS Implementation Reconfigurability
[3]	Merge	NO CMOS Implementation Reconfigurability
[28]	Odd-Even	NO CMOS Implementation Reconfigurability
[29]	Odd-Even Merged	NO CMOS Implementation Reconfigurability
[30]	Bitonic	NO CMOS Implementation Reconfigurability
[31]	Bitonic	NO CMOS Implementation Reconfigurability
[32]	Bitonic	NO CMOS Implementation Reconfigurability
[33]	Parallel	NO CMOS Implementation Reconfigurability
[34]	Odd-Even Merged	NO CMOS Implementation Reconfigurability
[35]	Linear	NO CMOS Implementation Reconfigurability
[36]	Multi-Dimensional Sorting Algorithm (MDSA)	NO CMOS Implementation Reconfigurability



2.3 Summary

The data sorter algorithms presented in Table 2.1 have shown different techniques in data sorting having distinct styles, methods, and hardware implementation for each design. A lot of improvement can be made by implementing the data sorters in CMOS to achieve more efficiency, lesser hardware implementation, and lower power consumption. A CMOS implementation of the best data sorter algorithm which is the data-panning sorter will be beneficial in keeping up with the trend of having lower values of hardware implementation, with an additional reconfigurable circuit for the entire data-panning sorter implementation.



Chapter 3

THEORETICAL CONSIDERATIONS



This chapter discusses the concepts and principles that are used in formulating the methodology of the study. The first section discusses the Data-Panning Algorithm Sorter diagram as well as the sub-circuits that composes it. Each diagram is shown in the succeeding sections of this chapter. In designing a data panning sorter, the conceptual framework in Figure 3-1 below will be the basis. A reconfigurable system will be inserted in the whole system in which it modifies the number of inputs to be sorted from 2 inputs up to 8 inputs. The circuit design of the Data-Panning Algorithm Sorter is shown in Figure 3-2 as illustrated by Pedroni.

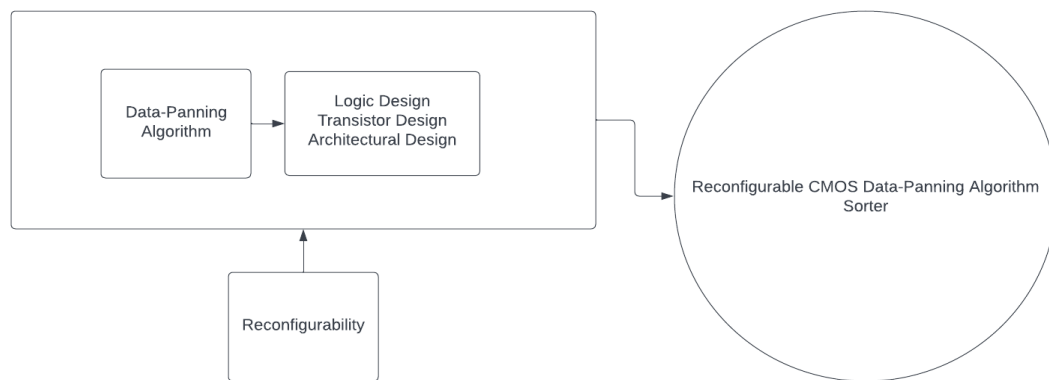


Figure 3- 1: Conceptual Framework of the Study

Data sorting has essentially been a key element in designing and implementing design and application systems. It includes applications such as filtering, ATM switches, and other important building blocks for electronic systems. A data sorter can be used in these said systems but will cover a number of considerations like: size of the circuit



component, speed, and power consumption. A number of data sorting algorithms have been considered based on the presented literatures but the Data-Panning Algorithm Based Data Sorter designed by Pedroni has been considered as the least number of hardware implementation [1], [2].

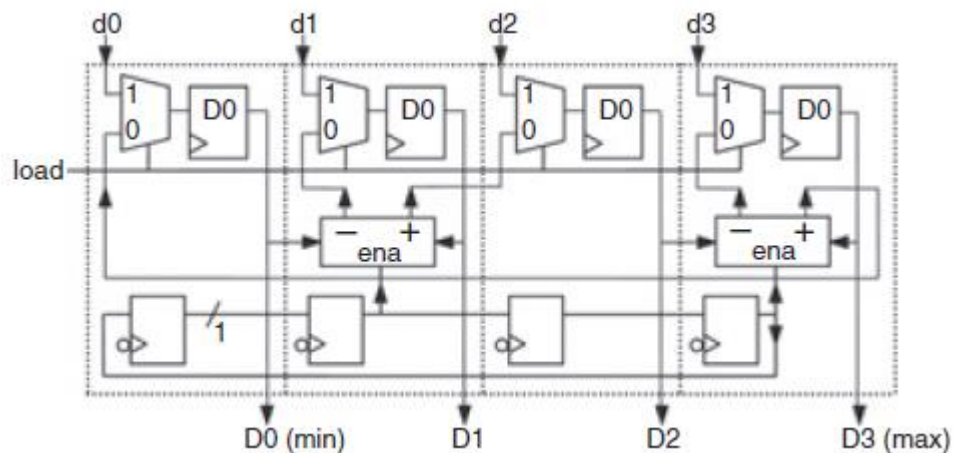


Figure 3- 2. Pedroni's 4 Input b-bit Data-Panning Algorithm Sorter [1], [2]

The design of the whole data sorter circuit is composed of four major sub-circuits namely: input and storing registers, compare-and-swap circuit, single bit circular enable register, and the reconfigurable circuit. Each sub-circuit has different approach and function to make the data sorter operate to perform the sorting algorithm of Pedroni's data panning sorter.

3.1 Logic Design

3.1.1 Input and Storing Registers



The input and storing registers act like the memory of the data sorter in the entire circuit algorithm[72]. It serves as storage for the initial inputs, then a temporary storage for the swapped data coming from the compare-and-swap circuit. It is composed of an 8-bit data multiplexer, and an 8-bit data register using D Flip-flops. The circuit implantation can be found in Figure 3-3.

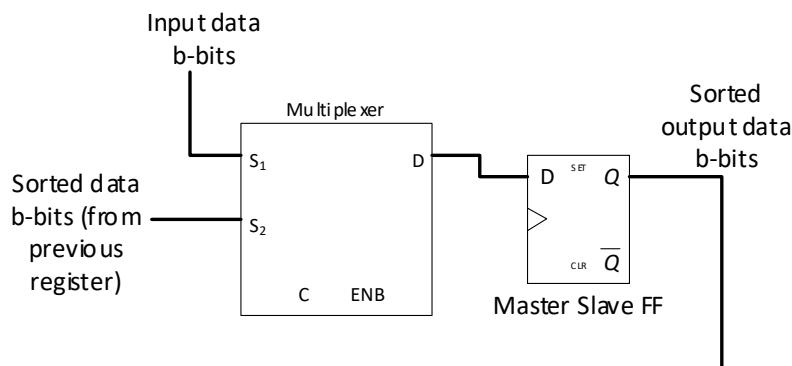


Figure 3- 3. Input Storing Registers [72]

The multiplexers act as the selector to the data sorter circuit whether the system intends to continue or distort the sorting process [73]. They are composed of n select lines and 2^b number of inputs. The output of the multiplexer is dependent on the values of the select lines. The multiplexers in the storing registers are 2x1 in size, having only one select line to be considered. The select line is an external signal that has a value of logic high in the first few clock cycles in order for the data register to store the input values. After 1 clock cycle, the voltage level at the load will be converted to logic zero, to enable the storage of the swapped values. If the load voltage is equal to voltage high,



it will forever store the input values and therefore, no swapping will be made. The logic low of the load voltage signifies that it will select the swapped values, either a minimum or maximum value coming from the compare-and-swap circuit. A sample of a 1-bit 2x1 Multiplexer can be seen in Figure 3-4. It is composed of 4 NAND Gates combined to form the multiplexer. NAND gates are used since they are considered universal and provides easier implementation in CMOS transistor designing phase[73]. The circuit will be combined to make it an 8bit 2x1 multiplexer, having the same node for the select line of the multiplexers.

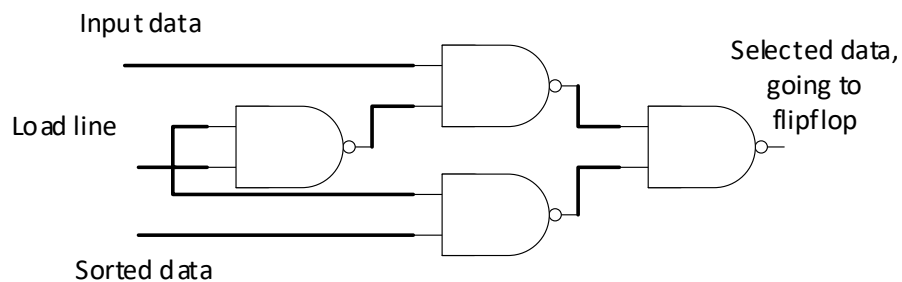


Figure 3- 4. 1 bit, 2x1 Multiplexer [72]

The storage registers are designed by using two D type Flip-Flops, one flip-flop is connected to an inverter for its clock input, making it an active low clock circuit. The output of the active low D flip-flop is connected to the input of the active high clock circuit. This circuit allows the data to be stored in the register of the data sorter. The



sample circuit diagram of the storage register can be found in Figure 3-5. The enable input is considered as the connection to the outside clock.

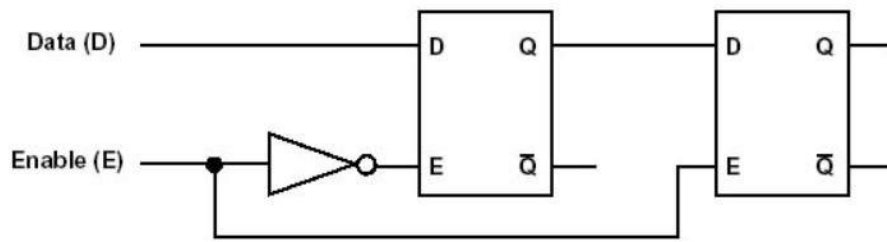


Figure 3- 5. Storage Register of the Data Sorter [72]

3.1.2 Compare and Swap Circuit

The compare-and-swap circuit can be considered as the main process of the data sorter. This is where the main function of the data sorter happens. The data are compared using a magnitude comparator and swapped depending on which is greater from the two data inputs, then stored in a data register similar to the storing register in the previous section. The complete circuit of the compare-and-swap can be found in Figure 3-6. Two multiplexers are connected to the data inputs and the select line is dependent on which of the data inputs is greater. An additional enable line is inserted to avoid the complexity of the bidirectional shift registers. The enable line is connected to an AND gate with the output of the magnitude comparator that will have a logic high if the first input is greater than the second input. If the first input is greater than the second input, then the select line will be equal to logic high, therefore, retaining the same order of the inputs. On the



other hand, if the first input is lower than the second input in terms of numerical value, the select line of the multiplexer will be logic low and swapping will occur in the multiplexers.

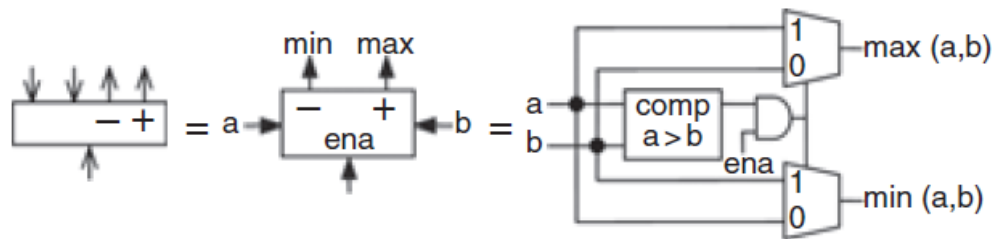


Figure 3- 6. Pedroni's Compare-and-Swap Circuit Implementation [1], [2]

A magnitude comparator is inside the compare-and-swap circuit to check which data input is higher or lower. The output of this magnitude comparator is connected with an enable voltage which in turn is connected to the single bit circular shift register. The magnitude comparator is composed of an 8-bit full adder connected to an 8 input NOR circuit. The only result considered is whether $A > B$. If this conditional statement is true, then the logic will be high. The other results $A < B$ and $A = B$ are not considered for the magnitude comparator circuit since they are not significant and also to minimize the circuit complexity, power consumption and size of the data sorter. The design of the magnitude comparator can be seen in Figure 3-7.

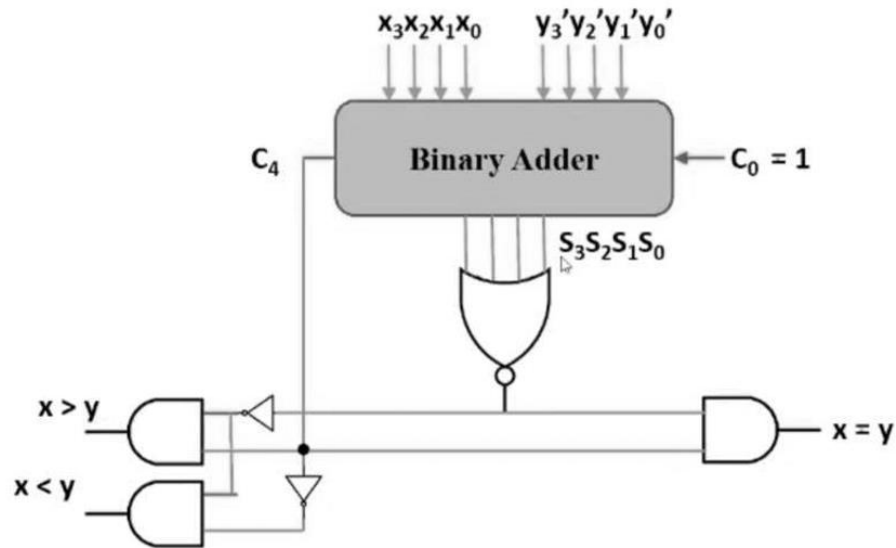


Figure 3- 7. Magnitude Comparator using N Bit Full Adder [74]

The 8-bit full adder's output is connected to an 8 input NOR gate, in which it is connected to an inverter and AND circuit. The equivalent circuit is transformed into a NOR circuit for the AND. Carry in value for the full adder is always equal to logic 1, so therefore it is always connected to the supply voltage. As for the carry out, the output is connected to the input of the AND circuit. Figure 3-8 shows the circuit of an 8 Bit Full adder while Figure 3-9 shows the Full adder implementation using NAND gates.

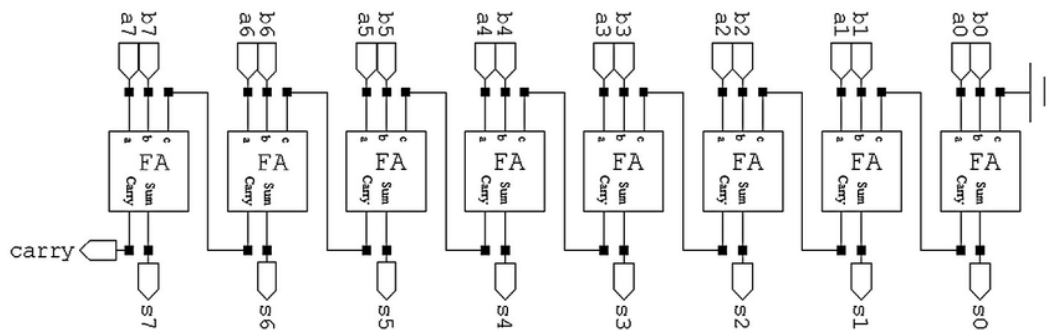


Figure 3- 8. 8-bit Full Adder Circuit Diagram [73]

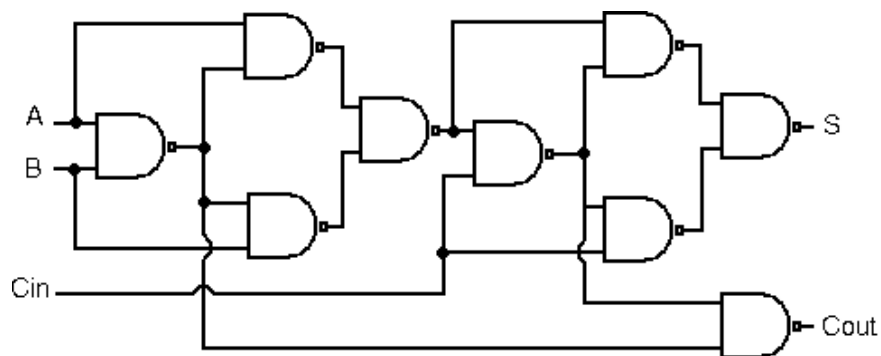


Figure 3- 9. Full Adder Implementation using NAND gates[73]

3.1.3 Single Bit Circular Shift Register

The presence of a single bit circular shift register is to avoid the complexity of bidirectional shift registers[2]. Every other clock cycle, one compare-and-swap (CAS) unit is turned off by the insertion of the enable node for the CAS units. The enable node is the output of the single bit circular shift register. Figure 3-10 shows the complete circuit of the Single Bit Circular Shift Register.

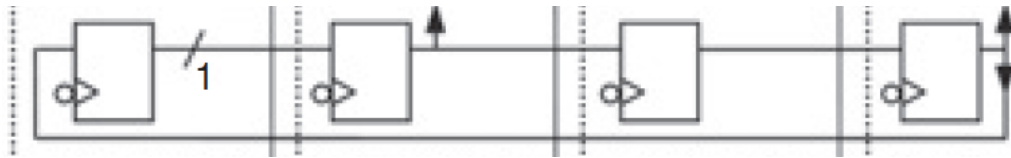


Figure 3- 10. Pedroni's Single bit Circular Shift Register [2]

Each register in the single bit circular shift register is a D type Flip-Flop with Preset and Reset inputs. The Preset of the first flip-flop is connected to the Reset of the remaining flip-flops and also connected to an external voltage signal which is equal to voltage zero since the flip flops are active low circuits. The detailed diagram of the flip flops is shown in Figure 3-11. The design is reflected as a ring counter which rotates the output logic high for each data register. The output is now connected to an inverter so that the value of the final output that is connected to the enable node of the compare-and-swap circuit is logic low.

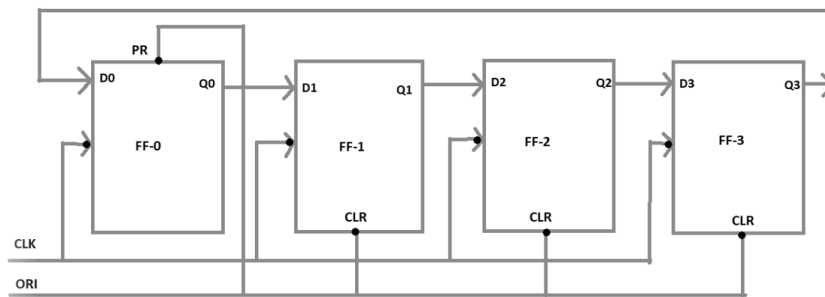


Figure 3- 11. Ring Counter with logic High as Circulated Output [73]

3.1.4 Reconfigurable Circuit



The reconfigurable circuit is an additional feature of the data sorter proposed by Pedroni. Providing a reconfigurable circuit will let the data sorter circuit select a number of inputs to be sorted, turning off the remaining data input registers. This would make the sorting process faster, as well as the power consumption also lower since the total number of active input pins are modified compared to which all of them are all working together in the sorting process. The reconfigurable circuit is composed of an 8 bit select lines in which they are connected with the Data Inputs in an AND circuit. If the select lines are logic low, it means, that the data inputs are off giving a decimal value of zero for that input (0000 0000). The selection of the reconfigurable circuit is due to the circuit simplification and implementation. By using basic AND gates for the system, the complexity has been reduced while maintaining the performance needed for the sorting process of the data sorter. Setting the other input to logic low, will turn off the AND gate and will not let the first input to pass by. Therefore, the second input for the AND gate is considered as the controlling parameter circuit for the reconfigurable circuit. The circuit implementation of an AND gate using NAND gate and an inverter is shown in Figure 3-12.



Figure 3- 12. Circuit Implementation of AND using NAND gate and inverter [72]



The design of the AND Gate circuit has been modified by using a combination of a NAND gate and an inverter. This is because of the usage of the universal gates which will be beneficial in designing the transistor logic of the entire circuit due to its minimal hardware implementation.

3.2 CMOS Implementation

The emergence of CMOS applications has been one of the considerations in implementing the data-panning based algorithm into its equivalent CMOS circuit. Together with the reconfigurable circuit, it will be implemented in a CMOS environment to further minimize the area of application of the circuit as well as its power consumption and frequency. Each circuit will be converted to its equivalent logic design, then transistor design, and finally using a Geometric Layout Tool which is Tanner Tools to create the architectural design of the data-panning algorithm sorter.

3.3 Tanner Tools

Learning a Computer-Aided-Design (CAD) Tool is another consideration for the architectural design of the reconfigurable data-panning algorithm-based sorter. It allows the completion of the CMOS design which can be readily available for future fabrication. As stated in the limitations, there would be no fabrication to be done. The



layout design will be evaluated based on the library presented, which is the 0.35um library. A lot of literature has been made using Tanner tools in designing their CMOS based circuits. After which will be extracted into their equivalent Spice Netlist.

3.4 Spice Netlist

The Spice Netlist tools are considered as aid for simulation of electric circuits. It uses certain sets of instructions to operate the circuit. Each netlist is patterned through naming of nodes for each component of an electronic circuit. A library must also be used for the operation of the circuit, specifically considering CMOS transistors. Waveforms are also shown after running each simulation to test the results.

3.5 Summary

Each discussion of the components of the Reconfigurable Data-Panning Algorithm Based Sorter, CMOS Implementation, Tanner Tools, and Spice Netlist will be essential for the development of the study. The minimization of hardware components and lower power requirement may be acquired through the combination of the said topics followed by a validation. The theories discussed are integrated with the next chapter's techniques to develop the methodologies.



Chapter 4

DESIGN CONSIDERATIONS



The previous chapter discusses the theories involved in this study. In this chapter, the design considerations can be applied to achieve the objectives stated in Chapter 1. It is divided into six parts – the first is a discussion of the selection of the sorter algorithm, followed by the complete schematic of the reconfigurable data-panning algorithm-based sorter, then the logic design, then the transistor design and sizing, the Tanner layout, and finally the extraction, simulation and testing of the Spice Netlist.

4.1 Selection of Data Panning Algorithm Based Sorter

The researcher intended to use the data panning algorithm of Pedroni since it provides a lower number of circuit implementation making it have a lesser power consumption, lesser complexity, and a simpler circuit design in the CMOS design tools to be used. As shown in Table 4-1, it follows the different sizes of each of the sorter algorithms presented.

Table 4- 1. Comparison of the Performance of Data Sorting Algorithms. Upper part is resource usage; lower part: numeric example for $b=8$ bits and $N=8$ inputs and 24 inputs

Sorter Type	Flip-Flops	Comparator $2 \times b$	Muxes $2 \times b$	Muxes 3×1	Clock Cycles
Systolic	$N(N+1)b$	$(N/2)(N-1)$	$2(N/2)(N-1)$	-	1 (*)
Weave	$N(2b+1)$	$N/2$ (**)	$3N+1$	N	N



Insertion	2Nb	N	3N	-	N	
Panning	N(b+1)	N/2	2N	-	N	
(*) After pipeline is full (Latency = N) (**) More complex (requires control signals)						
Sorter Type	N	Flip-Flops	Comparator 2xb	Muxes 2xb	Muxes 3x1	Clock Cycles
Systolic	8	576	28	56	-	1(*)
	24	4800	276	552	-	1(*)
Weave	8	136	4 (**)	25	8	8
	24	408	12 (**)	73	24	24
Insertion	8	128	8	24	-	8
	24	384	12	72	-	24
Panning	8	72	4	16	-	8
	24	216	12	48	-	24

4.2 Complete Block Diagram of the Reconfigurable Data-Panning Algorithm Based Sorter

To come up with the final output of the research study, the researcher will undergo these certain steps and methods. Each step will provide a better understanding for the development of the study. The schematic of both reconfigurable and data panning sorters will be simulated using any simulation tool software available. In this study, Multisim 7 and Logisim were the tools used for the simulation of the system. The logic design will be coming from the schematic and be analyzed piece by piece for the study. Then after, the logic design will be converted into an equivalent transistor design for the system and



will be implemented in the Architectural or Physical Design using Tanner Tools. The library that has been used is the 0.35 um library for the Layout of the Data-Panning Algorithm Based Sorter. After checking the design for any errors and violations based on the rule set of the library, it will now be extracted to an equivalent Spice Net List and be tested for its performance. The researcher has tested two sets of data in order to check the effectivity of the data sorter circuit. The Spice Net List will also be simulated with the four corner process libraries namely: typical-typical, fast-fast, slow-fast, fast-slow, and slow-slow. The circuit must pass and achieve a favorable performance for each of the process corner library.

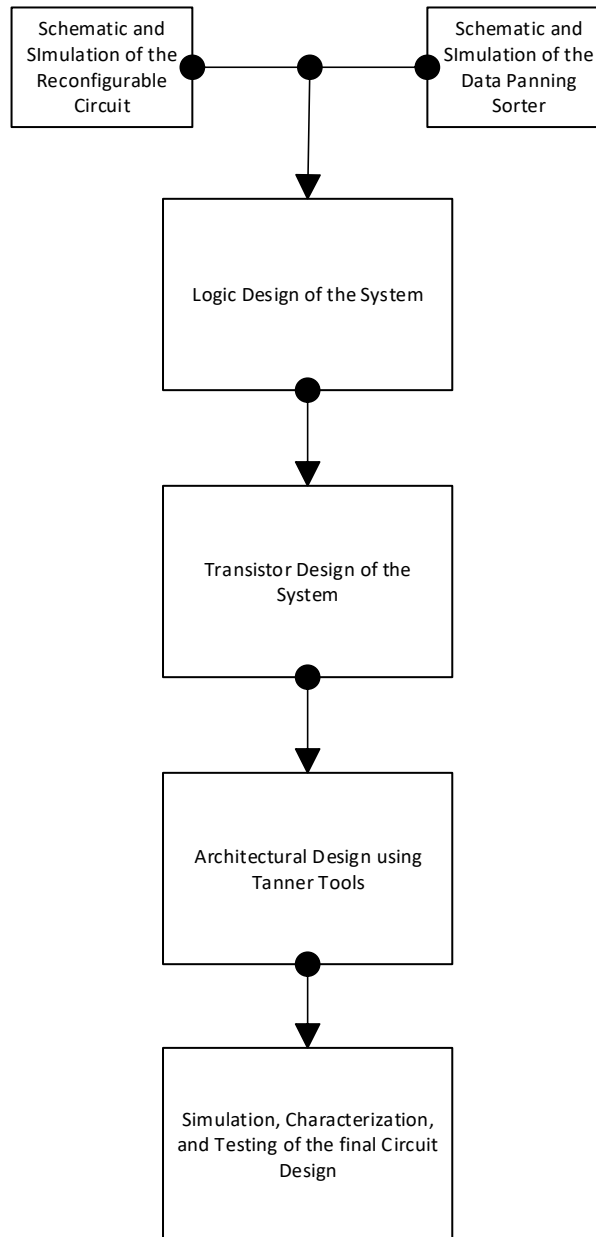


Figure 4- 1. Process Flow Chart of the Study.

The reconfigurable data sorter is composed of 4 major parts namely: input registers, compare-and-swap circuit, single bit circular shift register, and the multiple



AND reconfigurable circuit. Each block diagram will be discussed in this chapter with their functions and detailed circuit. The complete block diagram is shown in Figure 4-2 and Figure 4-3.

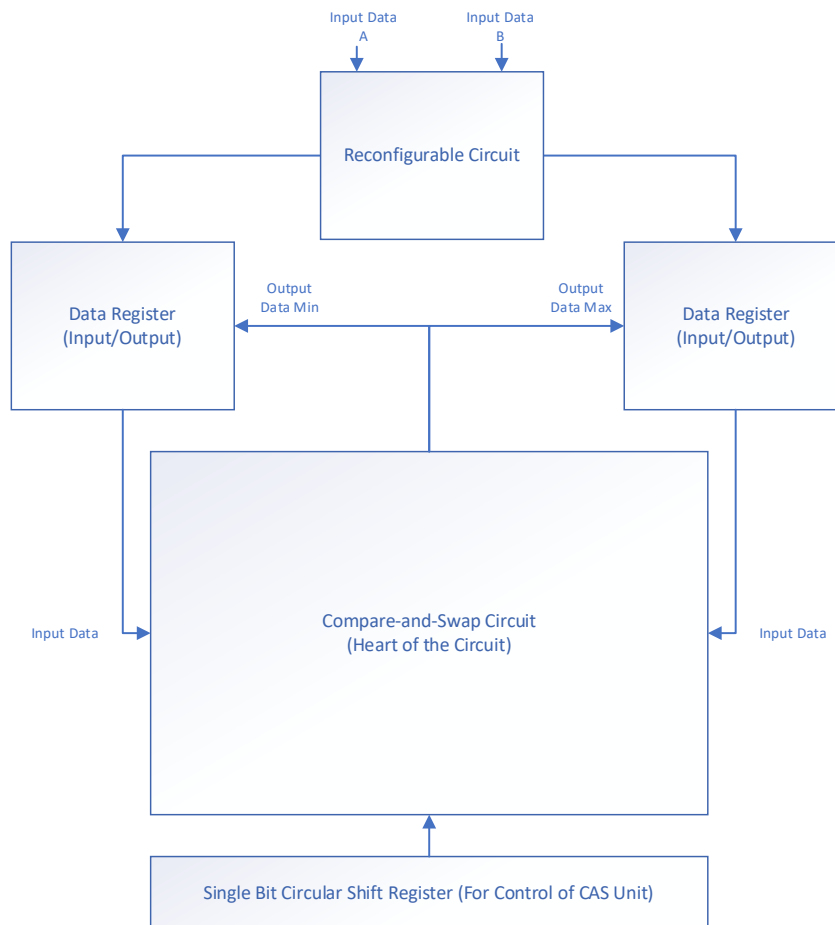


Figure 4- 2. Complete Block Diagram of a 2 Input Reconfigurable Data Sorter

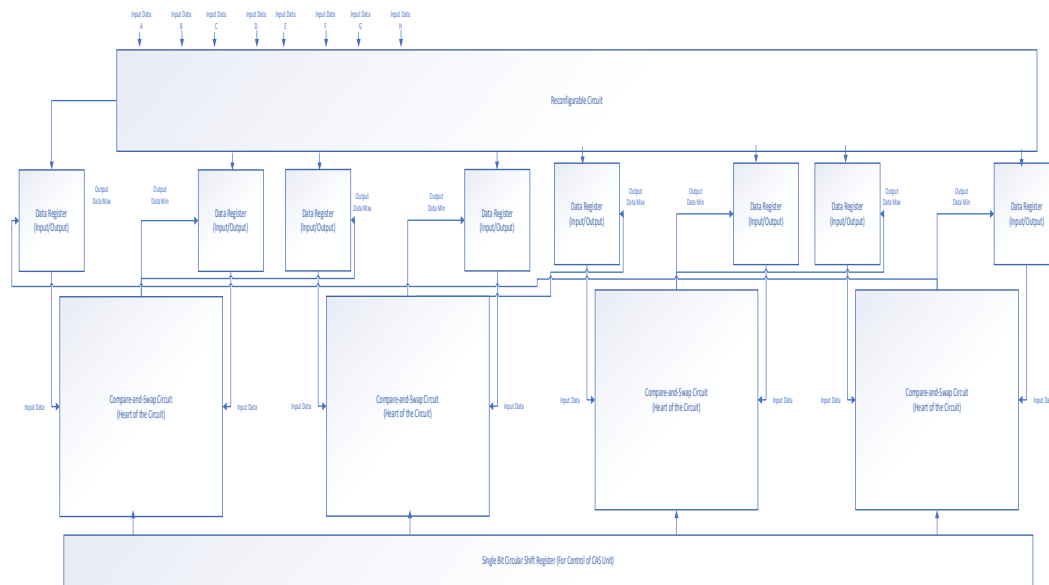


Figure 4- 3. Complete Block Diagram of an 8 Input Reconfigurable Data Sorter

The sorting process starts with the reconfigurable circuit and lets the user to select which data input will be turned on. Each data input has an equivalent reconfigurable circuit connected to the select lines. If the select line is logic low, then the data registers will not accept the input and will provide a value of zero for all of its bits. After the reconfigurable process, the data registers will now store the input data. The first clock cycle will store the inputs, which will be fed to the compare-and-swap circuit. A load voltage connected to the select line of the data registers' multiplexer will determine start of the sorting process. At the first clock cycle, it is set to logic high for the data register to store the initial inputs, then after the first clock cycle, it will continuously be equal to logic low to initiate the storing of the sorted data outputs. The compare-and-swap circuit will determine which of the two input values have a greater



value. The swapped data will now be shifted one position to the right and be stored inside the next set of data registers. The sorting process will continue for a total of 8 clock cycles. An additional single bit circular shift register is connected to the compare-and-swap circuit so that to avoid the complexity of bidirectional shift registers. One compare-and-swap circuit is turned off every two clock cycles, and its controller is the single bit circular shift register. The turning off is necessary to avoid the complexities and to follow the data sorting algorithm of the data-panning principle of the sorter. The outputs of the data registers will now show the sorted data from lowest to highest. A total of 8 clock cycles plus another cycle of delay (for storage of inputs) is needed to complete the data sorting process of the data-panning algorithm.

4.3 Logic Design

4.3.1 Data Storing Registers

The data storing registers are composed of an 2×1 N input Multiplexer and a Single Signal Synchronizer Using a Double D type Flip Flops in which stores the input signal as well as the sorted data. The Logic Diagram is shown in Figure 4-4. The input B is connected to the data inputs of the sorter while the Input A is connected to the shifted value of the entire data sorter. The initial selector voltage (load voltage) is set to logic high in order for the inputs to be stored, then after the clock delay expires, it will be set to logic low until the entire circuit turns off in order for the sorter circuit to begin



shifting values. The shifting is continuous as long as the load voltage is equal to logic low. If the load voltage is high, it will select INPUT A (S_2) which represents the input data of the sorter, else if load voltage is equal to logic low, then it will select INPUT B (S_1) which represents the shifted values.

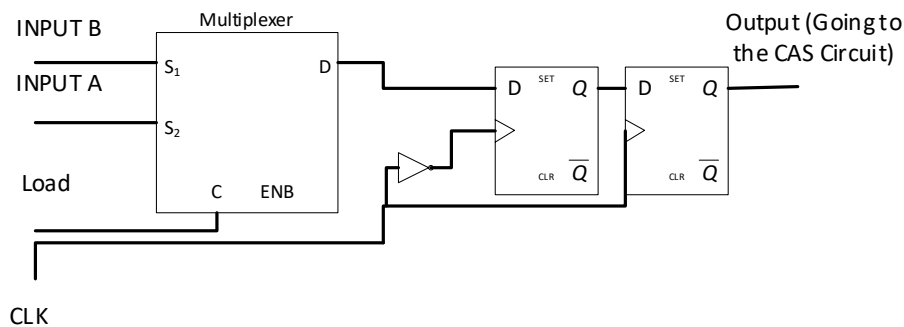


Figure 4- 4. Logic Diagram of the Data Storing Register [72]

4.3.1.1 Multiplexer

The data storing registers contains multiplexers that selects either the initial data inputs or the sorted value per compare-and-swap circuit. At the first clock cycle, the load selector is logic high to initially select the first inputs of the data sorter. After another clock cycle, the load selector is set to logic low so that it will continuously select the sorted value per each compare-and-swap circuit. The multiplexer circuit in this data sorter is an 8-bit 2x1 multiplexer and is designed using multiple NAND Gates as shown in Figure 4-5.

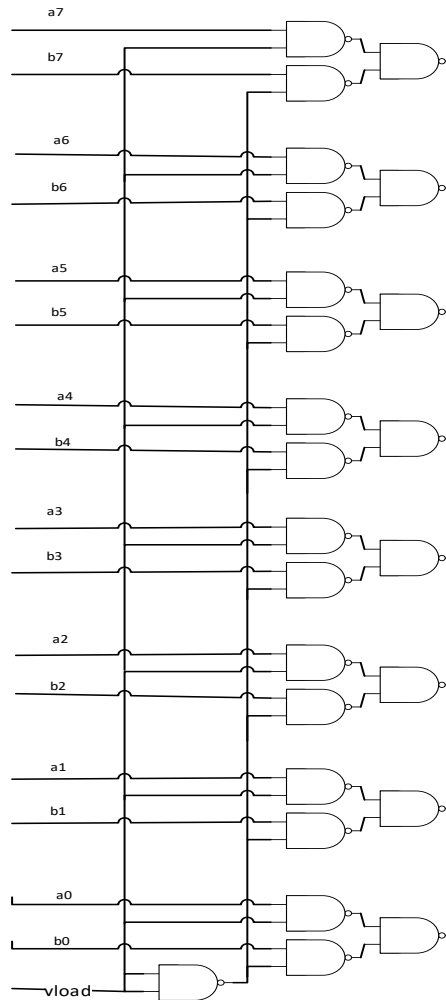


Figure 4- 5. 8-bit 2x1 Multiplexer using NAND Gates [72]

4.3.1.2 D-Type Flipflops

The D-type flip flops were designed using NAND gates to have a simpler layout since the NAND gate has been used by other components of the data sorter circuit. The designed D-type flip flops for the storing register is similar to the Master-Slave D-type Flip Flop Circuit. This is to acquire the value of the Q



from the D only when it is a complete clock cycle. The Master-Slave flip flop is composed of two D-type flipflop that allows only one of these two flipflops to be turned on making the value of the D passed to the Q after a complete clock cycle is applied to the clock terminal of the flipflops. The circuit of the storing register can be found in Figure 4-6.

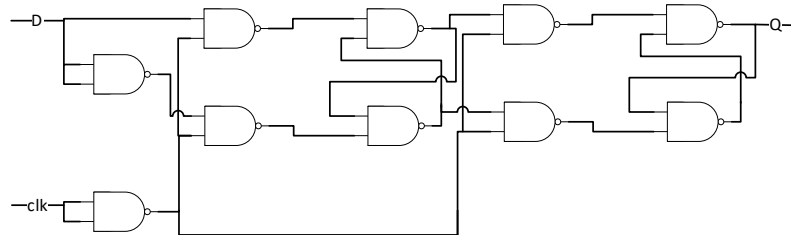


Figure 4- 6. Master-Slave D-Type Flip Flop Circuit using NAND gates [73]

4.3.2 Compare-and-Swap Circuit

The compare-and-swap circuit is composed of a Magnitude Comparator, and two Multiplexer, with an additional enable signal coming from the Single Bit Circular Shift Register. The block diagram can be found in Figure 4-7. If Input A is greater than Input B, then the value of the magnitude comparator will be equal to logic high. This will be the select signal of the multiplexers and will result in selecting the Input A in the MAX multiplexer while Input B to the MIN multiplexer. On the other hand, if Input A is lesser than B, the enable signal to the multiplexer will be equal to logic low, thus Input B will be stored in the MAX multiplexer while Input A will be stored in the MIN multiplexer.

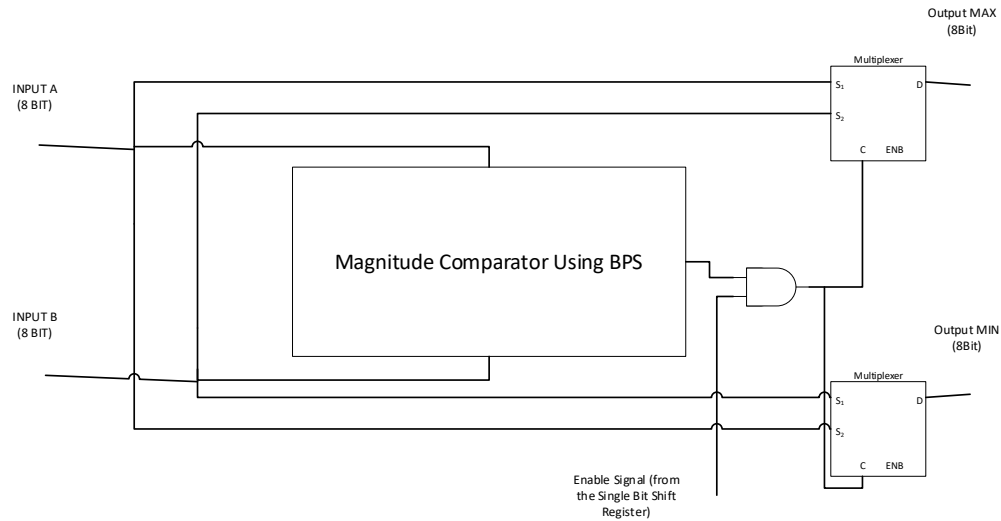


Figure 4- 7. Compare-and-Swap Circuit [72]

The magnitude comparator is a combination of an 8-bit full adder, with carry in set as logic high. It is basically an 8-bit Subtractor using 2's complement. The inputs are fed to the subtractor using a full adder and the complement of the subtrahend. The output sum of the full adder which is also considered as the difference of the subtractor is then connected to an 8-input NOR gate. After which, the output of the 8-input NOR gate is connected to the input of a 2-input NOR gate together with the Inverted output of the Carry Out signal of the Full Adder. The block diagram of the Magnitude Comparator can be seen in Figure 4-8.

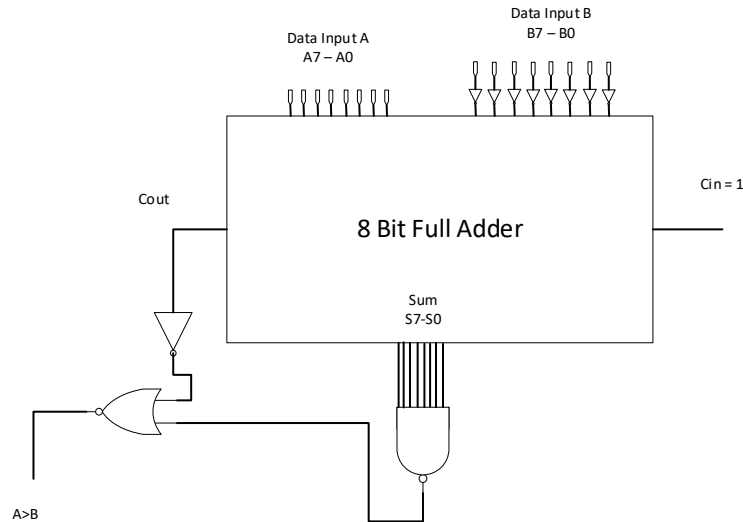


Figure 4- 8. Magnitude Comparator, showing $A > B$ [72]

4.3.3 Single Bit Circular Shift Register

The purpose of inserting a single bit circular shift register is to remove the complexity of the algorithm of the bidirectional shift registers by turning off a CAS (compare-and-swap) unit every other clock cycle. As seen in Figure 4-9, the output enable is inserted every other d-type flip flop. The circuit also needs a preset to insert the value of logic high shifting from flip flop to the other. The circuit needs also an inverter after the output enable since the signal in the CAS circuit requires a logic low to turn off the CAS unit.

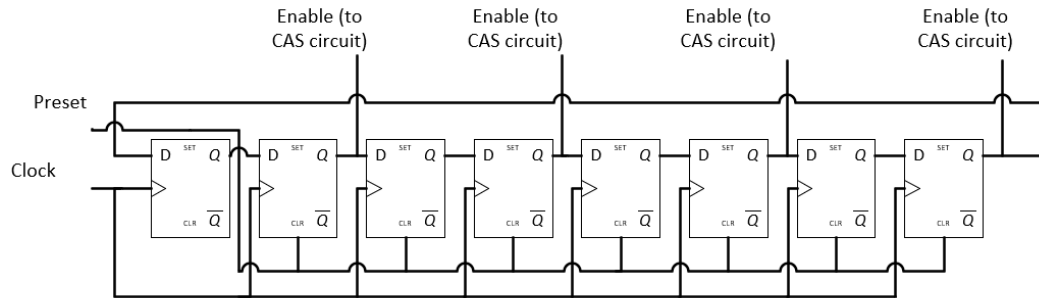


Figure 4- 9. Single Bit Circular Shift Register [72]

4.3.4 Reconfigurable Circuit

The reconfigurable circuit is composed of multiple AND gates with an input of a similar select line for all the bits, and the other is the input bit of the data. If the select line is logic low, then all of the output will be logic low, thus having a value of zero regardless of the values of the bits of the input signal. If the user wishes to sort only 5 data inputs, 3 select lines must be set to logic low to turn them off. The circuit diagram of the reconfigurable circuit is shown in Figure 4-10.



Select Input 1 for ON,
0 for OFF

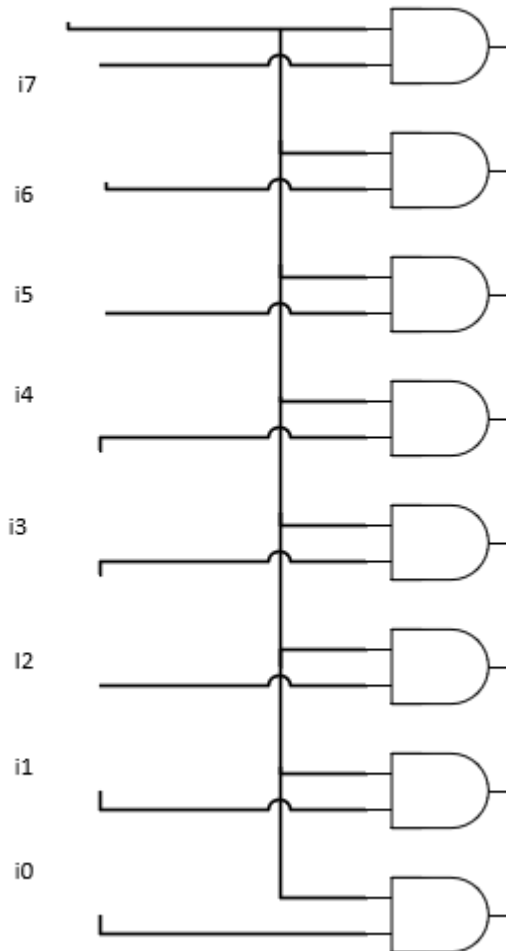


Figure 4- 10. Reconfigurable Circuit Using AND gates with similar Select Input [72]

4.4 Transistor Design and Sizing

For the transistor design, the researcher used different types of gates to represent the logic circuit design of the data sorter. Mostly, NAND gates are used for the design of



the Multiplexers, D type Flip-flops, and the full adder circuit. The NOR gates are used in the CAS units through the Magnitude Comparator. The researcher also used a minimal size for the transistor with W/L ratio of 1u/1u for the NMOS while 2u/1u ratio for the PMOS. This is to minimize the current flowing through the drains of the MOS. This is evident in the MOS drain current formula given by:

$$I_d = \mu C_{ox} \frac{w}{l} [(V_{gs} - V_{th})V_{ds} - \frac{V_{ds}^2}{2}]$$

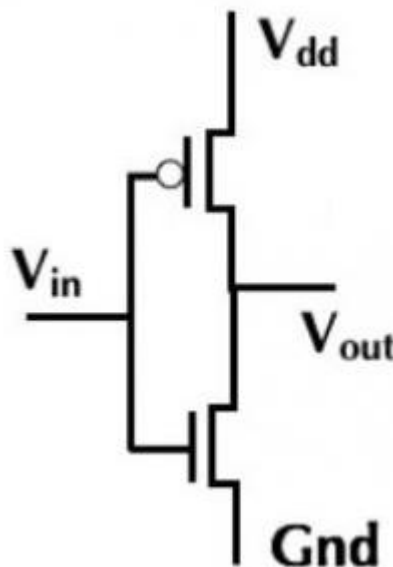


Figure 4- 11. Inverter Diagram Using CMOS Transistors [75]



The inverter is designed by connecting a PMOS and an NMOS to each other as shown in Figure 4-11. The gates are connected to each, as well as the source of both transistors. The drain of the PMOS is connected to the supply voltage, while the drain of the NMOS is connected to the ground. The output of the inverter is always opposite its input, that is if the input is logic low, then the output is logic high. On the other hand, if the input is logic high, then the output is logic low. The inverter design is one of the key components of the blocks inside the data sorter circuit.

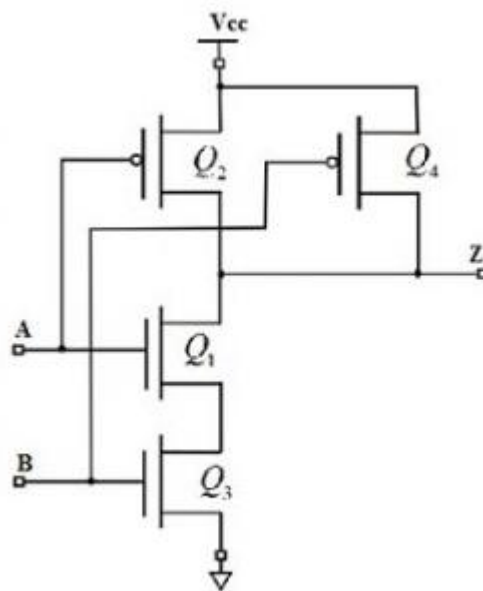


Figure 4- 12. Nand Gate using CMOS Transistors [75]

The NAND Gate is composed of two pairs of PMOS and NMOS. The PMOS are connected in parallel while the NMOS are connected in series as shown in Figure 4-12. The output will only be active low if both inputs are active high. If only one of them is



logic high or logic low, then the output is logic high. The NAND gate has been used by the flip-flops inside the shift registers and storing registers. They are the main component of those circuits.

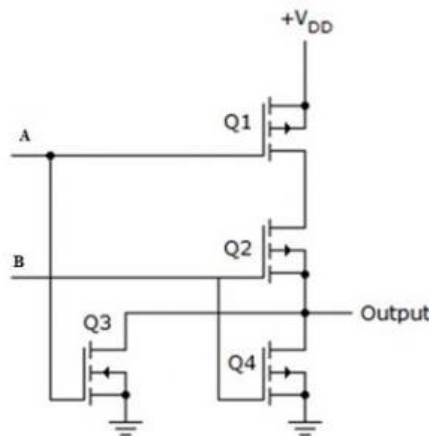


Figure 4- 13. NOR Gate using CMOS Transistors [75]

The NOR Gate is composed of two pairs of NMOS and PMOS. The NMOS are connected in parallel while the PMOS are connected in series. If both inputs logic low, then the output is logic high. If only one of the inputs is logic high or logic low, then the output is logic low. The NOR gate is widely used in the diagram of the compare-and-swap (CAS) circuit specifically the full adders under the magnitude comparator circuit component of the data sorter. The circuit can be seen in Figure 4-13.

4.5 Tanner L-Edit Layout



The design used the 0.35um library for the Tanner L-edit to check for the rules and regulations set by the said library. Every design is carefully drawn in the Tanner Tool and checked for any types of errors before proceeding to the netlist extraction. A sample layout using Tanner Tools is shown in Figure 4-14.

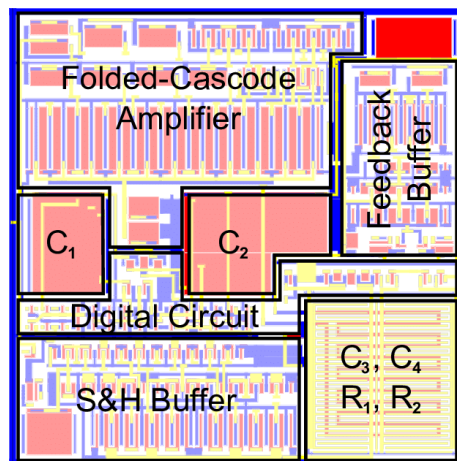


Figure 4- 14. Sample Tanner Layout for Low Noise Amplifier MEA Pixel by Muller and Tanner [76]

4.6 Summary

The design considerations of the study were discussed in this chapter. From the individual physical hardware of the components, up to its architectural and geometric layout were presented using Tanner Tools and LTSPICE. The layout will be extracted and be converted to its equivalent SPICE Netlist, and will be simulated using the 0.35um CMOS library.



Chapter 5

METHODOLOGY



This chapter discusses the implementation of the approaches to answer and resolve the problems indicated in Chapter 1. To add, this chapter also provides information regarding the step by step designing of the Reconfigurable Data-Panning Algorithm Based Sorter – from physical design, logic design, transistor design, and architectural design. This also tackles the netlist extracted from the architectural design, and simulation techniques of the data sorter. The process to be able to achieve objectives and methodology can be seen in Table 5-1. and Figure 5-1, respectively.

TABLE 5- 1 .SUMMARY OF METHODS FOR REACHING THE OBJECTIVES

Objectives	Methods	Locations
GO: To design CMOS Based reconfigurable data panning sorter using the 0.35um library	Design and simulate the performance of a reconfigurable data-panning algorithm-based sorter using the 0.35um CMOS library	Sec. 5.1 and Sec 5.2
SO1: To design a CMOS Based Panning Sorter circuit that can accept up to 8 inputs	Comparison of different data sorter in terms of hardware size, then selection of the data-panning sorter and simulation of its performance for verification of the sorting process	Sec 5.1 on Sec 5.1.2
SO2: To make the sorter reconfigurable by making the inputs variable from minimum of 2 to maximum of 8	Implementation and simulation of a Reconfigurable Circuit in the selection of the total number of inputs for the system.	Sec 5.1 on Sec 5.1.3
SO3: To design the Geometric Layout of the Sorter Circuit	1. Usage of Tanner Tools in creating the layout of the Reconfigurable Data Panning Sorter. 2. Individual creation of design of each sub-components of the data sorter and combining them together to form the final circuit	Sec 5.1 on Sec 5.1.4



SO4: To evaluate the performance of the sorter circuit using each of the 5 corner libraries	Individual testing using a library file in LTSpice to verify the response of the final circuit in each of the five corner libraries: fast-fast, slow-slow, slow-fast, fast-slow and typical-typical.	Sec 5.2 on Sec 5.2.2
SO5: To characterize the circuit designed in terms of area, power consumption, temperature, and frequency	1. For area, using Tanner Tools and ruler tool to determine the final area consumption of the circuit. 2. For power consumption, insertion of a “dummy” external resistor to solve for the total current flowing through the sorter and using the power formula to get the total power consumption. 3. For temperature, usage of the .temp command in LTSpice and verification of the maximum temperature it can accept to have an acceptable performance.	Sec 5.2 on Sec 5.2.3

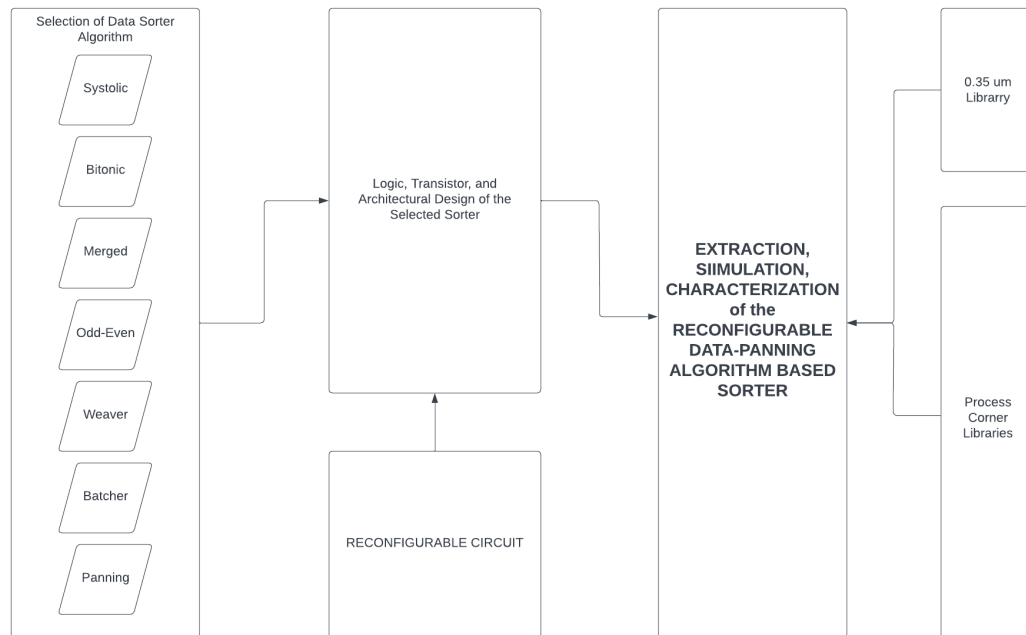


Figure 5- 1. Methodology Flowchart of the Study



5.1 Implementation

5.1.1 Selection of Data-Panning Sorter

The study's starting point is the selection of the proper data sorting algorithm that provides the lowest value of hardware implementation. A comparison in Table 4-1 shows the comparison of data sorter algorithms with their respective implementation sizes. It shows that the data-panning gives the smallest possible hardware for all of the presented sorters.

5.1.2 Design and Testing of Individual Components

In section 4.2, the overview of the entire reconfigurable data sorter circuit is discussed. All of the individual components of the data sorter are tested to their functions using LTSpice. Each circuit is analyzed to match the desired performance and function of the data-panning sorter and converted to their equivalent logic and transistor design. The transistor design will be implemented for the layout of the reconfigurable data panning sorter.



5.1.3 Layout of the Data-Panning Algorithm Sorter

The final transistor design will be geometrically drawn using Tanner Tools specifically L-Edit Tool. The library to be used is the 0.35um CMOS library which has specific rules and guidelines in designing the layout of the data sorter. It will be implemented and be checked from time to time for certain errors in the design rules using the DRC command. Each individual component will be designed separately for their layout, and be combined together in single functional data sorter layout using the “Cell Instance” feature of the L-Edit Tool. Since transistors will be drawn in the L-Edit Tool, the ratio W/L for all NMOS will be 1u/1u while PMOS will have a size ratio W/L of 2u/1u

5.1.4 Extraction of the Layout to Spice Netlist

After the layout, it will be extracted into an equivalent Spice Netlist for further testing and evaluation. The 0.35um library will be used for the simulation of the final data sorter design. To get SO4 as indicated in Table 5-1, a file will be used to simulate the five corner libraries of CMOS namely: fast-fast, slow-slow, fast-slow, slow-fast, and typical-typical. The performance specifications to achieve SO5 will also be made using



the LTSPICE Tool. All circuit simulation will have a voltage of 3V as the supply and considered voltage 3V as logic high while 0V will be for logic low.

5.1.5 Input/Output Node Naming

The data that will be sorted will have a unique port name for each of the input and output of the data sorter simulation. This is because of the features of the LTSPICE in naming nodes of the circuit. All data will be converted to their respective binary equivalents, inputs are ranging from 0 (0000 0000) to 255 (1111 1111) for 8 bits of data. Each input will be given an equivalent voltage, which convertible to a logic level. Logic high (3V) for high state while logic low (0V) for low state. A summary of inputs and output name is shown in Table 5-2 and 5-3 respectively.



Table 5- 2. Summary of Input Naming Nodes

Inputs								
Input Order	Input Name	Node Name	Description		Input Order	Input Name	Node Name	Description
1st input	va*	va0	LSB		5th input	ve*	ve0	LSB
		va1					ve1	
		va2					ve2	
		va3					ve3	
		va4					ve4	
		va5					ve5	
		va6					ve6	
		va7	MSB				ve7	MSB
2nd input	vb*	vb0	LSB		6th input	vf*	vf0	LSB
		vb1					vf1	
		vb2					vf2	
		vb3					vf3	
		vb4					vf4	
		vb5					vf5	
		vb6					vf6	
		vb7	MSB				vf7	MSB
3rd input	vc*	vc0	LSB		7th input	vg*	vg0	LSB
		vc1					vg1	
		vc2					vg2	
		vc3					vg3	
		vc4					vg4	
		vc5					vg5	
		vc6					vg6	
		vc7	MSB				vg7	MSB
4th input	vd*	vd0	LSB		8th input	vh*	vh0	LSB
		vd1					vh1	
		vd2					vh2	
		vd3					vh3	
		vd4					vh4	
		vd5					vh5	
		vd6					vh6	
		vd7	MSB				vh7	MSB
NOTE:	MSB:	MOST SIGNIFICANT BIT						
	LSB:	LEAST SIGNIFICANT BIT						



Table 5- 3. Summary of Output Naming Nodes

Outputs								
Output Order	Output Name	Node Name	Description		Output Order	Output Name	Node Name	Description
Least Value (8th Place)	v8*	v80	LSB		4th place	v4*	v40	LSB
		v81					v41	
		v82					v42	
		v83					v43	
		v84					v44	
		v85					v45	
		v86					v46	
		v87	MSB				v47	MSB
7th Place	v7*	v70	LSB		3rd place	v3*	v30	LSB
		v71					v31	
		v72					v32	
		v73					v33	
		v74					v34	
		v75					v35	
		v76					v36	
		v77	MSB				v37	MSB
6th Place	v6*	v60	LSB		2nd place	v2*	v20	LSB
		v61					v21	
		v62					v22	
		v63					v23	
		v64					v24	
		v65					v25	
		v66					v26	
		v67	MSB				v27	MSB
5th Place	v5*	v50	LSB		Largest Value (1st Place)	v1*	v10	LSB
		v51					v11	
		v52					v12	
		v53					v13	
		v54					v14	
		v55					v15	
		v56					v16	
		v57	MSB				v17	MSB
NOTE:	MSB:	MOST SIGNIFICANT BIT						
	LSB:	LEAST SIGNIFICANT BIT						



5.2 Evaluation

5.2.1 Comparison of Sorted Data

Using LTSPICE tool, the netlist code will be simulated and tested if the input data are already sorted from lowest to highest. The digital data inserted based on the node naming on Table 5-2 should reflect the proper sorted data in Table 5-3. After being sorted, the circuit must be able to stop the sorting process to have a consistent output after exactly $N+1$ clock cycles.

5.2.2 Evaluation using the Process Corner Libraries

The netlist will also be tested using the five CMOS corner libraries namely: fast-fast, slow-slow, fast-slow, slow-fast, and typical-typical. The netlist code will be modified each simulation for the performance of the data sorter. A file library must be stored inside a directory in your computer in order for the netlist to read the said library. The simulation and testing must show an undistorted waveform for each of the corner libraries and it is expected due to its digital form in nature.



5.2.3 Data Sorter Characteristics

The data sorter simulation will be characterized in terms of power consumption, temperature, and frequency. Inserting an external dummy resistor will be the key for getting the total power consumption. The current flowing through the resistor will be measured using the .meas command of LTSPICE and will be multiplied to the supply voltage 3V to get the total power consumption. As for the temperature, use the .temp command to get the maximum temperature it can absorb for the circuit not to be distorted.

5.3 Summary

In this chapter, the methodology was outline in order to achieve the objectives of the study. The methodology presented the steps in designing the Reconfigurable Data-Panning Algorithm Based Sorter starting from its physical design, logic design, transistor design, and its geometric design. The layout is extracted into an equivalent SPICE netlist and simulated along with the CMOS libraries presented. The simulation presented will be characterized by its performance.



Chapter 6

RESULTS AND DISCUSSIONS



The previous chapter discusses the methodology of the study. The results of the simulation are shown in this chapter as well as how the objectives were met. Layouts of the individual components of the data sorter are inserted and tested as well as the graphs of the sorted data. The design was able to sort the data from lowest to highest, as well as a selection of the number of inputs to be sorted to make it reconfigurable.

6.1 Sorted Data

As shown in Table 5-2, each of the data are given a unique input node name. The first input is named va0-v7, having va0 as the least significant bit while va7 as the most significant bit. For the second input it is vb0-vb7, third input is vc0-vc7, fourth input is vd0-vd7, fifth input as ve0-ve7, sixth input is vf0-vf7, seventh input is vg0-vg7, and vh0-vh7 for the eighth input.

On the output side, the nodes are named as v10-v17 for the highest sorted value, v20-v27 for the second highest sorted value, v30-v37 for the third highest sorted value, v40-v47 for the fourth highest sorted value, v50-v57 for the fifth highest sorted value, v60-v67 to the sixth highest sorted value, v70-v77 for the seventh highest sorted value and v80-v87 for the eight highest or considered as the least sorted value of the inputs.



6.1.1 First Trial – First Set of Data Values to be Sorted

Initial values (8 bits) are considered for the data sorter. The minimum value for an input is zero (0000 0000) while the maximum value for the input is 255 (1111 1111). Each input corresponds to an assigned value, also assigning each bits with an exclusive node name.

The chosen values for the 1st testing are the following:

26 (binary 0001 1010) – Input A

100 (binary 0110 0100) – Input B

21 (binary 0001 0101) – Input C

200 (binary 1100 1000) – Input D

5 (binary 0000 0101) – Input E

150 (binary 1001 0110) – Input F

17 (binary 0001 0001) – Input G

120 (binary 0111 1000) – Input H

Using Microsoft Excel for initial testing and verification, the expected outputs of the sorted value per clock cycle are as follows:



Table 6-1. Expected Sorted Value per clock cycle of the data sorter (1st Trial)

PORT NAME	va0-v7	vb0-vb7	vc0-vc7	vd0-vd7	ve0-ve7	vf0-vf7	vg0-vg7	vh0-vh7		
INPUTS	26	100	21	200	5	150	17	120	INITIAL CLOCK	
cas	26	100	21	200	5	150	17	120		
store	120	26	100	21	200	5	150	17	1st CLOCK CYCLE	
cas	120	26	21	100	5	200	17	150		
store	150	120	26	21	100	5	200	17	2nd CLOCK CYCLE	
cas	120	150	21	26	5	100	17	200		
store	200	120	150	21	26	5	100	17	3rd CLOCK CYCLE	
cas	120	200	150	21	5	26	17	100		
store	100	120	200	150	21	5	26	17	4th CLOCK CYCLE	
cas	100	120	150	200	5	21	17	26		
store	26	100	120	150	200	5	21	17	5th CLOCK CYCLE	
cas	26	100	120	150	200	5	17	21		
store	21	26	100	120	150	200	5	17	6th CLOCK CYCLE	
cas	21	26	100	120	150	200	5	17		
store	17	21	26	100	120	150	200	5	7th CLOCK CYCLE	
cas	17	21	26	100	120	150	200	5		
store	5	17	21	26	100	120	150	200	8th CLOCK CYCLE	(FINAL SORTED VALUE)
PORTNAME	v80-v87	v70-v77	v60-v67	v50-v57	v40-v47	v30-v37	v20-v27	v10-v17		

The sorted values are arranged lowest to highest as per Table 6-1 for the first set of trial values. During the 8th clock cycle, the 8 inputs are already sorted according to their numerical sizes. Also, at the final clock cycle, Input A register is considered as the initial input value while the last value is stored in Input H register, other inputs are stored in nodes vb up to nodes vg. Writing and naming these inputs can be done randomly depending on the choice of the user. The output which is considered as the highest value is named as v10-v17 while the lowest value is named v80-v87.

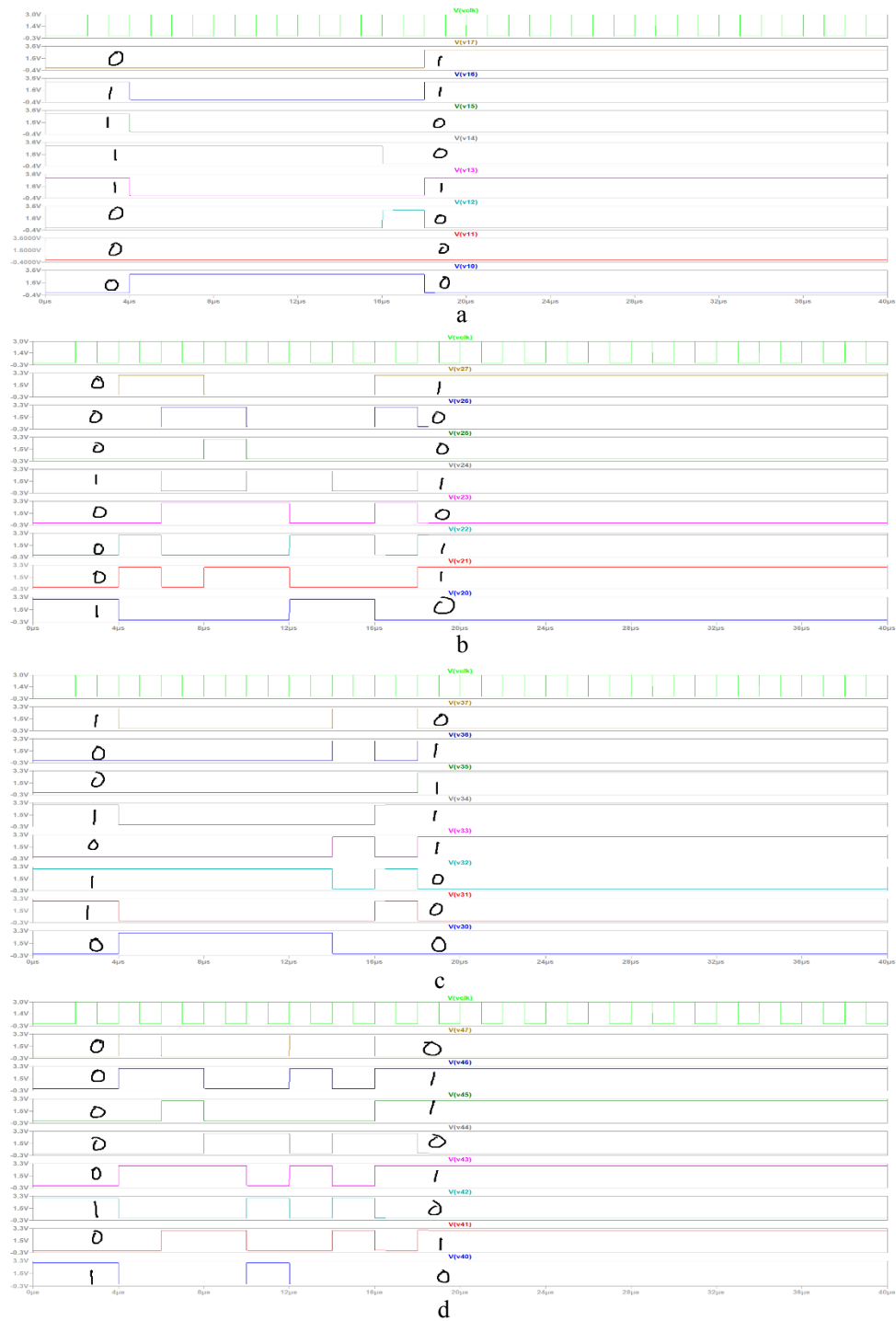


Figure 6- 1. Graphs of Sorted Data of the 1st place (Highest Value) up to the 4th place Sorted Value for the 1st Trial



The sorted value for the highest input is shown in Figure 6-1a. It is initially 120 (0111 1000) and sorted the values to 200 (1100 1000). The input values are named vh0-vh7 while the outputs of the highest sorted value are named v10-v17. The circuit implementation was able to sort the initial input of the data sorter successfully. The algorithm was also followed in the process of sorting. For Figure 6-1b, the graphical representation for data input G is shown. The 2nd to the highest value must be shown in the final sorted value. Initially, the input value is 17 (0001 0001) and was sorted after 8 clock cycles the value of 150 (1001 0110). Input vg0-vg7 was initially stated, then the output values for the 2nd place are v20-v27. Input F has an initial value of 150 (1001 0110) and was sorted to 120 (0111 1000). It is still consistent to the values presented at Table 6-2. The port names for the inputs are vf0-vf7, then the sorted values for the 3rd highest are named as v30-v31. For the next graph in Figure 6-1d, the initial value for Input E (ve0-ve7) is 5 (0000 0101) and sorted each clock cycle correctly as per Table 6-2. The final sorted value is 100 (0110 0100) and it appeared correctly after 8 clock cycles with port names v40-v47. All graphs are simulated simultaneously and separated in multiple windows due to the lack of size and visibility of the LTSpice but shows that after 8 clock cycles, they are all in their correct sorted places during the 18us-20us section the period of the data sorting process.

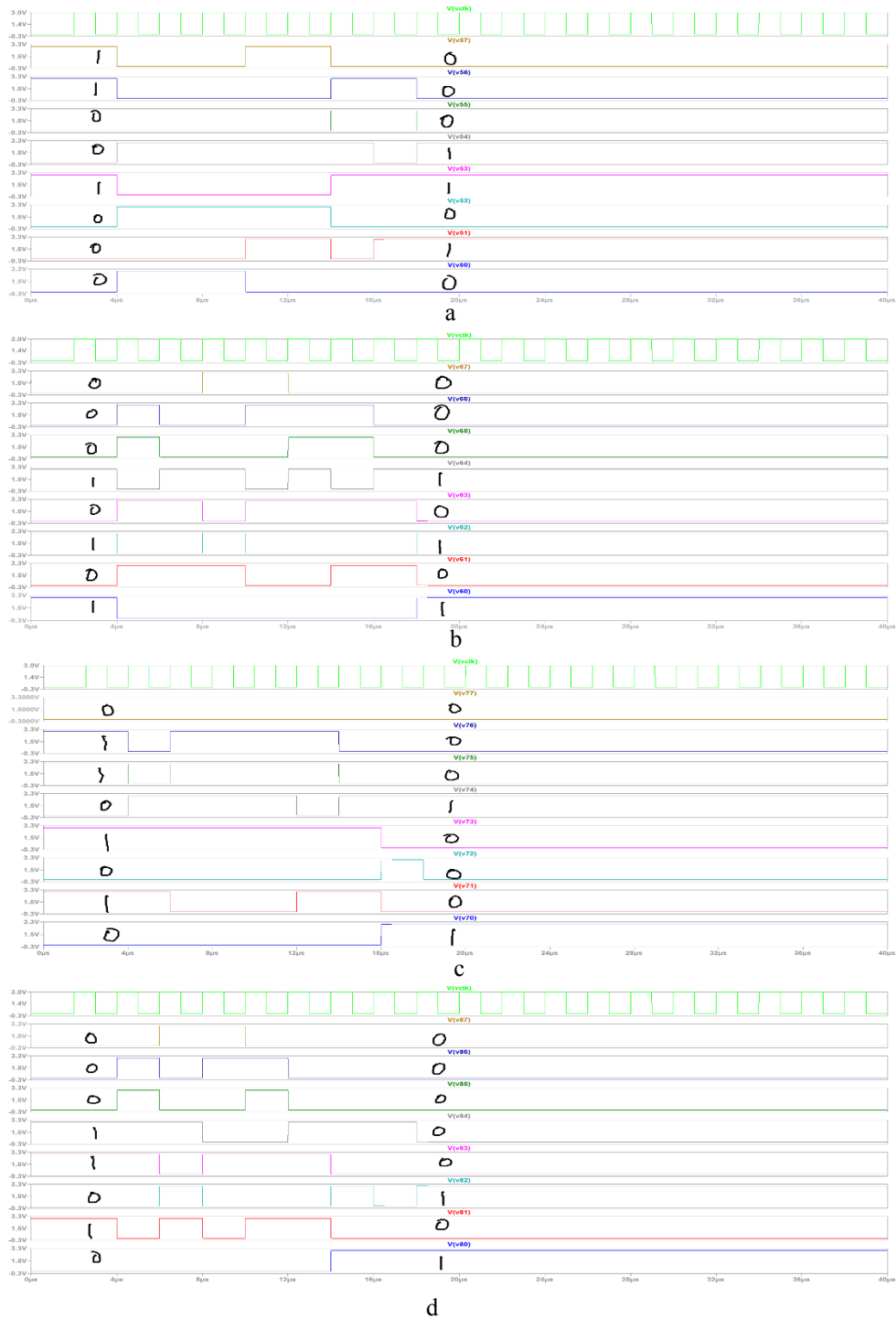


Figure 6- 2. Graphs of Sorted Data of the 5th place up to the 8th (lowest value) place
Sorted Value



The continuation of the graphs of the simulation can be seen in the next figure, Figure 6-2, and it shows the graphs of the 5th place-8th place. Again, they are simultaneous simulations and were in proper position of the sorting process. For Figure 6-2a the initial value is 200 (1100 1000) while the final sorted value is 26 (0001 1010). It is still consistent with the expected values as per Table 6-2. The input values are named vd0-vd7 and the final sorted values are named v50-v57. For Figure 6-2b, the initial values for Input C is 21 (0001 0101) with port names vc0-vc7 and after a total of eight clock cycles, the final sorted value is also 21 (0001 0101) with port names v60-v67. This is true as per Table 6-2. For the next input in Figure 6-2c, the initial data for the Input B is 100 (0110 0100) with port names vb0-vb7, as shown in the initial clock cycle. After one clock cycle, it became 26. The final sorted value after eight clock cycles is 17 (0001 0001) with port names v70-v77. In Figure 6-2d, the graph shows the values of each bit for every clock cycle of the 8th number. The stored data at first clock cycle is 26 (0001 1010) with port names va0-v7 then it became 120 for the second cycle which is expected from the values of Table 6-2. The final sorted value as shown in Figure 6-2d is 5 (0000 0101) with port names v80-v87. The most significant bit of the data is v87 while the least significant bit is v80.



6.1.2 Second Trial – Second Set of Data Values to be Sorted

To prove the effectiveness of the data sorter, the researcher has done another test with different sets of values. The values are randomly selected with values of 0-255, converted into digital values (0000 0000 – 1111 1111). The random values for the second testing are:

255 (binary 1111 1111) – Input A

146 (binary 1001 0010) – Input B

99 (binary 0110 0011) – Input C

165 (binary 1011 0101) – Input D

205 (binary 1100 1101) – Input E

139 (binary 1000 1011) – Input F

220 (binary 1101 1100) – Input G

8 (binary 0000 1000) – Input H

Once again, an initial testing and verification using Microsoft Excel is done before the actual Spice Simulation. Eight numerical inputs are inserted in each of the cells, and programed accordingly using the algorithm of the data panning based digital sorter. The test is shown in Table 6-2.



Table 6- 2. Expected Sorted Value per clock cycle of the data sorter (2nd Trial)

PORT NAME	va0-vb7	vb0-vb7	vc0-vc7	vd0-vd7	ve0-ve7	vf0-vf7	vg0-vg7	vh0-vh7		
INPUTS	255	146	99	165	205	139	220	8	INITIAL CLOCK	
cas	146	255	99	165	139	205	8	220		
store	220	146	255	99	165	139	205	8	1st CLOCK CYCLE	
cas	220	146	99	255	139	165	8	205		
store	205	220	146	99	255	139	165	8	2nd CLOCK CYCLE	
cas	205	220	99	146	139	255	8	165		
store	165	205	220	99	146	139	255	8	3rd CLOCK CYCLE	
cas	165	205	220	99	139	146	8	255		
store	255	165	205	220	99	139	146	8	4th CLOCK CYCLE	
cas	165	255	205	220	99	139	8	146		
store	146	165	255	205	220	99	139	8	5th CLOCK CYCLE	
cas	146	165	205	255	220	99	8	139		
store	139	146	165	205	255	220	99	8	6th CLOCK CYCLE	
cas	139	146	165	205	220	255	8	99		
store	99	139	146	165	205	220	255	8	7th CLOCK CYCLE	
cas	99	139	146	165	205	220	255	8		
store	8	99	139	146	165	205	220	255	8th CLOCK CYCLE	(FINAL SORTED VALUE)
PORTNAME	v80-v87	v70-v77	v60-v67	v50-v57	v40-v47	v30-v37	v20-v27	v10-v17		

In table 6-2, the expected values are stored every clock cycle. After 8 clock cycles (period 18us-20us), the input values are now sorted according to their sizes. This table and simulation is done using Microsoft Excel just like in the first set of trial input values.



Figure 6- 3. Graphs of Sorted Data of the 1st place (Highest Value) up to the 4th place Sorted Value for the 2nd Trial



The graph is still consistent as per the second trial and the simulation performed using Microsoft Excel. Input H has a value of 8 (0000 1000) and its expected output must be the largest sorted data. After 8 clock cycles, the value has become 255 (1111 1111) which is shown in Figure 6-3a. Each clock cycle also shows its expected shifted value in the presented graphs. The output sorted data is stored in nodes v10-v17 (v10 as the least significant bit, and v17 as the most significant bit). For Input G, the value of the inserted data is 220 (1101 1100) and shows a sorted value of 220 (1101 1100) which is also expected based from the Excel Simulation. The sorted value is stored in nodes v20-v27 as shown in Figure 6-3b. The next graph shows the input F which is 139 (1000 1011) and has a sorted value of 205 (1100 1101). It is also consistent with the simulation performed in Excel. As for the Input E, the inserted value is 205 (1100 1101) while the sorted output is 165 (1011 0101). The output data for the 4th place is stored in v40-v47 having v40 as the least significant bit, and v47 as the most significant bit.

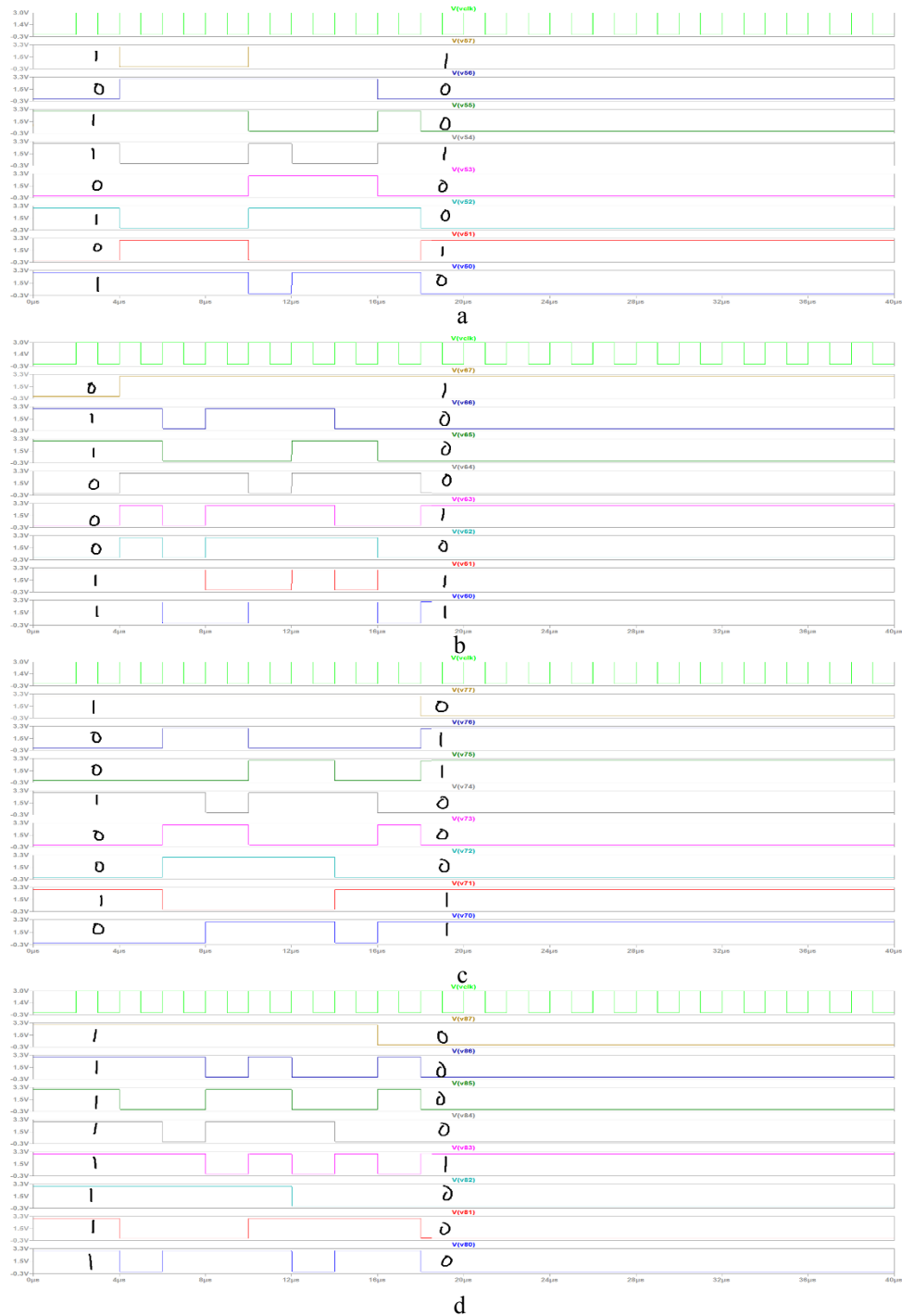


Figure 6- 4. Graphs of Sorted Data of the 4th place up to the 8th place (Lowest Value)
Sorted Value for the 2nd Trial



For the continuation of the simulation, Figure 6-4a shows the input D (vd0-vd7) having vd0 as the least significant bit and vd7 as the most significant bit. The input value for D is 165 (1011 0101) and the sorted output is 146 (1001 0010). It is saved inside the nodes v40-v47, with v40 as the least significant bit and v47 as the most significant bit. It also follows the proper shifted values per clock cycle and after 8 clock cycles, it shows the final sorted value. Input C has an initial stored value of 99 (0110 0011) inside the nodes vc0-vc7. After 8 clock cycles, the sorted value is now 139 (1000 1011) and stored in data output node v30-v37, having v30 as the least significant bit and v37 as the most significant bit. Input B has an initial stored value of 146 (1001 0010) and after 8 clock cycles, it became 99 (0110 0011). It is still consistent with the table of values of the simulations performed in MS Excel. For the last graph in Figure 6-4d, the output shows an expected value of 8 (0000 1000) with 255 (1111 1111) as its initial stored value.

The two trials have met and satisfied the algorithm of the data-panning sorter. More trials have been done and been consistent with its performance in sorting the values from lowest to highest.



6.2 Simulation of the Reconfigurable Circuit

Using the same set of inputs for trial 1, the reconfigurable circuit was tested by inserting it before the input ports of the data sorter. The select lines determine the allowed inputs to enter the data sorter. If the select line is logic low, the input will be all equal to zero based on the AND gate truth table [73]. If the select line is logic high, it will just let pass that input for that port.

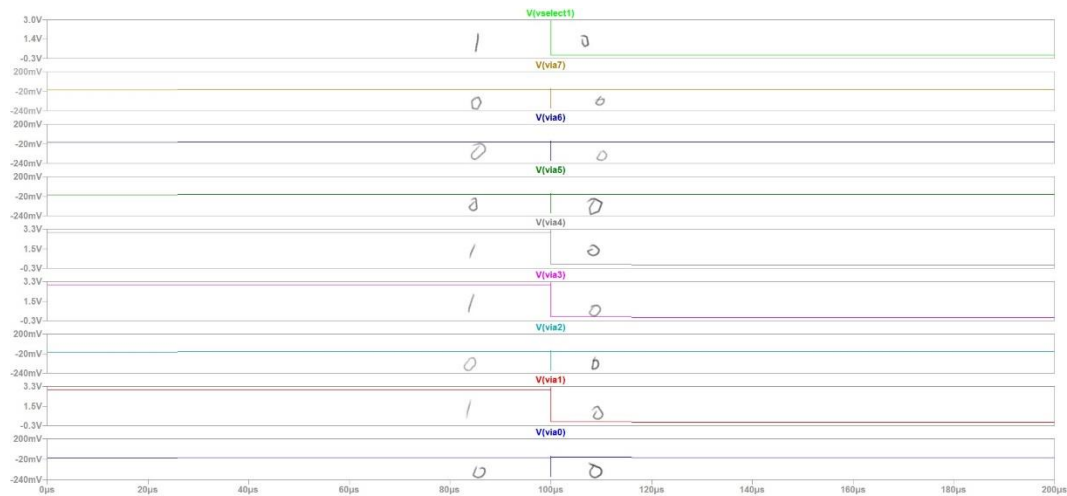


Figure 6- 5. Graph for Input A, for select line equal to logic high and low

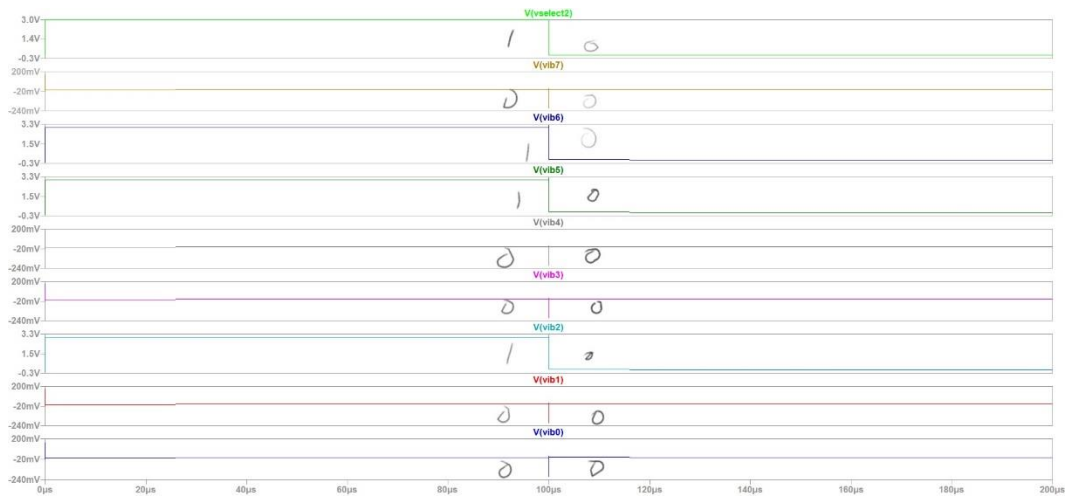


Figure 6- 6. Graph for Input B, for select line equal to logic high and low

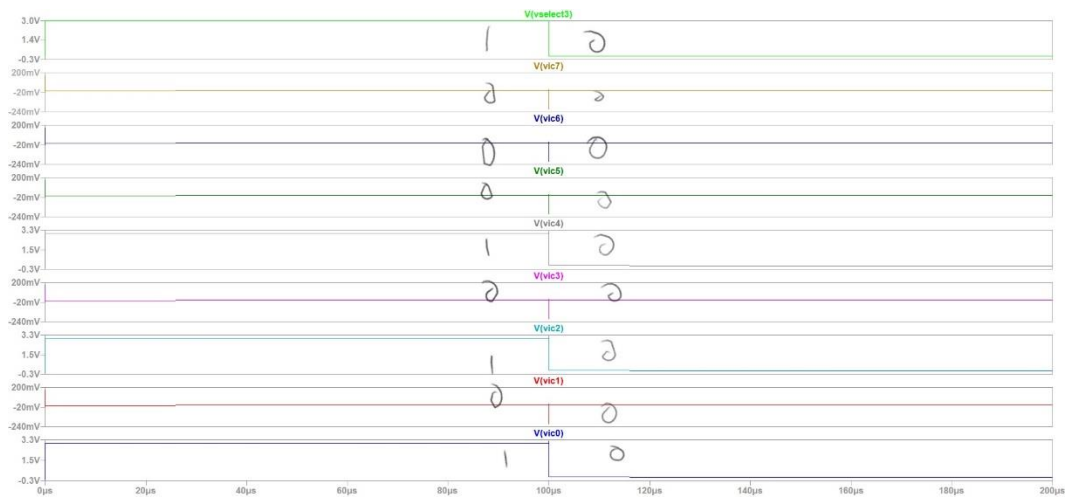


Figure 6- 7. Graph for Input C, for select line equal to logic high and low

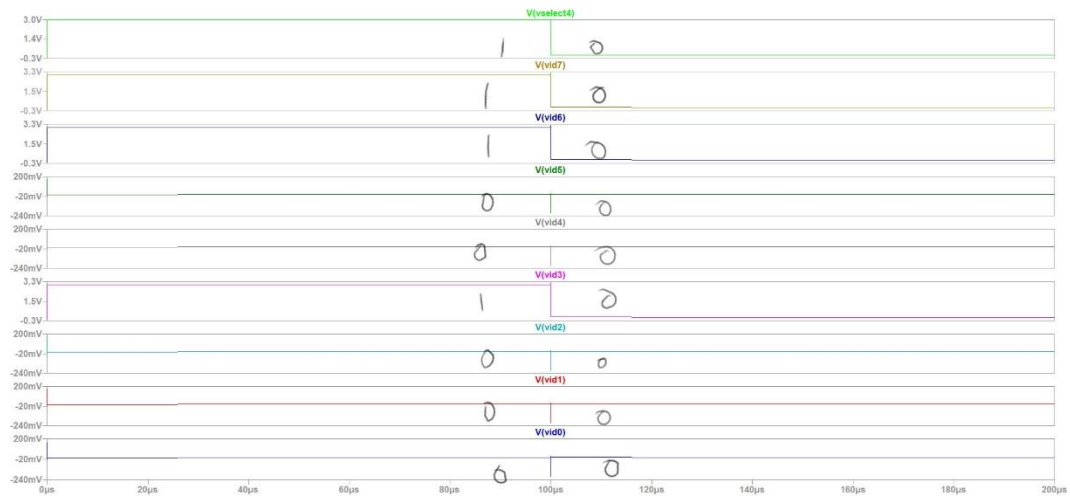


Figure 6- 8. Graph for Input D, for select line equal to logic high and low

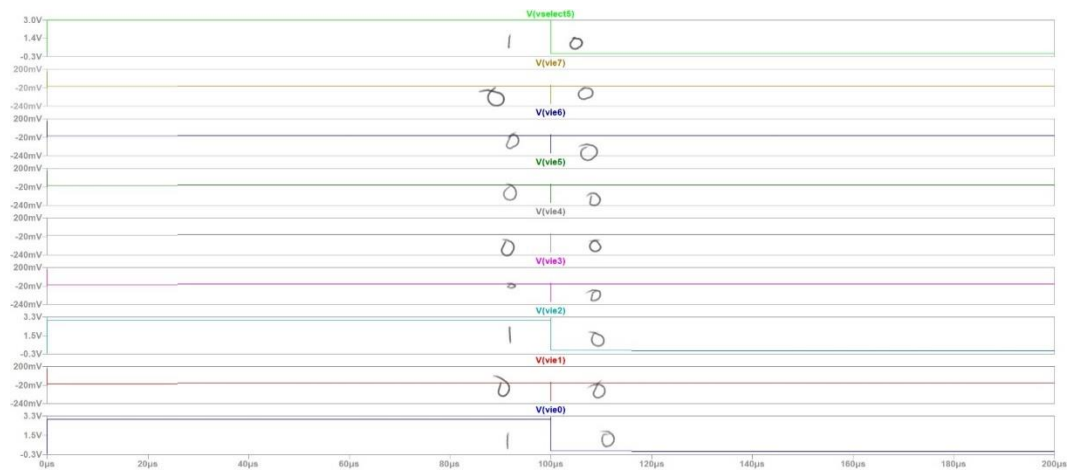


Figure 6- 9. Graph for Input E, for select line equal to logic high and low

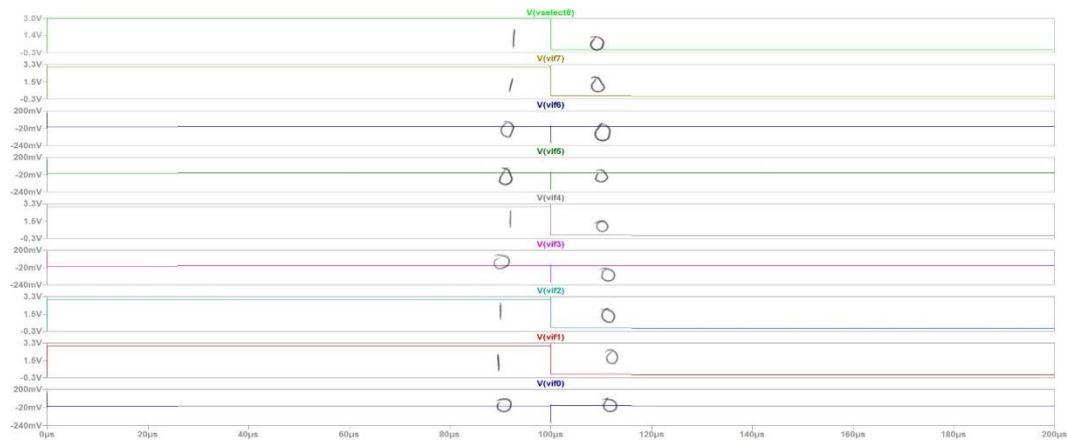


Figure 6- 10. Graph for Input F, for select line equal to logic high and low

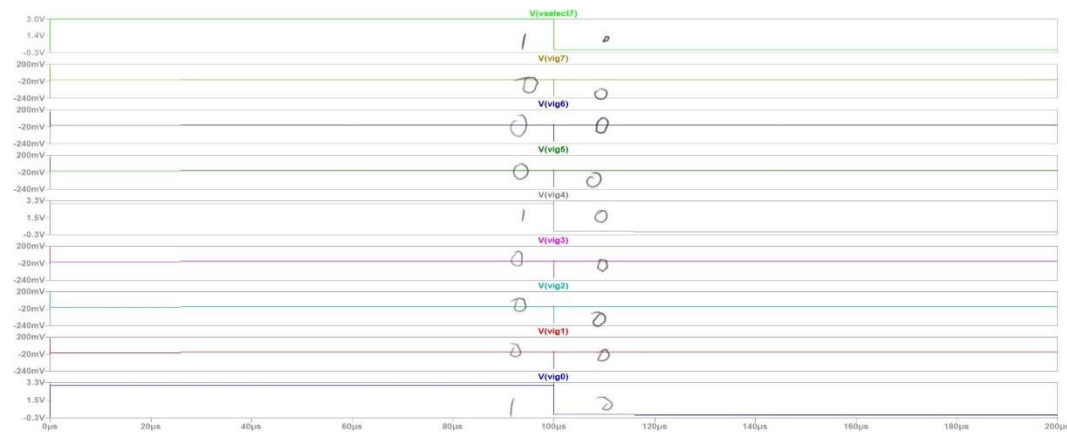


Figure 6- 11. Graph for Input G, for select line equal to logic high and low

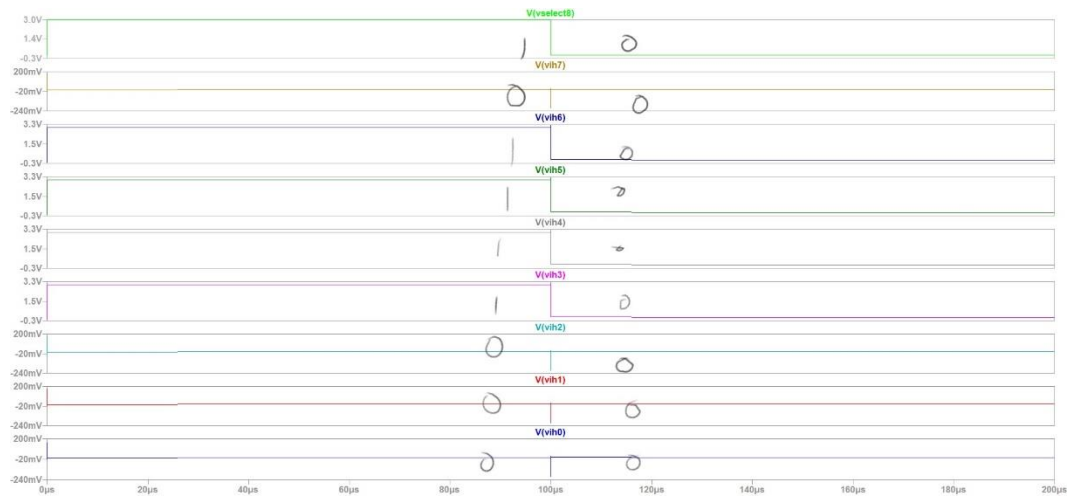


Figure 6- 12. Graph for Input H, for select line equal to logic high and low

The graphs from Figure 6-5 up to Figure 6-12, show the response of the reconfigurable circuit based on the select lines inserted by the user. If the select line is logic zero, the value of the inputs will all be equal to zero. On the contrary, if the logic of the select line is high, then the inputs will be buffered to the main data sorter.

6.3 Geometric Layout

Each transistor-based schematic has been designed using Tanner Tools L-Edit. Every components have been designed, and used again for repetitive circuits using the Cell Instance Command and shows no errors in terms of the DRC checking.

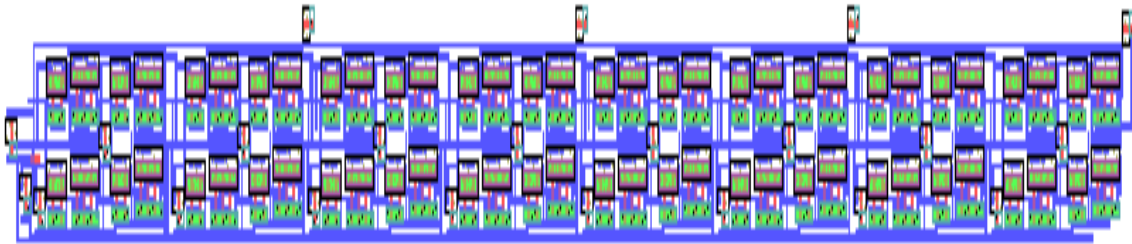


Figure 6- 13: Tanner Layout for Single Bit Circular Shift Register

The geometric layout of the Single Bit Circular Shift register is shown in Figure 6-13. It is composed of multiple flipflops which are connected and converted into equivalent NAND Gates. It has a width of 776.1 μm and 48.1 μm height. For its total area, it has 36,894 μm^2 as shown in Table 6-4.

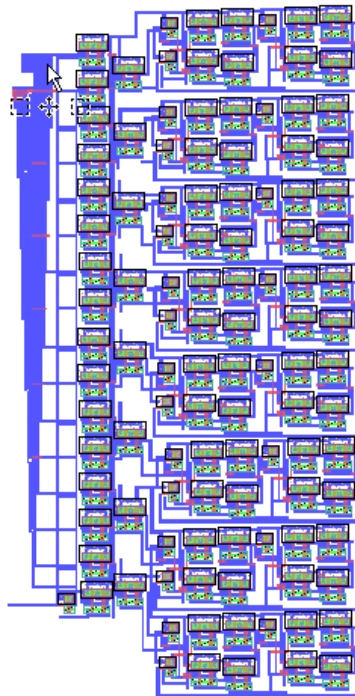


Figure 6- 14. Tanner Layout for Data Registers



For the data registers, it is composed of 16 flipflops for an 8-Input Data. Each of the flipflops were designed individually and used the function “Cell Instance” in the L-Edit tool for the repetitive diagrams. The geometric layout of the data register is shown in Figure 6-14. The total width, length, and area of the data registers are $149.5\text{ }\mu\text{m}$, $294.5\text{ }\mu\text{m}$, and $44027\text{ }\mu\text{m}^2$ respectively.

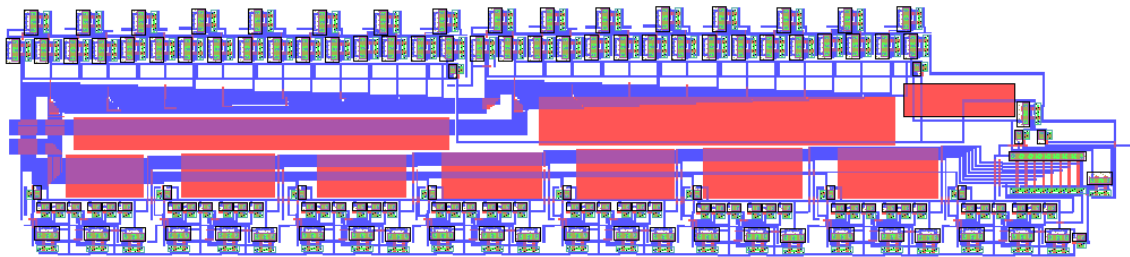


Figure 6- 15.Tanner Layout for Compare and Swap Circuit

In Figure 6-15, the geometric layout for the Compare and Swap Circuit is shown. The CAS circuit is made up of magnitude comparators, multiplexers, and combinational gates. The layout was designed using Tanner L-Edit Tool and followed the $0.35\text{ }\mu\text{m}$ library, following its design rules and regulations. The total width, length, and area of the CAS circuit are $605.7\text{ }\mu\text{m}$, $133\text{ }\mu\text{m}$, and $80558.1\text{ }\mu\text{m}^2$ respectively.

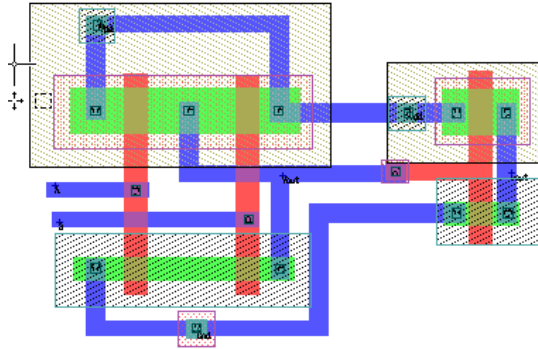


Figure 6- 16: Tanner Layout for Reconfigurable Circuit

The sample geometric layout of a Reconfigurable circuit is shown in Figure 6-16. It is made up of a combination circuit of NAND and an Inverter. The sets of reconfigurable circuits are inserted before each of the data register inputs. It has a width, height, and area of $23.1\ \mu\text{m}$, $15.5\ \mu\text{m}$, and $358.05\ \mu\text{m}^2$ respectively.

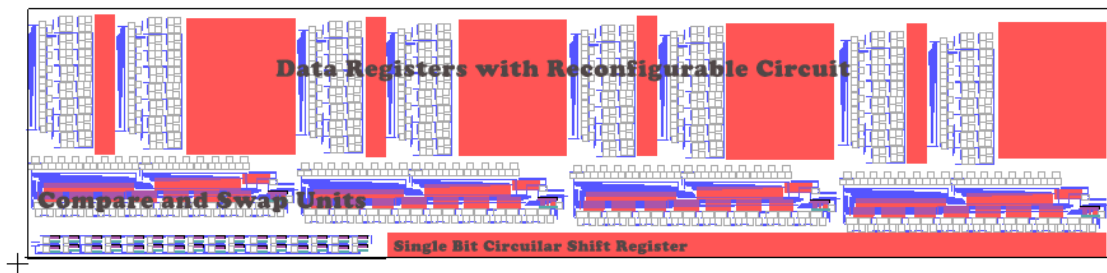


Figure 6- 17: Tanner Layout for the Final Data Sorter

For the final layout, each of the individual units are combined to form the data sorter. It has total of $2493.3\ \mu\text{m}$ width, $557.4\ \mu\text{m}$ length, and an area of $1.36\ \text{mm}^2$.



6.4 Process Corner Library Simulation

The purpose of checking the simulation of the spice netlist in the process corner libraries is to determine the effects of the design with respect to the transistor characteristics provided by the manufacturers, just in case the design's intention is for fabrication.



Figure 6- 18. Graphical Representation for Typical-Typical Process Corner Library



Figure 6- 19. Graphical Representation for Fast-Fast Process Corner Library



Figure 6- 20. Graphical Representation for Slow-Fast Process Corner Library



Figure 6- 21. Graphical Representation for Fast-Slow Process Corner Library



Figure 6- 22. Graphical Representation for Slow-Slow Process Corner Library



The graphs indicated by the five corner libraries provide the same response for the output of the data sorter. In the simulation and testing, the minimum value was simulated for all the five corner libraries. Almost no change has been observed in each of the five corner libraries namely: typical-typical, fast-fast, slow-slow, slow-fast, and fast-slow. This represents the characteristics of the CMOS, that is for fast-slow, it means fast PMOS and slow NMOS respectively.

6.5 Size, Power Consumption, and Frequency

6.5.1 Size

The size of the circuit design is based from the ruler function of the Tanner Tool. Each individual circuit is measured as well as the total measurement of the data sorter.

Table 6- 3. Summary of Sizes of Circuits

CIRCUIT	Width (μm)	Height (μm)	Perimeter (μm)	Area (μm) ²
Compare-and-Swap Circuit	605.7	133	1477.4	80558.1



Data Registers	149.5	294.5	888	44027.75
Single Bit Circular Shift Register	776.1	48.1	1628.4	36894.41
Reconfigurable Circuit	23.1	15.5	77.2	358.05
DATA SORTER	2439.3	557.4	5993.4	1359665

Table 6-3 shows the sizes of the circuits in terms of its width, height, perimeter and area in the Tanner Tool Layout. There are 4 compare-and-swap units, 8 data registers, 1 single bit circular shift register, and 64 reconfigurable circuits for the data sorter.

6.5.2 Power Consumption

For the computation of total power consumption, the researcher inserted an external small valued resistor, 0.001 ohms, to compute for the current flowing through the entire circuit. The current measured is multiplied to the supply voltage 3V to get the total power consumption of the data sorter circuit. The block diagram can be seen in Figure 6-23.

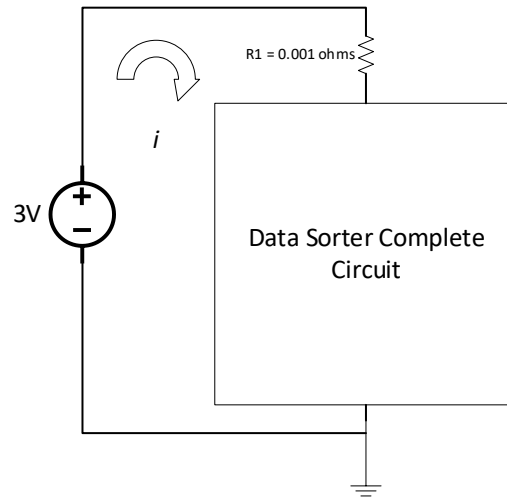


Figure 6- 23. Block Diagram of Data sorter Circuit with external Resistor for solving Power Consumption.

The computed value of the current is 185 mA based on the .print command of the LTSpice. The total power consumption can be computed using the formula:

$$P = VI$$

P = Total Power Consumption

V = Supply Voltage, 3V

I = Current Flowing through R1

$$P = (3V) * (185 \text{ mA}) = \mathbf{555 \text{ mW}}$$

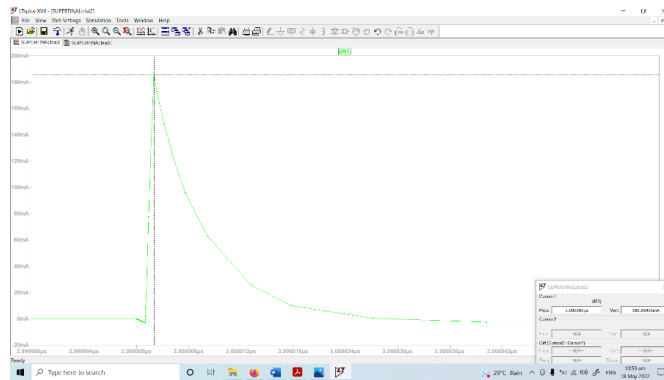


Figure 6- 24. Current Measurement using LTSpice and .print command

6.5.3 Frequency

The clock circuit has a period of $2\mu\text{S}$, giving the frequency a 500 kHz. A total of 8 clock cycles is needed for the completion of the data sorting. This means that after 8 clock cycles ($16\mu\text{S}$), the output data is now sorted. A delay of $2\mu\text{S}$ is implemented to allow the storage of input data in the registers. The load line signifies the trigger of the shifting of the registers in the circuit. After a total of $18\mu\text{S}$ ($16\mu\text{S} + 2\mu\text{S}$ delay), the data are sorted from lowest to highest depending on the position of its nodes.

6.5.4 Temperature

Using the .temp command will generate a simulation that states the input temperature of the data sorter circuit. The researcher used temperatures from 0



up to 250 with an increment of 10 degrees per simulation. Each simulation provides the output of the sorting, with a graph that shows distortion if temperature reaches 240 degrees C as shown in Figure 6-25.

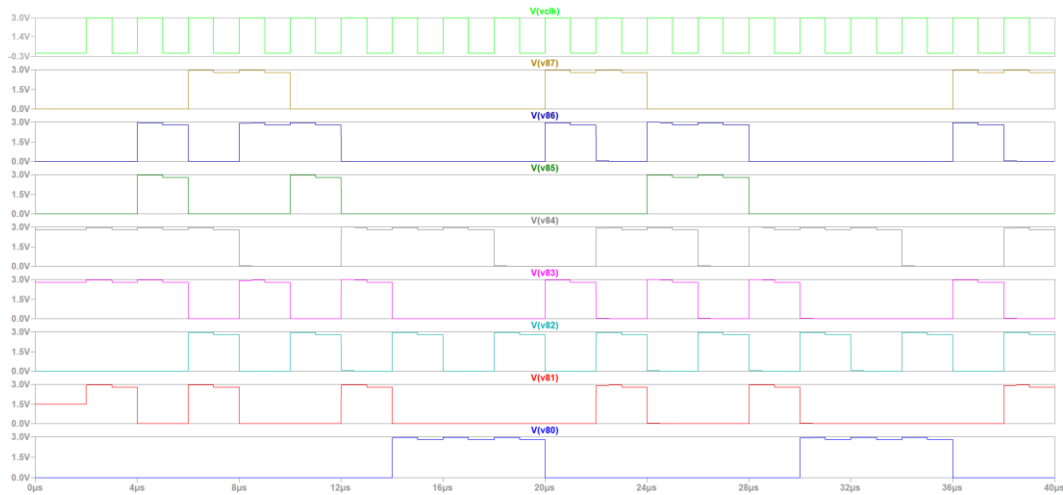


Figure 6- 25. Sorted Data of the Least Value, with temperature = 240 deg C

A detailed explanation is shown in Table 6-1 regarding the specific outcomes for each of the objectives of the study.

TABLE 6- 4 . SUMMARY OF RESULTS FOR ACHIEVING THE OBJECTIVES

Objectives	Results	Locations
GO: To design CMOS Based reconfigurable data panning sorter using the 0.35um library	Using two sets of data, the digital data inserted in the inputs were successfully sorted from lowest to highest, as shown in waveforms presented using minimal hardware implementations and power consumption. Each process corner library was tested and shown no signs of distortions in the waveforms of the sorted digital data for each of the simulations.	Sec. 6.1
SO1: To design a CMOS Based Panning Sorter circuit that can accept up to 8 inputs	A data-panning sorter was designed and simulated using LTSPICE Tool and had working results in terms of sorting the data. It accepted maximum inputs of 8 and 8 bits for its composition.	Sec. 6.1



SO2: To make the sorter reconfigurable by making the inputs variable from minimum of 2 to maximum of 8	A reconfigurable circuit was designed and simulated using AND Gate logic to signify the number of inputs to be sorted. If an input is turned off, it will include the data in the sorting process.	Sec. 6.2
SO3: To design the Geometric Layout of the Sorter Circuit	Tanner Tools L-EDIT was used in designing the geometric layout of the reconfigurable data panning sorter with the 0.35um library and no found DRC errors.	Sec. 6.3
SO4: To evaluate the performance of the sorter circuit using each of the 5 corner libraries	No distortions have been seen in the waveform for each of the five process corner libraries.	Sec. 5.1 on p. 31
SO5: To characterize the circuit designed in terms of area, power consumption, temperature, and frequency	The total area is approximately 1.36 mm ² while the power consumption is 555 mW, frequency of 20 kHz, and temperature tolerance of 240 degrees Celsius.	Sec. 5.1 on p. 31

6.6 Summary

This chapter provides the key results per objectives of the study. A CMOS based reconfigurable data panning algorithm sorter was designed using Tanner Tools, evaluated and simulated using LTSPICE and showed the effectivity of the data sorting process. It was also characterized in terms of power consumption, area, frequency, and temperature. Furthermore, it was evaluated using the five process corner libraries of CMOS.



Chapter 7

CONCLUSIONS, RECOMMENDATIONS, AND FUTURE DIRECTIVES



7.1 Concluding Remarks

In this study, a reconfigurable 8 input, 8 bit data panning algorithm based sorter was designed, characterized, and presented. Using a 0.35um CMOS Technology library with an external voltage supply of 3V in the main circuit, the data sorter was able to arrange the inputs from lowest to highest. Two trials have been made to verify the effectivity of the data sorter circuit. The software Tanner Tools was used for the designing of the CMOS circuit and LTSpice was used for the simulation. There is also an additional feature for the circuit to be reconfigurable in which there are extra select lines for the user to select how many inputs to be sorted. The data was sorted after a total of 8 clock cycles based on the graphs presented. A simulation was also created using Microsoft Excel to verify the correctness of the algorithm presented by Pedroni and to check the progress of data sorting, and as expected in the final graphical representations, it shows exactly the data outputs of the simulation per clock cycle. The final data outputs can be seen after 8 clock cycles. The cycle and algorithm stops after the data has been sorted. The first sorted output can be seen during the 18 μ s-20 μ s period, and remained after the sorted data has been generated. The data sorter is also tested using the five corner libraries namely: typical-typical, fast-fast, slow-fast, fast-slow, and slow-slow. Each library shows almost exactly the same graphical output representation for each sorted data.



The size, power consumption, and frequency were also presented in the paper. It has a total size of 1.36 mm^2 for its Area while the total power consumption reads 555 mW for all of the data outputs and a frequency of 2 μS for the clock circuit. A total of 77 inputs are connected to the circuit comprising of: 8 inputs with 8 bits (64 inputs), 8 inputs for select lines, 1 supply voltage, 1 ground, 1 load voltage for the register, 1 preset voltage for the single bit circular shift register, and 1 clock voltage. As for the output, there are 64 pins, similar to the input pins of the data sorter. The simulation satisfied any inputs that has been inserted to the circuit, and had an output equal to the expected sorted data.

7.2 Contributions

The interrelated contributions and supplements that have been developed by the author(s) in this Thesis are listed as follows. Only those that are unique to the authors' work are included.

- the CMOS design of a Reconfigurable Data Panning Algorithm Based Sorter was designed, simulated, and characterized;
- the Tanner Tools were used in creating the layout of the entire sorter;
- the Reconfigurable System was designed and inserted in the Panning Sorter;



7.3 Recommendations

To further improve the data sorting process, another type of data sorting algorithm maybe developed in the future. Other forms of software may also be considered for improved testing and simulation simplicity. It would also be beneficial for the circuit if the number of pins can be minimized as well as the total operating size of the circuit by using another algorithm that accepts serial instead of parallel inputs.

7.4 Future Prospects

There are several prospects that may be extended for further studies which can further improve the sorting process. So the suggested topics are listed in the following.

1. the design of another Data Sorting Algorithm with even lower number of hardware requirements
2. the usage of other CAD Tools for layout design
3. the reduction of the number of pins through another algorithm design



REFERENCES

- [1] V. A. Pedroni, R. P. Jasinski, and R. U. Pedroni, "Panning sorter: An approach to the design of minimal-hardware parallel-input data sorters," *Electronics Letters*, vol. 46, no. 18, 2010, doi: 10.1049/el.2010.1568.
- [2] V. A. Pedroni, R. P. Jasinski, and R. U. Pedroni, "Panning sorter: A minimal-size architecture for hardware implementation of 2D data sorting coprocessors," 2010. doi: 10.1109/APCCAS.2010.5775035.
- [3] E. A. Elsayed and K. Kise, "High-performance and hardware-efficient odd-even based merge sorter," *IEICE Transactions on Information and Systems*, vol. E103D, no. 12, 2020, doi: 10.1587/transinf.2020PAP0017.
- [4] R. Wankar and R. Akerkar, "Reconfigurable architectures and algorithms: A research survey," *International Journal of Computer Science and Applications*, vol. 6, no. 1, 2009.
- [5] H. Kareemullah, N. Janakiraman, and P. Nirmal Kumar, "A survey on embedded reconfigurable architectures," in *Proceedings of the 2017 IEEE International Conference on Communication and Signal Processing, ICCSP 2017*, 2018, vol. 2018-January. doi: 10.1109/ICCSP.2017.8286636.
- [6] K. Tiri and I. Verbauwhede, "A digital design flow for secure integrated circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 25, no. 7, 2006, doi: 10.1109/TCAD.2005.855939.
- [7] G. J. García, C. A. Jara, J. Pomares, A. Alabdo, L. M. Poggi, and F. Torres, "A survey on FPGA-based sensor systems: Towards intelligent and reconfigurable low-power sensors for computer vision, control and signal processing," *Sensors (Switzerland)*, vol. 14, no. 4, 2014. doi: 10.3390/s140406247.
- [8] A. P. Dhande, S. S. Narkhede, and S. S. Dudam, "VLSI implementation of ternary gates using Tanner Tool," 2014. doi: 10.1109/ICDCSyst.2014.6926187.
- [9] L. S, "A REVIEW OF LOW VOLTAGE AND LOW POWER CMOS ADDERS USING VLSI DESIGN IN VERILOG/VHDL," *International Journal of Engineering Applied Sciences and Technology*, vol. 6, no. 3, 2021, doi: 10.33564/ijeast.2021.v06i03.044.
- [10] D. Rasaily, R. Mehra, and U. Rai, "CMOS design of 5-stage ring oscillator as temperature sensor," 2016. doi: 10.1109/ICEEOT.2016.7755119.
- [11] D. S. Rashmi, R. S. Rukhsar, H. R. Shilpa, C. R. Vidyashree, K. D. Shinde, and H. v. Nithin, "Modeling of adders using CMOS and GDI logic for multiplier



- applications: A VLSI based approach," 2016. doi: 10.1109/ICCPCT.2016.7530213.
- [12] A. Chua *et al.*, "Driving philippine microelectronics education development with multi-university collaboration," 2013. doi: 10.1109/MSE.2013.6566703.
- [13] M. Bozanic and S. Sinha, "A Survey of Current Trends in Master's Programs in Microelectronics," *IEEE Transactions on Education*, vol. 61, no. 2, 2018, doi: 10.1109/TE.2017.2778697.
- [14] S. K. Saha, "Emerging Business Trends in the Microelectronics Industry," *Open Journal of Business and Management*, vol. 04, no. 01, 2016, doi: 10.4236/ojbm.2016.41012.
- [15] R. Zimmermann and W. Fichtner, "Low-power logic styles: CMOS versus pass-transistor logic," *IEEE Journal of Solid-State Circuits*, vol. 32, no. 7, 1997, doi: 10.1109/4.597298.
- [16] B. V. B. M. Prabhu, "Design of Low Power Novel Viterbi Decoder Using Multiple Threshold CMOS Logic," *International Journal of Science and Research (IJSR)*, vol. 3, no. 2, 2014.
- [17] A. A. Beltran, K. C. Nones, R. L. M. Salanguit, J. B. D. Santos, J. R. G. Santos, and K. J. T. Dizon, "Low power NAND gate-based half and full adder / subtractor using CMOS technique," *Journal of Robotics and Control (JRC)*, vol. 2, no. 4, 2021, doi: 10.18196/jrc.2487.
- [18] L. E. Winslow and Y. C. Chow, "The Analysis and Design of Some New Sorting Machines," *IEEE Transactions on Computers*, vol. C-32, no. 7, 1983, doi: 10.1109/TC.1983.1676299.
- [19] W. Dou, Y. Li, and S. Miao, "A data partitioning method for parallel digital terrain analysis," *Journal of Algorithms and Computational Technology*, vol. 9, no. 3, 2015, doi: 10.1260/1748-3018.9.3.251.
- [20] Y. Zhang and S. Q. Zheng, "Efficient parallel VLSI sorting architecture," *VLSI Design*, vol. 11, no. 2, 2000, doi: 10.1155/2000/14617.
- [21] S. Horiguchi, "Hybrid systolic sorters," *Parallel Computing*, vol. 17, no. 9, 1991, doi: 10.1016/S0167-8191(05)80044-4.
- [22] Y. H. Choi and M. Malek, "A Fault-Tolerant Systolic Sorter," *IEEE Transactions on Computers*, vol. 37, no. 5, 1988, doi: 10.1109/12.4615.
- [23] F. S. Wong and M. R. Ito, "PARALLEL SORTING ON A RE-CIRCULATING SYSTOLIC SORTER," *Computer Journal*, vol. 27, no. 3, 1984, doi: 10.1093/comjnl/27.3.260.
- [24] B. Parhami and D. M. Kwai, "Data-driven control scheme for linear arrays: Application to a stable insertion sorter," *IEEE Transactions on Parallel and Distributed Systems*, vol. 10, no. 1, 1999, doi: 10.1109/71.744832.
- [25] R. Perez-Andrade, R. Cumplido, C. Feregrino-Urbe, and F. Martin Del Campo, "A versatile linear insertion sorter based on an FIFO scheme,"



- Microelectronics Journal*, vol. 40, no. 12, 2009, doi: 10.1016/j.mejo.2009.08.006.
- [26] R. Perez-Andrade, R. Cumplido, C. Feregrino-Urbe, and F. M. del Campo, "A versatile hardware architecture for a constant false alarm rate processor based on a linear insertion sorter," *Digital Signal Processing: A Review Journal*, vol. 20, no. 6, 2010, doi: 10.1016/j.dsp.2010.02.001.
- [27] D. P. Yousefian Barfeh, J. G. Ebron, and J. P. Pabico, "Insertion Sorter in P Systems," in *Journal of Physics: Conference Series*, 2018, vol. 976, no. 1. doi: 10.1088/1742-6596/976/1/012004.
- [28] B. Atila and B. Kelleci, "Recursive odd-even sorter for vector quantizer," 2017. doi: 10.1109/MOCAST.2017.7937665.
- [29] E. A. Elsayed and K. Kise, "Towards an efficient hardware architecture for odd-even based merge sorter," 2019. doi: 10.1109/MCSoC.2019.00043.
- [30] T. Nakatani, S. T. Huang, B. W. Arden, and S. K. Tripathi, "K-Way Bitonic Sort," *IEEE Transactions on Computers*, vol. 38, no. 2, 1989, doi: 10.1109/12.16506.
- [31] T. H. Lee and J. J. Chou, "Some topological properties of bitonic sorters," *IEEE Transactions on Computers*, vol. 47, no. 9, 1998, doi: 10.1109/12.713317.
- [32] F. R. Beyette, P. A. Mitkas, S. A. Feld, and C. W. Wilmsen, "Bitonic sorting using an optoelectronic recirculating architecture," *Applied Optics*, vol. 33, no. 35, 1994, doi: 10.1364/ao.33.008164.
- [33] R. Cypher and J. L. C. Sanz, "Cubesort: A parallel algorithm for sorting N data items with S-sorters," *Journal of Algorithms*, vol. 13, no. 2, 1992, doi: 10.1016/0196-6774(92)90016-6.
- [34] W. Song, D. Koch, M. Lujan, and J. Garside, "Parallel Hardware Merge Sorter," 2016. doi: 10.1109/FCCM.2016.34.
- [35] D. Li, L. Huang, T. Gao, Y. Feng, A. Tavares, and K. Wang, "An Extended Nonstrict Partially Ordered Set-Based Configurable Linear Sorter on FPGAs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 5, 2020, doi: 10.1109/TCAD.2020.2977074.
- [36] A. Norollah, Z. Kazemi, H. Beitollahi, and D. Hely, "Hardware Support for Efficient and Low-power data Sorting in Massive Data Application: The 3D Sorting Method," *IEEE Consumer Electronics Magazine*, 2021, doi: 10.1109/MCE.2021.3076979.
- [37] Y. Taur, "CMOS design near the limit of scaling," *IBM Journal of Research and Development*, vol. 46, no. 2-3, 2002, doi: 10.1147/rd.462.0213.
- [38] L. Ding *et al.*, "CMOS-based carbon nanotube pass-transistor logic integrated circuits," *Nature Communications*, vol. 3, 2012, doi: 10.1038/ncomms1682.
- [39] T. C. Huang *et al.*, "Pseudo-CMOS: A design style for low-cost and robust flexible electronics," *IEEE Transactions on Electron Devices*, vol. 58, no. 1, 2011, doi: 10.1109/TED.2010.2088127.



- [40] M. S. J. Steyaert, B. de Muer, P. Leroux, M. Borremans, and K. Mertens, "Low-voltage low-power CMOS-RF transceiver design," *IEEE Transactions on Microwave Theory and Techniques*, vol. 50, no. 1, 2002, doi: 10.1109/22.981281.
- [41] X. J. Li and Y. P. Zhang, "CMOS low noise amplifier design for microwave and mmWave applications," *Progress in Electromagnetics Research*, vol. 161, 2018. doi: 10.2528/PIER18012410.
- [42] E. V. V. Cambero, R. R. Munoz, I. R. S. Casella, A. S. Filho, H. X. de Araujo, and C. E. Capovilla, "CMOS Low Noise Amplifier Topology Using Compact Wu Folded Active Inductor," *IEEE Latin America Transactions*, vol. 15, no. 10, 2017, doi: 10.1109/TLA.2017.8071223.
- [43] W. M. H. W. M. Sharif *et al.*, "Hybrid memristor-CMOS implementation of logic gates design using LTSpice," *International Journal of Electrical and Computer Engineering*, vol. 11, no. 3, 2021, doi: 10.11591/ijece.v11i3.pp2003-2010.
- [44] D. M. Binkley, C. E. Hopper, S. D. Tucker, B. C. Moss, J. M. Rochelle, and D. P. Foty, "A CAD methodology for optimizing transistor current and sizing in analog CMOS design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 22, no. 2, 2003, doi: 10.1109/TCAD.2002.806606.
- [45] M. Fujishima, S. Amakawa, K. Takano, K. Katayama, and T. Yoshida, "Terahertz CMOS design for low-power and high-speed wireless communication," *IEICE Transactions on Electronics*, vol. E98C, no. 12, 2015, doi: 10.1587/transele.E98.C.1091.
- [46] F. Daryabari and A. Nabavi, "Design of an adjustable active CMOS true time delay circuit for wideband beamforming applications," 2017. doi: 10.1109/IranianCEE.2017.7985476.
- [47] A. Olmos, F. Ferreira, F. P. Cortes, F. Chavez, and M. S. Lubaszewski, "A 2-transistor sub-1V low power temperature compensated CMOS voltage reference: Design and application," *Journal of Integrated Circuits and Systems*, vol. 10, no. 2, 2015, doi: 10.29292/jics.v10i2.408.
- [48] D. Colombo, C. Fayomi, F. Nabki, L. F. Ferreira, G. Wirth, and S. Bampi, "A design methodology using the inversion coefficient for low-voltage low-power CMOS voltage references," *Journal of Integrated Circuits and Systems*, vol. 6, no. 1, 2011, doi: 10.29292/jics.v6i1.333.
- [49] S. A. Z. Murad, A. Harun, M. N. M. Isa, S. N. Mohyar, R. Sapawi, and J. Karim, "Design of CMOS low-dropout voltage regulator for power management integrated circuit in 0.18- μm technology," in *AIP Conference Proceedings*, 2020, vol. 2203. doi: 10.1063/1.5142098.



- [50] C. N. Shilpa, K. D. Shinde, and H. v. Nithin, "Design, Implementation and Comparative Analysis of Kogge Stone Adder Using CMOS and GDI Design: A VLSI Based Approach," 2017. doi: 10.1109/CICN.2016.117.
- [51] V. Y. Stenin, A. v. Antonyuk, P. v. Stepanov, and Y. v. Katunin, "Design of logical elements for the 65-nm CMOS translation lookaside buffer with compensation of single events effects," 2017. doi: 10.1109/SIBCON.2017.7998513.
- [52] M. H. Ghazizadeh, F. Daryabari, and A. Medi, "CMOS integrated delay chain for X-Ku band applications," *Analog Integrated Circuits and Signal Processing*, vol. 102, no. 1, 2020, doi: 10.1007/s10470-019-01569-w.
- [53] H. Akhamal, M. Chakir, I. Lokman, H. Ameziane, and H. Qjidaa, "CMOS design of a boost-buck regulator with high efficiency for a wireless sensor network deployed for large aircraft in-flight tests," in *International Conference on Multimedia Computing and Systems -Proceedings*, 2017, vol. 0. doi: 10.1109/ICMCS.2016.7905618.
- [54] C. Piguet, "Low-power and low-voltage CMOS digital design," *Microelectronic Engineering*, vol. 39, no. 1-4, 1997, doi: 10.1016/S0167-9317(97)00175-5.
- [55] . H. R., "POWER ANALYSIS OF 4T SRAM BY STACKING TECHNIQUE USING TANNER TOOL," *International Journal of Research in Engineering and Technology*, vol. 03, no. 07, 2014, doi: 10.15623/ijret.2014.0307028.
- [56] S. Nagaraj, G. M. S. Reddy, and S. A. Mastani, "Analysis of MAC unit for conventional CMOS and low power CPL & DPL logics," *Journal of Critical Reviews*, vol. 7, no. 6. 2020. doi: 10.31838/jcr.07.06.34.
- [57] R. Bhanuchandar, B. Harikrishna, and S. Mishra, "10T sram cell design using single ended decoupled read bit line," *Indian Journal of Public Health Research and Development*, vol. 9, no. 11, 2018, doi: 10.5958/0976-5506.2018.01757.6.
- [58] R. Anitha and B. Vijayalakshmi, "Energy-efficient arithmetic & logic unit in quantum cellular automata & cmos technology," *Journal of Green Engineering*, vol. 10, no. 7, 2020.
- [59] S. Syed Ameer Abbas, P. Vishnoopriyaa, and A. Vinitha, "Realization of analog circuits in near field communication process for better throughput performance," *International Journal of Advanced Science and Technology*, vol. 29, no. 7 Special Issue, 2020.
- [60] H. Y. Huang and H. F. Lee, "A digitally programmable rail-to-rail CMOS operational amplifier as reusable silicon IP," *Tamkang Journal of Science and Engineering*, vol. 12, no. 4, 2009.
- [61] C. Y. Chen, J. H. Wang, and T. P. Sun, "A novel mini-LVDS receiver in 0.35-um CMOS," 2006. doi: 10.1109/SOCC.2006.283845.



- [62] F. K. Jérôme *et al.*, "A 0.35 μ m Low-Noise Stable Charge Sensitive Amplifier for Silicon Detectors Applications," *Informacije MIDE*M, vol. 50, no. 1, 2020, doi: 10.33180/InfMIDEM2020.101.
- [63] B. Miodrag, J. Radic, K. Nikolic, and M. Damnjanovic, "A temperature compensated MOSFET-C band-pass filter in 0.35 μ m CMOS technology," 2020. doi: 10.1109/DDECS50862.2020.9095661.
- [64] R. Cellier and P. Fiedorow, "A 0.35 μ m CMOS operational amplifier using multi-path frequency compensation," *Journal of Low Power Electronics*, vol. 11, no. 1, 2015, doi: 10.1166/jolpe.2015.1367.
- [65] Y.-J. Han and H.-J. Song, "Integrated Circuit of a Peak Detector for Flyback Converter using a 0.35 μ m CMOS Process," *Journal of the Korea Academia-Industrial cooperation Society*, vol. 17, no. 7, 2016, doi: 10.5762/kais.2016.17.7.42.
- [66] M. U. Mian, J. O. Dennis, M. H. M. Khir, T. B. Tang, and N. Y. Sutri, "Noise and Bandwidth analysis for PMOS based wide swing cascode current mirror using 0.35 μ m CMOS-MEMS technology," 2014. doi: 10.1109/ICIAS.2014.6869546.
- [67] X. Hu, A. S. Abraham, J. A. C. Incorvia, and J. S. Friedman, "Hybrid Pass Transistor Logic with Ambipolar Transistors," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 1, 2021, doi: 10.1109/TCSI.2020.3034042.
- [68] J. R. Rusli, S. Shafie, R. M. Sidek, H. A. Majid, W. Z. Wan Hassan, and M. A. Mustafa, "Optimized low voltage low power dynamic comparator robust to process, voltage and temperature variation," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 17, no. 2, 2019, doi: 10.11591/ijeecs.v17.i2.pp783-792.
- [69] S. Etezadi and S. A. Hosseini, "Novel ternary logic gates design in nanoelectronics," *Advances in Electrical and Electronic Engineering*, vol. 17, no. 3, 2019, doi: 10.15598/aeee.v17i3.3156.
- [70] K. Singh, S. C. Tiwari, and M. Gupta, "A modified implementation of tristate inverter based static master-slave flip-flop with improved power-delay-area product," *The Scientific World Journal*, vol. 2014, 2014, doi: 10.1155/2014/453675.
- [71] J. F. Lin, Z. J. Hong, C. M. Tsai, B. C. Wu, and S. W. Yu, "Novel low-complexity and low-power flip-flop design," *Electronics (Switzerland)*, vol. 9, no. 5, 2020, doi: 10.3390/electronics9050783.
- [72] S. L. Harris and D. Harris, "Sequential Logic Design," in *Digital Design and Computer Architecture*, 2022. doi: 10.1016/b978-0-12-820064-3.00003-9.
- [73] M. D. C. M. Morris Mano, *Digital Design With an Introduction to the Verilog HDL*. 2021.



- [74] F. Ntouskas, C. Efstathiou, and K. Pekmestzi, "Efficient design of magnitude and 2's complement comparators," *Integration*, vol. 71. 2020. doi: 10.1016/j.vlsi.2019.11.015.
- [75] "What is a CMOS : Working Principle & Its Applications," <https://www.elprocus.com/cmos-working-principle-and-applications/>.
- [76] U. A. Müller, S. Tanner, and P. A. Farine, "Low-noise amplifier with sample&hold for high-density stimulation and recording of neural signals," 2011. doi: 10.1109/NEWCAS.2011.5981268.

Produced: August 15, 2022, 12:45PM



Appendix A **STUDE RESEARCH ETHICS CLEARA**

SPICE NETLIST



```
* SUPER FINAL

.lib 'C:\Users\astol\Downloads\mm0355v.1'ff
* tt, ff, sf, fs, ss
.global vdd gnd vc1k

*CAS circuit
.subckt cas voutswapmax0 voutswapmax1 voutswapmax2 voutswapmax3 voutswapmax4 voutswapmax5 voutswapmax6 voutswapmax7
+ voutswapmin0 voutswapmin1 voutswapmin2 voutswapmin3 voutswapmin4 voutswapmin5 voutswapmin6 voutswapmin7
+ va0 va1 va2 va3 va4 va5 va6 va7 vb0 vb1 vb2 vb3 vb4 vb5 vb6 vb7 vs
*INVERTER
.subckt inv vout vin
m1 vout vin gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m2 vout vin vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
*INVERTER
*NOR 2 INPUT
.subckt nor2input vout vin1 vin2
m1 vout vin1 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m2 va vin1 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m3 vout vin2 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m4 vout vin2 va vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
*NOR 2 INPUT
*NOR 8 INPUT
.subckt nor8input vout vin1 vin2 vin3 vin4 vin5 vin6 vin7 vin8
m1 vout vin1 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m2 va vin1 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m3 vout vin2 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m4 vb vin2 va vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m5 vout vin3 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m6 vc vin3 vb vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m7 vout vin4 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m8 vd vin4 vc vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m9 vout vin5 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m10 ve vin5 vd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m11 vout vin6 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m12 vf vin6 ve vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m13 vout vin7 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m14 vg vin7 vf vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m15 vout vin8 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m16 vout vin8 vg vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
*NOR 8 INPUT
*FULLADDER
.subckt fulladder vsum vcout va vb vcin
```



```
.subckt nand vout vin1 vin2
m1 vout vin1 va gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m2 vout vin1 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m3 va vin2 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m4 vout vin2 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
x1 vfa3 va vb nand
x2 vfa5 va vfa3 nand
x3 vfa4 vfa3 vb nand
x4 vfa6 vfa5 vfa4 nand
x5 vfa7 vfa6 vcin nand
x6 vfa8 vfa6 vfa7 nand
x7 vfa9 vfa7 vcin nand
x8 vcout vfa7 vfa3 nand
x9 vsum vfa8 vfa9 nand
.ends
*FULLADDER
*MAGNITUDE COMPARATOR USING BPS
.subckt comparator vfout va0 va1 va2 va3 va4 va5 va6 va7 vb0 vb1 vb2 vb3 vb4 vb5 vb6 vb7
x1 vbi0 vb0 inv
x2 vbi1 vb1 inv
x3 vbi2 vb2 inv
x4 vbi3 vb3 inv
x5 vbi4 vb4 inv
x6 vbi5 vb5 inv
x7 vbi6 vb6 inv
x8 vbi7 vb7 inv
x9 vsum0 vcout0 va0 vbi0 vcin0 fulladder
x10 vsum1 vcout1 va1 vbi1 vcout0 fulladder
x11 vsum2 vcout2 va2 vbi2 vcout1 fulladder
x12 vsum3 vcout3 va3 vbi3 vcout2 fulladder
x13 vsum4 vcout4 va4 vbi4 vcout3 fulladder
x14 vsum5 vcout5 va5 vbi5 vcout4 fulladder
x15 vsum6 vcout6 va6 vbi6 vcout5 fulladder
x16 vsum7 vcout7 va7 vbi7 vcout6 fulladder
x17 vout vsum0 vsum1 vsum2 vsum3 vsum4 vsum5 vsum6 vsum7 nor8input
x18 vcomp vcout7 inv
x19 vfout vout vcomp nor2input
vcin0 vcin0 gnd dc 3V
.ends
*MAGNITUDE COMPARATOR USING BPS
*8 INPUT 2x1 MUX
.subckt mux8in vout0 vout1 vout2 vout3 vout4 vout5 vout6 vout7 va0 va1 va2 va3 va4 va5 va6 va7 vb0 vb1 vb2 vb3 vb4 vb5 vb6 vb7 vs
.subckt mux voutmux va vb vs
.subckt nand vout vin1 vin2
m1 vout vin1 va gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
```



```
m2 vout vin1 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m3 va vin2 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m4 vout vin2 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
x1 vn1 vs vs nand
x2 vn2 va vs nand
x3 vn3 vn1 vb nand
x4 voutmux vn3 vn2 nand
.ends
x1 vout0 va0 vb0 vs mux
x2 vout1 va1 vb1 vs mux
x3 vout2 va2 vb2 vs mux
x4 vout3 va3 vb3 vs mux
x5 vout4 va4 vb4 vs mux
x6 vout5 va5 vb5 vs mux
x7 vout6 va6 vb6 vs mux
x8 vout7 va7 vb7 vs mux
.ends
*8 INPUT 2x1 MUX
x1 vfout va0 va1 va2 va3 va4 va5 va6 va7 vb0 vb1 vb2 vb3 vb4 vb5 vb6 vb7 comparator
x2 vq vfout inv
x3 vw vs inv
x4 vnor vq vw nor2input
x5 voutswapmax0 voutswapmax1 voutswapmax2 voutswapmax3 voutswapmax4 voutswapmax5 voutswapmax6 voutswapmax7
+ va0 va1 va2 va3 va4 va5 va6 va7 vb0 vb1 vb2 vb3 vb4 vb5 vb6 vb7 vnor mux8in
x6 voutswapmin0 voutswapmin1 voutswapmin2 voutswapmin3 voutswapmin4 voutswapmin5 voutswapmin6 voutswapmin7
+ vb0 vb1 vb2 vb3 vb4 vb5 vb6 vb7 va0 va1 va2 va3 va4 va5 va6 va7 vnor mux8in
.ends
*CAS circuit

*MUX WITH DFLIP/ INPUT REGISTER
.subckt reg vout0 vout1 vout2 vout3 vout4 vout5 vout6 vout7 va0 va1 va2 va3 va4 va5 va6 va7 vb0 vb1 vb2 vb3 vb4 vb5 vb6 vb7 vs vc1k
.subckt mux8bit vout0 vout1 vout2 vout3 vout4 vout5 vout6 vout7 va0 va1 va2 va3 va4 va5 va6 va7 vb0 vb1 vb2 vb3 vb4 vb5 vb6 vb7 vs
.subckt mux voutmux va vb vs
.subckt nand vout vin1 vin2
m1 vout vin1 va gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m2 vout vin1 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m3 va vin2 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m4 vout vin2 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
x1 vn1 vs vs nand
x2 vn2 va vs nand
x3 vn3 vn1 vb nand
x4 voutmux vn3 vn2 nand
.ends
```



```
x1 vout0 va0 vb0 vs mux
x2 vout1 va1 vb1 vs mux
x3 vout2 va2 vb2 vs mux
x4 vout3 va3 vb3 vs mux
x5 vout4 va4 vb4 vs mux
x6 vout5 va5 vb5 vs mux
x7 vout6 va6 vb6 vs mux
x8 vout7 va7 vb7 vs mux
.ends
.subckt dflip8bit voflip0 voflip1 voflip2 voflip3 voflip4 voflip5 voflip6 voflip7
+ vinflip0 vinflip1 vinflip2 vinflip3 vinflip4 vinflip5 vinflip6 vinflip7 vclk
.subckt dflip vna6 vna1 vclk
.subckt nand vout vin1 vin2
m1 vout vin1 va gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m2 vout vin1 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m3 va vin2 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m4 vout vin2 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
x1 vna2 vna1 vna1 nand
x2 vna3 vna1 vclk nand
x3 vna4 vna2 vclk nand
x4 vna6 vna3 vna5 nand
x5 vna5 vna6 vna4 nand
.ends
x1 voflipnot0 vinflip0 vclknot dflip
x2 voflipnot1 vinflip1 vclknot dflip
x3 voflipnot2 vinflip2 vclknot dflip
x4 voflipnot3 vinflip3 vclknot dflip
x5 voflipnot4 vinflip4 vclknot dflip
x6 voflipnot5 vinflip5 vclknot dflip
x7 voflipnot6 vinflip6 vclknot dflip
x8 voflipnot7 vinflip7 vclknot dflip
x9 vclknot vclk inv
x10 voflip0 voflipnot0 vclk dflip
x11 voflip1 voflipnot1 vclk dflip
x12 voflip2 voflipnot2 vclk dflip
x13 voflip3 voflipnot3 vclk dflip
x14 voflip4 voflipnot4 vclk dflip
x15 voflip5 voflipnot5 vclk dflip
x16 voflip6 voflipnot6 vclk dflip
x17 voflip7 voflipnot7 vclk dflip
*INVERTER
.subckt inv vout vin
m1 vout vin gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m2 vout vin vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
```



```
*INVERTER
.ends
x1 vx0 vx1 vx2 vx3 vx4 vx5 vx6 vx7 va0 va1 va2 va3 va4 va5 va6 va7 vb0 vb1 vb2 vb3 vb4 vb5 vb6 vb7 vs mux8bit
x2 vout0 vout1 vout2 vout3 vout4 vout5 vout6 vout7 vx0 vx1 vx2 vx3 vx4 vx5 vx6 vx7 vclk dflip8bit
.ends
*MUX WITH DFLIP/ INPUT REGISTER

*CIRCULAR SINGLE BIT REGISTER
.subcckt circular vena1 vena3 vena5 vena7 vclk vpreset
.subcckt dfsingle vna1 vinflip vclk vpreset vreset
*nand
.subcckt nand vout vin1 vin2
m1 vout vin1 va gnd nch w=1u l=1u
m2 vout vin1 vdd vdd pch w=2u l=1u
m3 va vin2 gnd gnd nch w=1u l=1u
m4 vout vin2 vdd vdd pch w=2u l=1u
.ends
*nand3input
.subcckt nand3input vout vin1 vin2 vin3
m1 vout vin1 va gnd nch w=1u l=1u
m2 vout vin1 vdd vdd pch w=2u l=1u
m3 va vin2 vb gnd nch w=1u l=1u
m4 vout vin2 vdd vdd pch w=2u l=1u
m5 vb vin3 gnd gnd nch w=1u l=1u
m6 vout vin3 vdd vdd pch w=2u l=1u
.ends
*INVERTER
.subcckt inv vout vin
m1 vout vin gnd gnd nch w=1u l=1u
m2 vout vin vdd vdd pch w=2u l=1u
.ends
*INVERTER
x1 vclk1 vclk inv
x2 vpreset1 vpreset inv
x3 vreset1 vreset inv
x4 vq vinflip inv
x5 vw vinflip vclk1 nand
x6 ve vclk1 vq nand
x7 vr vpreset1 vw vt nand3input
x8 vt vreset1 ve vr nand3input
x9 vy vclk1 inv
x10 vu vr vy nand
x11 vp vy vt nand
x12 vna1 vpreset1 vu vna2 nand3input
x13 vna2 vreset1 vp vna1 nand3input
```



```
.ends
*INVERTER
.subckt inv vout vin
m1 vout vin gnd gnd nch w=1u l=1u
m2 vout vin vdd vdd pch w=2u l=1u
.ends
*INVERTER
.subckt dflip vna6 vna1 vclk
.subckt nand vout vin1 vin2
m1 vout vin1 va gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m2 vout vin1 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m3 va vin2 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m4 vout vin2 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
x1 vna2 vna1 vna1 nand
x2 vna3 vna1 vclk nand
x3 vna4 vna2 vclk nand
x4 vna6 vna3 vna5 nand
x5 vna5 vna6 vna4 nand
.ends
x1 vna0 vna0x vclk vpreset1 gnd dfsingle
x2 vna10 vna10x vclk gnd vpreset1 dfsingle
x3 vna2 vna2x vclk gnd vpreset1 dfsingle
x4 vna30 vna30x vclk gnd vpreset1 dfsingle
x5 vna4 vna4x vclk gnd vpreset1 dfsingle
x6 vna50 vna50x vclk gnd vpreset1 dfsingle
x7 vna6 vna6x vclk gnd vpreset1 dfsingle
x8 vna70 vna70x vclk gnd vpreset1 dfsingle
x9 vna1 vna10 inv
x10 vna3 vna30 inv
x11 vna5 vna50 inv
x12 vna7 vna70 inv
x13 vpreset1 vpreset inv
*x14 vreset1 vreset inv
x15 vclknot vclk inv
x16 vna0x vna70 vclknot dflip
x17 vna10x vna0 vclknot dflip
x18 vna2x vna10 vclknot dflip
x19 vna30x vna2 vclknot dflip
x20 vna4x vna30 vclknot dflip
x21 vna50x vna4 vclknot dflip
x22 vna6x vna50 vclknot dflip
x23 vna70x vna6 vclknot dflip
.ends
*CIRCULAR REGISTER
|
```



```
*AND
.subckt and vout vinput vselect
*INVERTER
.subckt inv vout vin
m1 vout vin gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m2 vout vin vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
*INVERTER
*NAND 2 INPUT
.subckt nand vout vin1 vin2
m1 vout vin1 va gnd nch w=1u l=1u
m2 vout vin1 vdd vdd pch w=2u l=1u
m3 va vin2 gnd gnd nch w=1u l=1u
m4 vout vin2 vdd vdd pch w=2u l=1u
.ends
*NAND 2 INPUT
x1 voutn vinput vselect nand
x2 vout voutn inv
.ends
*AND
*RECONFIGURABLE CIRCUIT

*INVERTER
.subckt inv vout vin
m1 vout vin gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m2 vout vin vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
*INVERTER

*STOPPER CIRCUIT
.subckt mux8in vout0 vout1 vout2 vout3 vout4 vout5 vout6 vout7 va0 va1 va2 va3 va4 va5 va6 va7 vb0 vb1 vb2 vb3 vb4 vb5 vb6 vb7 vs
.subckt mux voutmux va vb vs
.subckt nand vout vin1 vin2
m1 vout vin1 va gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m2 vout vin1 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
m3 va vin2 gnd gnd nch w=1u l=1u ad=1.25e-012 as=1.1e-012 pd=4.5e-006 ps=4.2e-006 $ (32.1 14.2 33.1 15.2)
m4 vout vin2 vdd vdd pch w=2u l=1u ad=4.8e-012 as=4.6e-012 pd=1.04e-005 ps=1.03e-005 $ (32.1 18.25 33.1 22.25)
.ends
x1 vn1 vs vs nand
x2 vn2 va vs nand
x3 vn3 vn1 vb nand
x4 voutmux vn3 vn2 nand
.ends
x1 vout0 va0 vb0 vs mux
x2 vout1 va1 vb1 vs mux
```



```
x3 vout2 va2 vb2 vs mux
x4 vout3 va3 vb3 vs mux
x5 vout4 va4 vb4 vs mux
x6 vout5 va5 vb5 vs mux
x7 vout6 va6 vb6 vs mux
x8 vout7 va7 vb7 vs mux
.ends
*STOPPER CIRCUIT
```

```
x111 via0 va0 vselect1 and
x211 via1 va1 vselect1 and
x311 via2 va2 vselect1 and
x411 via3 va3 vselect1 and
x511 via4 va4 vselect1 and
x611 via5 va5 vselect1 and
x711 via6 va6 vselect1 and
x811 via7 va7 vselect1 and
x911 vib0 vb0 vselect2 and
x1011 vib1 vb1 vselect2 and
x1111 vib2 vb2 vselect2 and
x1211 vib3 vb3 vselect2 and
x1311 vib4 vb4 vselect2 and
x1411 vib5 vb5 vselect2 and
x1511 vib6 vb6 vselect2 and
x1611 vib7 vb7 vselect2 and
x1711 vic0 vc0 vselect3 and
x1811 vic1 vc1 vselect3 and
x1911 vic2 vc2 vselect3 and
x2011 vic3 vc3 vselect3 and
x2111 vic4 vc4 vselect3 and
x2211 vic5 vc5 vselect3 and
x2311 vic6 vc6 vselect3 and
x2411 vic7 vc7 vselect3 and
x2511 vid0 vd0 vselect4 and
x2611 vid1 vd1 vselect4 and
x2711 vid2 vd2 vselect4 and
x2811 vid3 vd3 vselect4 and
x2911 vid4 vd4 vselect4 and
x3011 vid5 vd5 vselect4 and
x3111 vid6 vd6 vselect4 and
x3211 vid7 vd7 vselect4 and
x3311 vie0 ve0 vselect5 and
x3411 vie1 ve1 vselect5 and
x3511 vie2 ve2 vselect5 and
x3611 vie3 ve3 vselect5 and
```



```
x3711 vie4 ve4 vselect5 and
x3811 vie5 ve5 vselect5 and
x3911 vie6 ve6 vselect5 and
x4011 vie7 ve7 vselect5 and
x4111 vif0 vf0 vselect6 and
x4211 vif1 vf1 vselect6 and
x4311 vif2 vf2 vselect6 and
x4411 vif3 vf3 vselect6 and
x4511 vif4 vf4 vselect6 and
x4611 vif5 vf5 vselect6 and
x4711 vif6 vf6 vselect6 and
x4811 vif7 vf7 vselect6 and
x4911 vig0 vg0 vselect7 and
x5011 vig1 vg1 vselect7 and
x5111 vig2 vg2 vselect7 and
x5211 vig3 vg3 vselect7 and
x5311 vig4 vg4 vselect7 and
x5411 vig5 vg5 vselect7 and
x5511 vig6 vg6 vselect7 and
x5611 vig7 vg7 vselect7 and
x5711 vih0 vh0 vselect8 and
x5811 vih1 vh1 vselect8 and
x5911 vih2 vh2 vselect8 and
x6011 vih3 vh3 vselect8 and
x6111 vih4 vh4 vselect8 and
x6211 vih5 vh5 vselect8 and
x6311 vih6 vh6 vselect8 and
x6411 vih7 vh7 vselect8 and
```

```
*INVERTER FOR VLOAD, @VLOAD=5V, store inputs, then VLOAD=0V, SYNCHRONOUS STORAGE
x1 vload1 vload inv
```

```
*STORING REGISTERS
```

```
x2 vt80 vt81 vt82 vt83 vt84 vt85 vt86 vt87 via0 via1 via2 via3 via4 via5 via6 via7
+vcmax30 vcmax31 vcmax32 vcmax33 vcmax34 vcmax35 vcmax36 vcmax37 vload1 vclk reg
x3 vt70 vt71 vt72 vt73 vt74 vt75 vt76 vt77 vib0 vib1 vib2 vib3 vib4 vib5 vib6 vib7
+vcmin00 vcmin01 vcmin02 vcmin03 vcmin04 vcmin05 vcmin06 vcmin07 vload1 vclk reg
x4 vt60 vt61 vt62 vt63 vt64 vt65 vt66 vt67 vic0 vic1 vic2 vic3 vic4 vic5 vic6 vic7
+vcmax00 vcmax01 vcmax02 vcmax03 vcmax04 vcmax05 vcmax06 vcmax07 vload1 vclk reg
x5 vt50 vt51 vt52 vt53 vt54 vt55 vt56 vt57 vid0 vid1 vid2 vid3 vid4 vid5 vid6 vid7
+vcmin10 vcmin11 vcmin12 vcmin13 vcmin14 vcmin15 vcmin16 vcmin17 vload1 vclk reg
x6 vt40 vt41 vt42 vt43 vt44 vt45 vt46 vt47 vie0 vie1 vie2 vie3 vie4 vie5 vie6 vie7
+vcmax10 vcmax11 vcmax12 vcmax13 vcmax14 vcmax15 vcmax16 vcmax17 vload1 vclk reg
x7 vt30 vt31 vt32 vt33 vt34 vt35 vt36 vt37 vif0 vif1 vif2 vif3 vif4 vif5 vif6 vif7
+vcmin20 vcmin21 vcmin22 vcmin23 vcmin24 vcmin25 vcmin26 vcmin27 vload1 vclk reg
x8 vt20 vt21 vt22 vt23 vt24 vt25 vt26 vt27 vig0 vig1 vig2 vig3 vig4 vig5 vig6 vig7 |
```



```
+vcmax20 vcmax21 vcmax22 vcmax23 vcmax24 vcmax25 vcmax26 vcmax27 vload1 vclk reg
x9 vt10 vt11 vt12 vt13 vt14 vt15 vt16 vt17 vih0 vih1 vih2 vih3 vih4 vih5 vih6 vih7
+vcmin30 vcmin31 vcmin32 vcmin33 vcmin34 vcmin35 vcmin36 vcmin37 vload1 vclk reg

*COMPARE AND SWAP CIRCUITS
x10 vcmax00 vcmax01 vcmax02 vcmax03 vcmax04 vcmax05 vcmax06 vcmax07
+vcmin00 vcmin01 vcmin02 vcmin03 vcmin04 vcmin05 vcmin06 vcmin07
+v80 v81 v82 v83 v84 v85 v86 v87 v70 v71 v72 v73 v74 v75 v76 v77 vena1 cas
x11 vcmax10 vcmax11 vcmax12 vcmax13 vcmax14 vcmax15 vcmax16 vcmax17
+vcmin10 vcmin11 vcmin12 vcmin13 vcmin14 vcmin15 vcmin16 vcmin17
+v60 v61 v62 v63 v64 v65 v66 v67 v50 v51 v52 v53 v54 v55 v56 v57 vena3 cas
x12 vcmax20 vcmax21 vcmax22 vcmax23 vcmax24 vcmax25 vcmax26 vcmax27
+vcmin20 vcmin21 vcmin22 vcmin23 vcmin24 vcmin25 vcmin26 vcmin27
+v40 v41 v42 v43 v44 v45 v46 v47 v30 v31 v32 v33 v34 v35 v36 v37 vena5 cas
x13 vcmax30 vcmax31 vcmax32 vcmax33 vcmax34 vcmax35 vcmax36 vcmax37
+vcmin30 vcmin31 vcmin32 vcmin33 vcmin34 vcmin35 vcmin36 vcmin37
+v20 v21 v22 v23 v24 v25 v26 v27 v10 v11 v12 v13 v14 v15 v16 v17 vena7 cas

*CIRCULAR REGISTER for CONTROL OF CAS Circuits
x14 vena1 vena3 vena5 vena7 vclk vpreset circular

*STOPPER
x15 v80 v81 v82 v83 v84 v85 v86 v87 v80 v81 v82 v83 v84 v85 v86 v87 vt80 vt81 vt82 vt83 vt84 vt85 vt86 vt87 vload1 mux8in
x16 v70 v71 v72 v73 v74 v75 v76 v77 v70 v71 v72 v73 v74 v75 v76 v77 vt70 vt71 vt72 vt73 vt74 vt75 vt76 vt77 vload1 mux8in
x17 v60 v61 v62 v63 v64 v65 v66 v67 v60 v61 v62 v63 v64 v65 v66 v67 vt60 vt61 vt62 vt63 vt64 vt65 vt66 vt67 vload1 mux8in
x18 v50 v51 v52 v53 v54 v55 v56 v57 v50 v51 v52 v53 v54 v55 v56 v57 vt50 vt51 vt52 vt53 vt54 vt55 vt56 vt57 vload1 mux8in
x19 v40 v41 v42 v43 v44 v45 v46 v47 v40 v41 v42 v43 v44 v45 v46 v47 vt40 vt41 vt42 vt43 vt44 vt45 vt46 vt47 vload1 mux8in
x20 v30 v31 v32 v33 v34 v35 v36 v37 v30 v31 v32 v33 v34 v35 v36 v37 vt30 vt31 vt32 vt33 vt34 vt35 vt36 vt37 vload1 mux8in
x21 v20 v21 v22 v23 v24 v25 v26 v27 v20 v21 v22 v23 v24 v25 v26 v27 vt20 vt21 vt22 vt23 vt24 vt25 vt26 vt27 vload1 mux8in
x22 v10 v11 v12 v13 v14 v15 v16 v17 v10 v11 v12 v13 v14 v15 v16 v17 vt10 vt11 vt12 vt13 vt14 vt15 vt16 vt17 vload1 mux8in

*RECONFIGURABLE CIRCUIT
*SELECT INPUTS
*5V for ON, 0V for off.
*vselect1 for first input, vselect2 for 2nd input and so on..
vselect1 vselect1 gnd dc 3V
vselect2 vselect2 gnd dc 3V
vselect3 vselect3 gnd dc 3V
vselect4 vselect4 gnd dc 3V
vselect5 vselect5 gnd dc 3V
vselect6 vselect6 gnd dc 3V
vselect7 vselect7 gnd dc 3V
vselect8 vselect8 gnd dc 3V
*SELECT INPUTS
*RECONFIGURABLE CIRCUIT
|
```



*26decimal to binary 0001 1010

va0 va0 gnd dc 0V
va1 va1 gnd dc 3V
va2 va2 gnd dc 0V
va3 va3 gnd dc 3V
va4 va4 gnd dc 3V
va5 va5 gnd dc 0V
va6 va6 gnd dc 0V
va7 va7 gnd dc 0V

*100decimal to binary 0110 1010

vb0 vb0 gnd dc 0V
vb1 vb1 gnd dc 3V
vb2 vb2 gnd dc 0V
vb3 vb3 gnd dc 3V
vb4 vb4 gnd dc 0V
vb5 vb5 gnd dc 3V
vb6 vb6 gnd dc 3V
vb7 vb7 gnd dc 0V

*21decimal to binary 0001 0101

vc0 vc0 gnd dc 3V
vc1 vc1 gnd dc 0V
vc2 vc2 gnd dc 3V
vc3 vc3 gnd dc 0V
vc4 vc4 gnd dc 3V
vc5 vc5 gnd dc 0V
vc6 vc6 gnd dc 0V
vc7 vc7 gnd dc 0V

*200decimal to binary 1100 1000

vd0 vd0 gnd dc 0V
vd1 vd1 gnd dc 0V
vd2 vd2 gnd dc 0V
vd3 vd3 gnd dc 3V
vd4 vd4 gnd dc 0V
vd5 vd5 gnd dc 0V
vd6 vd6 gnd dc 3V
vd7 vd7 gnd dc 3V

*5decimal to binary 0000 0101

ve0 ve0 gnd dc 3V
ve1 ve1 gnd dc 0V
ve2 ve2 gnd dc 3V
ve3 ve3 gnd dc 0V
ve4 ve4 gnd dc 0V



```
ve5 ve5 gnd dc 0V
ve6 ve6 gnd dc 0V
ve7 ve7 gnd dc 0V

*150decimal to binary 1001 0110
vf0 vf0 gnd dc 0V
vf1 vf1 gnd dc 3V
vf2 vf2 gnd dc 3V
vf3 vf3 gnd dc 0V
vf4 vf4 gnd dc 3V
vf5 vf5 gnd dc 0V
vf6 vf6 gnd dc 0V
vf7 vf7 gnd dc 3V

*17decimal to binary 0001 0001
vg0 vg0 gnd dc 3V
vg1 vg1 gnd dc 0V
vg2 vg2 gnd dc 0V
vg3 vg3 gnd dc 0V
vg4 vg4 gnd dc 3V
vg5 vg5 gnd dc 0V
vg6 vg6 gnd dc 0V
vg7 vg7 gnd dc 0V

*120decimal to binary 0111 1000
vh0 vh0 gnd dc 0V
vh1 vh1 gnd dc 0V
vh2 vh2 gnd dc 0V
vh3 vh3 gnd dc 3V
vh4 vh4 gnd dc 3V
vh5 vh5 gnd dc 3V
vh6 vh6 gnd dc 3V
vh7 vh7 gnd dc 0V
*DATA INPUTS TO BE SORTED

vdd vdd gnd dc 3V
vload vload gnd pulse (0 3 3u 1p 1p 16u 40u)
vpreset vpreset gnd pulse (0 3 2u 1p 1p 800u 800u)
vclk vclk gnd pulse (0 3 2u 1p 1p 1u 2u)

.tran 0.1u 40u
.op
.prnt tran v(vclk) v(vena1) v(vena3) v(vena5) v(vena7) v(vload) v(vpreset)
```



De La Salle University

```
+ v(v10) v(v11) v(v12) v(v13) v(v14) v(v15) v(v16) v(v17)
+v(v20) v(v21) v(v22) v(v23) v(v24) v(v25) v(v26) v(v27)
+v(v30) v(v31) v(v32) v(v33) v(v34) v(v35) v(v36) v(v37)
+v(v40) v(v41) v(v42) v(v43) v(v44) v(v45) v(v46) v(v47)
+v(v50) v(v51) v(v52) v(v53) v(v54) v(v55) v(v56) v(v57)
+v(v60) v(v61) v(v62) v(v63) v(v64) v(v65) v(v66) v(v67)
+v(v70) v(v71) v(v72) v(v73) v(v74) v(v75) v(v76) v(v77)
+v(v80) v(v81) v(v82) v(v83) v(v84) v(v85) v(v86) v(v87)
.end
```



Appendix B **ANSWERS TO QUESTIONS TO THIS THESIS**

ANSWERS TO QUESTIONS TO THIS THESIS



B1 **How important is the problem to practice?**

The project would help the electronics industry in adding a CMOS based design of a data sorter that would also benefit systems that uses the application for a minimal hardware sizing and implementation as well as faster sorting responses and lower power consumption for electronics applications.

B2 **How will you know if the solution/s that you will achieve would be better than existing ones?**

The size of the design is lower compared to the other data sorting algorithms. With an application of the CMOS design, it will be much lower compared to the existing ones.

B2.1 **How will you measure the improvement/s?**

The sorting process itself will be the answer. If the inputs are sorted from smallest to largest using a minimal hardware compared to the other digital sorters, then it is a basis for measurement.

B2.1.1 **What is/are your basis/bases for the improvement/s?**

The number of hardware components which leads to lower power consumption and faster sorting process

B2.1.2 **Why did you choose that/those basis/bases?**



The emergence of technology has lead to complicated circuit implementations but designing the CMOS based data panning digital sorter caters the problem of having high hardware implementations.

B2.1.3 How significant are your measure/s of the improvement/s?

It shows lower number of hardware implementations in terms of its number of multiplexers, data registers, and comparators as well as having lower power consumption compared to the previous data sorter algorithms.

B3 What is the difference of the solution/s from existing ones?

There are no CMOS Designs of the Data Sorters presented as well as no reconfigurable circuits has been introduced. Applying the CMOS design will have a lower hardware implementation and lower power consumption.

B3.1 How is it different from previous and existing ones?

There are no CMOS designs of the existing data sorting algorithms.

B4 What are the assumptions made (that are behind for your proposed solution to work)?

No fabrication will be made, only up to the design of the data panning sorter.



B4.1 Will your proposed solution/s be sensitive to these assumptions?

Yes, because the main goal of the study is just to design a reconfigurable data-panning algorithm-based sorter.

B4.2 Can your proposed solution/s be applied to more general cases when some of the assumptions are eliminated? If so, how?

Yes, but it would be better for the data-sorter CMOS design to be fabricated so that it will be used in the market.

B5 What is the necessity of your approach / proposed solution/s?

The need for lower values of hardware implementation.

B5.1 What will be the limits of applicability of your proposed solution/s?

No fabrication will be made, and limited up to 8 inputs, 8 data bits.

B5.2 What will be the message of the proposed solution to technical people? How about to non-technical managers and businessmen?



This will be beneficial if fabricated for certain systems that uses digital data sorting, making their electronic systems faster and lower in hardware consumption.

B6 How will you know if your proposed solution/s is/are correct?

The digital data inputs were sorted from lowest to highest in the final output of the system.

B6.1 Will your results warrant the level of mathematics used (i.e., will the end justify the means)?

The results were done objectively therefore mathematics proves the engineering principles

B7 Is/are there an/_ alternative way/s to get to the same solution/s?

Another set of data-sorting algorithm may lead to a same solution.

B7.1 Can you come up with illustrating examples, or even better, counterexamples to your proposed solution/s?

For example, all of the data sorting algorithms are working in terms of sorting digital data. The only difference is the hardware implementation. If another algorithm is proposed, there is a chance for even lower hardware implementation.



B7.2 Is there an approximation that can arrive at the essentially the same proposed solution/s more easily?

Yes, by designing another algorithm for sorting digital data.

B8 If you were the examiner of your Thesis, how would you present the Thesis in another way? Give your remarks, especially for your methodology and the results and discussions.

If possible, the circuit implementation can be improved. The presented circuit implementations are generic in nature, but can be improved by using other digital logic design implementations.

B8.1 What are the weaknesses of your Thesis, specifically your methodology and the results and discussions?

A lot of design of circuit implementation can be applied to each of the hardware components of the data panning sorter which could further improve the number of the required hardware implementation. An additional fabrication is possible but would require expensive costing and further development.



Appendix C VITA

VITA



Anthony S. Tolentino received the B.Sc. degree in Electronics Engineering from De La Salle University, Manila, Philippines in 2012. He is currently taking up his M.Sc. Electronics and Communications Engineering studies at De La Salle University. His research interests include microelectronics, alternative energy systems, control systems, and educational technology.